B.Tech Computer Science & Engineering

Second Year • Third Semester • Course Code: 26CSEC305

Programming for Problem Solving – II

(C++)

Complete Zero-to-Mastery Notes,

Practical Guide & Exam Preparation

A comprehensive study & examination resource, prepared strictly around the official university syllabus (26CSEC305).

Ayush Jha

Batch 2CSE-4 • School of Computer Science & Engineering

IILM University, Greater Noida

Ayush Jha

Foreword & About This Book

Ye book **Programming for Problem Solving – II (C++)** course ke liye banaaya gaya ek **complete, self-contained study companion** hai — course code **26CSEC305**, jo IILM University, Greater Noida ke B.Tech Computer Science & Engineering programme ke third semester mein padaaya jaata hai. Poori book official university syllabus ke around hi banaayi gayi hai, aur syllabus mein likhe har topic ko detailed, beginner-friendly explanations, solved programs, comparison tables, revision material aur exam practice ke saath expand kiya gaya hai.

> **i IMPORTANT**
>
> **Syllabus authority ke baare mein note.** Ye notes official *Programming for Problem Solving – II (CPP)* syllabus ke around tayyaar kiye gaye comprehensive study material hain. Official course scope ke liye university ka syllabus hi authoritative source hai; is book mein us syllabus ke topics ko beginner-friendly explanations, examples, programs, revision material aur exam preparation ke saath detail mein samjhaaya gaya hai.

Is Book Ko Kaise Use Karein

Ye book is tarah design ki gayi hai ki jo student abhi C++, object-oriented programming, Standard Template Library, modern C++20 features, design patterns ya concurrent programming ke baare mein *bilkul kuch nahi jaanta*, wo pehle page se shuru karke sirf isi ek book se strong exam-level aur practical understanding tak pahunch sake.

Book ko padhne ka recommended tareeqa:

• **Sequence mein padho.** Har unit pehle waale units ke concepts par based hai. Aage skip kar doge to baad ka material unnecessarily mushkil lagega.

• **Code khud type karo.** Programs ko sirf mat padho — koi C++20 compiler kholo (GCC 11+, Clang 13+, MSVC 19.29+) aur har example ko physically type karo. Compile karo `-std=c++20` flag ke saath (ya MSVC par `/std:c++20`).

• **Practice sets zaroor attempt karo.** Har unit ke end mein practice questions hain, Level 1 (bahut easy) se Level 5 (application-oriented) tak.

• **Callout boxes ka use karo.** *Exam Tip*, *Important*, *Common Confusion*, *Remember* aur *One-Line Revision* boxes wahi material highlight karte hain jo university examinations mein sabse zyada poocha jaata hai.

• **Master Revision Guide ke saath finish karo.** Book ka last section poora syllabus ko ek one-hour, phir fifteen-minute, aur finally five-minute revision sheet mein compress kar deta hai.

Icon Legend

Icon	Meaning
★ EXAM TIP	Wo point jo university examinations mein baar-baar poocha jaata hai.
! WARNING	Common trap ya galti jo beginners aasaani se kar dete hain.
i IMPORTANT	Aisa concept jise samajhna baad ke topics ke liye zaroori hai.
◆ REMEMBER	Wo rule jise exactly yaad rakhna chahiye.
✎ EXAM POINT	Likely long-answer ya short-answer question.
≡ ONE-LINE REVISION	Wo ek sentence jo exam ke din yaad hona hi chahiye.
? COMMON CONFUSION	Do concepts jo students mix kar dete hain, unko alag-alag samjhaata hai.
★ MEMORY TRICK	Kisi tricky point ke liye mnemonic ya yaad rakhne ka tareeqa.

Ayush Jha

Difficulty Labels

Symbol	Level	Meaning
★	Beginner	Pehli reading ke baad bilkul easy.
★★	Basic	Standard textbook question.
★★★	Intermediate	Typical mid-semester question.
★★★★	Exam Level	End-semester papers mein frequently aata hai.
★★★★★	Advanced	Required scope se thoda zyada; depth chaahne waale students ke liye.

Ayush Jha

Table of Contents

FRONT MATTER

Foreword & About This Book 2

Is Book Ko Kaise Use Karein 2

Table of Contents 3

Your Zero-to-Master C++ Roadmap 7

PREREQUISITE FOUNDATION — C++ FROM ZERO

F.1 Programming kya hoti hai? 8

F.2 C++ kya hota hai? 8

F.3 C++ Program ka Structure 8

F.4 Variables, Data Types & Operators 9

F.5 Control Flow (Selection & Repetition) 10

F.6 Functions & Parameter Passing 11

F.7 Arrays & Strings 11

F.8 References & Pointers (essential minimum) 12

F.9 Classes, Objects, Constructors, Destructors 13

F.10 Inheritance & Basic Polymorphism 14

UNIT I — ADVANCED OBJECT-ORIENTED PROGRAMMING

1.1 Review of C++ Fundamentals and OOP Concepts 18

1.2 Runtime Polymorphism 22

1.3 Virtual Functions 25

1.4 Pure Virtual Functions 28

1.5 Abstract Classes 32

1.6 Function Overriding 35

1.7 Dynamic Binding 39

1.8 Virtual Destructors 42

1.9 Introduction to SOLID Principles 45

Unit I Revision Sheet 49

Ayush Jha

UNIT II — STL AND C++20 RANGES

2.1 STL kya hota hai? 53
2.2 STL Containers 55
2.3 Iterators 65
2.4 STL Algorithms 68
Unit II Revision Sheet 77

UNIT III — MODERN C++20 LANGUAGE FEATURES

3.1 Type Deduction (auto, decltype, decltype(auto)) 81
3.2 Compile-Time Programming (constexpr, consteval, constinit) 83
3.3 Modern Initialisation 85
3.4 Memory Management 87
3.5 Utility Library 92
3.6 Other C++20 Features 97
Unit III Revision Sheet 100

UNIT IV — OBJECT-ORIENTED DESIGN PATTERNS

4.1 Design Pattern kya hota hai? 105
4.2 Creational Design Patterns 106
4.3 Structural Design Patterns 115
4.4 Behavioural Design Patterns 122
Unit IV Revision Sheet 130

UNIT V — CONCURRENT PROGRAMMING & MODERN APPLICATIONS

5.1 Modern Concurrency 133
5.2 std::thread 135
5.3 std::jthread (C++20) 137
5.4 Mutex 139
5.5 std::lock_guard 140
5.6 std::scoped_lock 140
5.7 Atomic Operations 142
5.8 Futures and Async (Introduction) 143
5.9 Coroutines (Introduction) 145
5.10 Coroutine Basics 145
5.11 co_await 146
5.12 co_yield 147
5.13 co_return 147

Ayush Jha

5.14 Coroutines ke Applications **148**

Unit V Revision Sheet **149**

BACK MATTER

CPP Master Revision Guide **151**

Programming Practice Set (All Five Units) **167**

Complete Glossary of Terms **168**

Final Exam Checklist **174**

Closing Message **175**

Ayush Jha

Your Zero-to-Master C++ Roadmap

Neeche diya gaya roadmap batata hai ki is book ke concepts ko exactly kis order mein master karna chahiye. Har stage sirf apne se pehle waale stages ki knowledge assume karti hai, aur har stage official syllabus ke kisi specific unit ki taraf seedha le jaati hai.

Stage 0 — Absolute Zero

Aapko C++ ke baare mein kuch nahi aata. **Prerequisite Foundation** chapter ko dhyan se padho. Focus karo: program kya hota hai, `main()` ka structure, variables, input/output, if/else, loops, functions, arrays, strings, pointers, references, classes, objects, constructors, destructors, aur inheritance.

Stage 1 — Object-Oriented Thinking (Unit I)

Objects ke terms mein sochna seekho. Samjho kaise classes se objects bante hain, kaise inheritance hierarchies banata hai, aur kaise **runtime polymorphism** ek hi interface se kayi behaviours ko control karne deta hai. **Virtual functions**, **pure virtual functions**, **abstract classes**, **function overriding**, **dynamic binding** aur **virtual destructors** ko master karo. Unit ko **SOLID** principles ke saath close karo.

Stage 2 — The Standard Toolbox (Unit II)

Sab kuch haath se likhne ki jagah **Standard Template Library** ko reuse karna seekho. Gyarah container types, paanch iterator categories aur satrah algorithms seekho. Ye stage lambe, error-prone code ko chhote, saaf code mein badal deta hai.

Stage 3 — Modern C++20 (Unit III)

Wo features adopt karo jo professional modern C++ define karte hain: **auto**, **constexpr** family, smart pointers, move semantics, perfect forwarding, structured bindings, `optional / variant / any / span / string_view`, <=> aur standard attributes.

Stage 4 — Reusable Design (Unit IV)

Dus classic **design patterns** seekho. Har pattern *kab* appropriate hai, ye seekhna sabse important hai — patterns ko naam se pehchaan kar apply kar paana hi interview ya viva mein engineering maturity ka sabse bada signal hota hai.

Stage 5 — Concurrency (Unit V)

Finish karo **threads, mutexes, atomic operations, futures** aur finally **coroutines** ke saath. Ye stage student ko sequential programmer se aise programmer mein badal deta hai jo ek saath multiple cheezein chalane waale programs ke baare mein soch sakta hai.

Stage 6 — Exam Consolidation

Practice set complete karo, **Master Revision Guide** ko cover karo, one-line revisions ko yaad karo, aur last-day aur one-hour revision sheets ko drill karo.

> ◆ **REMEMBER**
>
> Speed se zyada consistency important hai. Roz do ghante focused practice, chhe hafte tak — ek complete beginner bhi comfortably examination level tak pahunch jaayega. Sab kuch ek hi hafte mein nikaalne ki koshish karoge to kaam nahi banega.

Ayush Jha

Prerequisite Foundation — C++ From Zero

Ye chapter sirf isliye exist karta hai taaki official syllabus ke paanch units samajh aa sakein. Ismein wahi *minimum* C++ background hai jo zaroori hai — na zyada, na kam. Agar aapne pehle kabhi kuch simple C ya C++ programs likhe hain, to is chapter ko jaldi se skim kar sakte ho; agar aapne zindagi mein kabhi program compile hi nahi kiya, to ise poora padho aur compiler par har snippet try karo.

F.1 Programming kya hoti hai?

Definition. Programming ka matlab hai computer ke liye aisi step-by-step instructions likhna jo bilkul precise hon aur jo computer execute karke koi problem solve kar sake ya koi task perform kar sake. Ye instructions kisi formal *programming language* mein likhi jaati hain, jaise C++.

Computer insaan ki tarah nahi hai — wo guess nahi kar sakta, samajh nahi sakta, apne aap kuch nahi kar sakta. Wo exactly wahi karta hai jo usse bola jaata hai, bilkul usi order mein, bilkul usi language ke rules ke saath jo wo samajhta hai. Isliye program ek likhi hui recipe jaisi hoti hai, aur programmer wo insaan hai jo ye recipe likhta hai.

F.2 C++ kya hota hai?

Definition. C++ ek general-purpose, compiled, statically-typed programming language hai jise originally Bjarne Stroustrup ne develop kiya tha. Ye kayi programming styles support karta hai — procedural programming, object-oriented programming, generic programming aur C++20 ke baad se coroutine-based programming bhi.

Kuch key characteristics jo aapko pata honi chahiye:

- **Compiled.** Source code (`.cpp` files) ko koi *compiler* jaise GCC, Clang ya MSVC pehle machine code mein translate karta hai, phir program run hota hai.
- **Statically typed.** Har variable ka type (wo integer hai, string hai, vector hai, etc.) compile time par hi fix ho jaata hai.
- **Zero-overhead abstractions.** High-level features jaise classes, templates aur smart pointers aise design kiye gaye hain ki unka runtime cost tabhi aata hai jab aap unhe actually use karte ho.
- **C++20.** Is course mein hum language ka yehi version padh rahe hain; ismein concepts, modules, ranges, coroutines, `consteval`, `constinit`, `std::span`, spaceship operator aur bahut kuch introduce hua.

> ★ **EXAM TIP**
>
> Is book ke har example ko flag `-std=c++20` (GCC/Clang) ya `/std:c++20` (MSVC) ke saath compile karo.

F.3 C++ Program ka Structure

Har C++ program mein exactly ek function hona chahiye jiska naam `main` hota hai. Execution `main` ke andar ke pehle executable statement se shuru hoti hai aur jab `main` return karta hai tab khatam hoti hai.

Minimum program

```C++
#include <iostream>          // standard input/output library
using namespace std;         // avoid writing std:: repeatedly

int main() {                 // program entry point
    cout << "Hello, C++!\n"; // print a line of text
    return 0;                // 0 means "finished successfully"
}
```

```
OUTPUT
Hello, C++!
```

Line-by-line explanation.

- `#include <iostream>` `cout` aur `cin` ki declarations laata hai.
- `using namespace std;` likhne se aap `std::cout` ki jagah sirf `cout` likh sakte ho. Study code ke liye theek hai; bade real projects mein avoid karte hain.
- `int main()` entry point hai. `int` ka matlab hai ki ye operating system ko ek integer return karta hai.
- `cout << "Hello, C++!"` standard output par text likhta hai. Operator `<<` ko *insertion operator* kehte hain.
- String literal ke andar `\n` newline character ko represent karta hai.
- `return 0;` batata hai ki program successfully khatam hua.

> **! WARNING**
>
> Statement ke end mein semicolon `;` **kabhi mat bhoolo**. Missing semicolon beginners ki sabse common galti hai.

F.4 Variables, Data Types & Operators

Definition. Variable memory ka ek named region hota hai jisme ek specific type ki value store hoti hai. C++ mein variable ko use karne se pehle uska type declare karna padta hai.

Type	Meaning	Example literal	Typical size
int	Whole number (bina decimal)	42	4 bytes
long long	Bahut bada integer	9000000000LL	8 bytes
float	Single-precision real number	3.14f	4 bytes
double	Double-precision real number	3.14159	8 bytes
char	Single character	'A'	1 byte
bool	true / false	true	1 byte
std::string	Text	"hello"	variable

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int    age    = 20;
    double price  = 199.75;
    char   grade  = 'A';
    bool   passed = true;
    string name   = "Ayush";

    cout << name << " is " << age << " years old\n";
    cout << "Price = " << price << ", Grade = " << grade << "\n";
    cout << boolalpha << "Passed? " << passed << "\n";
    return 0;
}
```

```
OUTPUT
Ayush is 20 years old
Price = 199.75, Grade = A
Passed? true
```

Operators jo aapko aane chahiye.

Category	Operators	Example
Arithmetic	+ − * / %	a + b, a % b
Assignment	= += −= *= /= %=	x += 5
Comparison	== != < > <= >=	a == b
Logical	&& || !	a && b
Increment/decrement	++ −−	i++, --i
Bitwise	& | ^ ~ << >>	x & y

> **! WARNING**
>
> **Division** operator `/` integer division karta hai jab dono operands integers hote hain: `7/2` ka result `3` aata hai, `3.5` nahi. Real division chahiye to kisi ek operand ko `double` banao.

F.5 Control Flow — Selection & Repetition

Selection: if / else if / else.

```cpp
int marks = 72;
if (marks >= 90)      cout << "O grade";
else if (marks >= 75) cout << "A grade";
else if (marks >= 60) cout << "B grade";
else                  cout << "Needs improvement";
```

Repetition: for, while, do-while.

```cpp
// Print 1..5 using each loop form
for (int i = 1; i <= 5; ++i) cout << i << ' ';
cout << '\n';

int j = 1;
while (j <= 5) { cout << j << ' '; ++j; }
cout << '\n';

int k = 1;
do   { cout << k << ' '; ++k; }
while (k <= 5);
```

```
OUTPUT
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

★ EXAM TIP

Container par iterate karne ke liye **range-based for** loop (C++11) prefer karo: `for (auto x : v) cout << x;`

F.6 Functions & Parameter Passing

Definition. Function statements ka ek named, reusable block hota hai jo optionally input parameters leta hai aur optionally ek value return karta hai. Functions se reuse badhta hai aur program chhote, samajh aane waale tukdon mein break ho jaata hai.

C++

```cpp
#include <iostream>
using namespace std;

// pass by value   – a copy is made
int  square(int n)             { return n * n; }
// pass by reference – no copy, caller's variable can change
void doubleIt(int& n)          { n = n * 2; }
// pass by const reference – no copy, cannot modify
double average(const int& a, const int& b) { return (a + b) / 2.0; }

int main() {
    int x = 5;
    cout << square(x) << '\n';   // 25
    doubleIt(x);
    cout << x << '\n';           // 10
    cout << average(4, 9) << '\n'; // 6.5
}
```

OUTPUT

```
25
10
6.5
```

Parameter pass karne ke teen tareeqe.

Mechanism	Syntax	Copy banti hai?	Caller modify ho sakta hai?
By value	int n	Haan	Nahi
By reference	int& n	Nahi	Haan
By const reference	const int& n	Nahi	Nahi

◆ REMEMBER

Koi bhi parameter jo *copy karne mein expensive* hai aur jise function ke andar modify nahi karna, uske liye **const reference** use karo. Sirf ye ek habit beginner code ke bahut saare performance problems door kar deti hai.

F.7 Arrays & Strings

Array ek fixed-size sequence hota hai jisme same type ke elements memory mein contiguous (ek ke baad ek) store hote hain. Modern C++ mein **string** koi raw character array nahi hota — wo `std::string` class hoti hai jo apni memory khud manage karti hai.

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int  marks[5] = {88, 91, 76, 65, 82};
    for (int i = 0; i < 5; ++i)
        cout << "marks[" << i << "] = " << marks[i] << '\n';

    string name = "Ayush Jha";
    cout << "Length = " << name.size() << '\n';
    cout << "Upper first char = " << char(toupper(name[0])) << '\n';
}
```

OUTPUT

```
marks[0] = 88
marks[1] = 91
marks[2] = 76
marks[3] = 65
marks[4] = 82
Length = 9
Upper first char = A
```

★ EXAM TIP

Raw C-style arrays ki jagah `std::array` (fixed size) ya `std::vector` (dynamic size) prefer karo. Dono Unit II mein detail se padhoge.

F.8 References & Pointers (essential minimum)

Reference. Reference kisi existing variable ka doosra naam (ek *alias*) hota hai. Ek baar bind ho gaya to use dobara kisi doosre variable par re-bind nahi kar sakte. Syntax: `T& r = original;`

Pointer. Pointer ek aisa variable hota hai jo kisi doosre variable ka memory address store karta hai. Ise dobara assign karke kahin aur point kar sakte hain, ya null bhi kar sakte hain. Syntax: `T* p = &original;`

C++

```cpp
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int& r = x;         // r is another name for x
    r = 20;             // this changes x
    cout << x << '\n';  // 20

    int* p = &x;        // p holds the address of x
    *p = 30;            // dereference: change what p points to
    cout << x << '\n';  // 30

    int* q = nullptr;   // safe "points nowhere" value
    if (q) *q = 99;     // guarded: won't crash
}
```

```
OUTPUT
20
30
```

Reference vs Pointer — quick comparison.

Aspect	Reference	Pointer
Null ho sakta hai?	Nahi	Haan (`nullptr` use karo)
Re-bind ho sakta hai?	Nahi	Haan
Access ka syntax	sidha naam likho	`*p` likhna padta hai
Initialise karna zaroori?	Haan	Nahi (par karna hi chahiye)
Typical use	function parameters	dynamic memory, data structures

> **! WARNING**
>
> Null ya dangling pointer ko dereference karne se **undefined behaviour** aata hai. Unit III mein aap raw pointers ko **smart pointers** se replace karna seekhoge.

F.9 Classes, Objects, Constructors, Destructors

Class. Class ek user-defined type hoti hai jo data members (*attributes*) aur un par kaam karne waale member functions (*methods*) ko ek saath group karti hai.

Object. Object class ka ek runtime instance hota hai — memory ka actual hissa jo class definition ke hisaab se layout hota hai.

Constructor. Ek special member function jiska naam class jaisa hi hota hai; ye object bante hi automatically call hota hai aur object ke data members ko initialise karta hai.

Destructor. Ek special member function jiska naam `~ClassName()` hota hai; ye object ki lifetime khatam hote hi automatically call hota hai aur object ke paas maujood resources ko release karta hai.

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    // data members
    string name;
    int    rollNo{};

    // constructor
    Student(string n, int r) : name(std::move(n)), rollNo(r) {
        cout << "Constructing " << name << '\n';
    }
    // destructor
    ~Student() {
        cout << "Destroying " << name << '\n';
    }
    // member function
    void display() const {
        cout << rollNo << " - " << name << '\n';
    }
};

int main() {
    Student s1("Ayush", 101);   // constructor runs here
    s1.display();
}                               // destructor runs at end of scope
```

OUTPUT

```
Constructing Ayush
101 - Ayush
Destroying Ayush
```

Access specifiers.

- `public` — kahin se bhi access ho sakta hai.
- `private` — sirf class ke andar se hi access ho sakta hai (`class` ke liye default).
- `protected` — class ke andar aur derived classes ke andar access ho sakta hai; jab inheritance ka plan ho tab use hota hai.

F.10 Inheritance & Basic Polymorphism

Inheritance ek class (jise *derived class* kehte hain) ko doosri class (jise *base class* kehte hain) ke members reuse aur extend karne deta hai. Relationship colon se likhi jaati hai: `class Derived : public Base`.

C++

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() { cout << "Animal eats\n"; }
};

class Dog : public Animal {   // Dog "is-an" Animal
public:
    void bark() { cout << "Woof!\n"; }
};

int main() {
    Dog d;
    d.eat();   // inherited from Animal
    d.bark();  // defined in Dog
}
```

OUTPUT

```
Animal eats
Woof!
```

Inheritance ke types.

Type	Meaning	Example
Single	Ek base, ek derived	class D : public B
Multilevel	Chain: B → D → E	class E : public D
Hierarchical	Ek base, kai derived	class D1 : public B; class D2 : public B
Multiple	Ek derived, kai bases	class D : public B1, public B2
Hybrid	In sabka combination	usually virtual base classes chahiye

◆ REMEMBER

`public` inheritance prefer karo. Ye **is-a** relationship ko sabse naturally express karta hai aur Unit I ke runtime polymorphism ke liye yahi chahiye.

UNIT I

Advanced Object-Oriented Programming

Runtime polymorphism, virtual functions, abstract classes, dynamic binding & SOLID

Ayush Jha

Unit I — Advanced Object-Oriented Programming

Ye unit kis baare mein hai

Unit I aapki object-oriented programming ko professional C++ codebase ke level tak le jaata hai. Ab aap classes aur inheritance ko sirf mechanically use karna chhod kar aise systems design karna seekhoge jo extensible hon, polymorphic use ke saath safe hon, aur mashhoor **SOLID** principles ke hisaab se organised hon.

Ye unit kyon important hai

Har aage waala unit strong OOP fluency assume karta hai. Unit II ke containers polymorphic objects store karte hain, Unit III ke smart pointers polymorphic objects ko own karte hain, Unit IV ke design patterns actually polymorphism hi hai, aur Unit V ka concurrent code aksar base pointers ke through virtual functions call karta hai. Agar Unit I weak hai, to Units II–V jitne hain utne mushkil lagenge.

Prerequisites

- Aap ek simple C++ program likh kar compile kar sakte ho.
- Aapko classes, objects, constructors aur destructors aate hain (F.9 dekho).
- Aapko single aur multi-level inheritance aata hai (F.10 dekho).
- Aap references aur pointers se comfortable ho (F.8 dekho).

Learning objectives

- **Compile-time** aur **runtime** polymorphism mein exact difference batana.
- Sahi virtual functions, pure virtual functions, aur abstract classes likhna.
- `override` keyword ke saath functions ko safely override karna.
- **Dynamic binding** ko virtual-table (*vtable*) mechanism ke terms mein explain karna.
- Kab destructor ko `virtual` banana chahiye aur kyon, ye pehchaanna.
- Paanchon **SOLID** principles ko C++ example ke saath yaad rakhna aur samjhaana.

Ayush Jha

1.1 Review of C++ Fundamentals and Object-Oriented Concepts

1.1.1 Object-Oriented Programming ke Chaar Pillars

Object-oriented programming chaar inter-related ideas par tiki hui hai. Examination ke liye in definitions ko verbatim yaad karna zaroori hai.

Encapsulation. Encapsulation ka matlab hai data members aur un par kaam karne waale member functions ko ek hi unit — class — ke andar bundle karna, aur access specifiers jaise `private` aur `protected` ke through state tak direct access ko restrict karna.

Abstraction. Abstraction ka matlab hai object ke sirf essential features ko baahar waali duniya ko dikhana, aur internal implementation details ko ek well-defined public interface ke peechhe chhupa dena.

Inheritance. Inheritance wo mechanism hai jisse ek nayi class (derived class) kisi existing class (base class) ke members aur behaviour ko acquire kar leti hai — isse code reuse hota hai aur *is-a* relationship express hota hai.

Polymorphism. Polymorphism ka matlab hai ki ek hi interface — koi operator, koi function name ya koi pointer type — object ke actual type ke hisaab se different behaviours ko represent kar sakta hai.

1.1.2 Class vs Object

Class	Object
Ek blueprint / template / user-defined type.	Class se bana hua actual instance.
Source code mein ek baar define hoti hai.	Run time par kai baar create ho sakta hai.
Khud koi memory nahi leti.	Instantiate hote hi memory leta hai.
Members aur member functions contain karti hai.	Un members ke liye actual values rakhta hai.
Example: `class Car {…};`	Example: `Car c1;`

1.1.3 Access Specifiers — Dobara

Specifier	Same class	Derived class	Outside class
private	Haan	Nahi	Nahi
protected	Haan	Haan	Nahi
public	Haan	Haan	Haan

i IMPORTANT

Base class ka `private` member derived class mein inherit to hota hai, par derived class ke andar se accessible nahi hota. Jo members derived classes ke liye hone chahiye, unhe `protected` mein rakho.

1.1.4 Compile-Time vs Runtime Polymorphism

C++ do flavours of polymorphism support karta hai. In dono mein difference batana examination ka sabse reliable short-answer question hai.

Aspect	Compile-time (static) polymorphism	Runtime (dynamic) polymorphism
Kab resolve hota hai	Compile time par	Run time par

Aspect	Compile-time (static) polymorphism	Runtime (dynamic) polymorphism
Mechanism	Function overloading, operator overloading, templates	Virtual functions, abstract classes
Inheritance chahiye?	Nahi	Haan
Base pointer/reference chahiye?	Nahi	Haan
Runtime cost	Kuch nahi	Thoda (vtable ke through ek indirect call)
Ise aur kya kehte hain	Static binding, early binding	Dynamic binding, late binding
Flexibility	Kam	Zyada

1.1.5 Function Overloading (Compile-Time Polymorphism ka Example)

Function overloading ek hi scope mein *same naam* ke kai functions allow karti hai, bas unki parameter lists alag honi chahiye. Compiler argument types dekh kar decide karta hai ki kaunsa function call karna hai.

C++

```cpp
#include <iostream>
using namespace std;

int  area(int side)                { return side * side; }             // square
int  area(int length, int breadth) { return length * breadth; }        // rectangle
double area(double radius)         { return 3.14159 * radius * radius; }// circle

int main() {
    cout << area(5)          << '\n';   // 25
    cout << area(4, 6)       << '\n';   // 24
    cout << area(3.0)        << '\n';   // 28.2743
}
```

OUTPUT

```
25
24
28.2743
```

> **! WARNING**
>
> Overloaded functions ke parameters ki *number ya types* alag honi hi chahiye. Sirf return type alag hone se overloading nahi banti.

1.1.6 `this` Pointer

Definition. Kisi bhi non-static member function ke andar `this` ek built-in pointer hota hai jisme us object ka address hota hai jis par wo member function call hua hai.

```cpp
class Counter {
    int n = 0;
public:
    Counter& increment() { ++n; return *this; }   // returns caller
    int value() const    { return this->n; }      // explicit use of this
};

Counter c;
c.increment().increment().increment();
cout << c.value() << '\n';   // 3
```

1.1.7 Constructors — Poori Family

Kind	Signature	Purpose
Default	`T()`	Bina arguments ke object banata hai.
Parameterised	`T(args…)`	Initial values ke saath object banata hai.
Copy	`T(const T&)`	Doosre object ki copy banata hai.
Move (C++11)	`T(T&&)`	Temporary ke resources ko steal karta hai.
Delegating	`T():T(0){}`	Ek constructor doosre ko call karta hai.
Explicit	`explicit T(int)`	Implicit conversions ko block karta hai.

◆ **REMEMBER**

Agar aap *koi bhi* constructor define nahi karte, to compiler khud ek public default constructor generate kar deta hai. Par agar aapne *koi bhi* constructor likh diya, to wo automatic default constructor **ab generate nahi** hota.

1.1.8 Destructors

Destructor tab call hota hai jab object ki lifetime khatam hoti hai. Uska main kaam hota hai un resources ko release karna — dynamic memory, files, network connections, mutexes — jo object ne acquire kiye the. Modern C++ mein aisi class ka destructor jo smart pointer ya standard container rakhti ho, usually kuch nahi karta, kyunki wo types apni safaai khud kar lete hain.

1.1.9 Ek Complete Worked Example

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
    string owner;
    double balance;
public:
    BankAccount(string name, double initial = 0.0)
        : owner(std::move(name)), balance(initial) {}

    void deposit(double amt)      { balance += amt; }
    bool withdraw(double amt) {
        if (amt > balance) return false;
        balance -= amt; return true;
    }
    void show() const {
        cout << owner << " : " << balance << '\n';
    }
};

int main() {
    BankAccount a("Ayush", 5000);
    a.deposit(1500);
    a.withdraw(2000);
    a.show();                 // Ayush : 4500
}
```

OUTPUT

```
Ayush : 4500
```

✎ EXAM POINT

Exam pattern. Bahut common question hai: *BankAccount class likhiye jisme deposit aur withdraw operations hon, encapsulation demonstrate karte hue.* Upar waala program iska model answer hai.

1.2 Runtime Polymorphism

Definition. Runtime polymorphism wo form hai jismein *kaunsa* member function execute hoga, ye decision program chalte waqt (runtime par) liya jaata hai, aur ye us object ke actual (dynamic) type par depend karta hai jise point kiya ja raha hai — na ki us pointer ya reference ke static type par jiske through call ho raha hai.

Simple language mein: aap ek function call likhte ho — maan lo `shape->draw();` — aur `draw()` ka kaunsa version actually chalega, ye is par depend karta hai ki us moment par `shape` asal mein *Circle* hai, *Square* hai, ya *Triangle*. Ye decision **run time par** hota hai, compile time par nahi. C++ mein ise support karne waala mechanism hai **virtual function**.

Runtime Polymorphism exist kyon karta hai

Runtime polymorphism ke bina, kai related types ko handle karne waala code aisa dikhta hai:

C++

```
void draw(const Shape& s) {
    if      (s.kind == CIRCLE)   drawCircle(s);
    else if (s.kind == SQUARE)   drawSquare(s);
    else if (s.kind == TRIANGLE) drawTriangle(s);
    /* … every time a new shape is added, this file must be edited … */
}
```

Har nayi shape aate hi har `if / else if` chain ko edit karna padta hai. Runtime polymorphism is poori chain ko ek hi virtual call se replace kar deta hai:

C++

```
void draw(const Shape& s) { s.draw(); }   // done
```

C++ mein Runtime Polymorphism ke liye kya chahiye

- Ek **base class** honi chahiye jisme kam se kam ek member function `virtual` keyword ke saath declared ho.
- Ek ya zyada **derived classes** honi chahiye jo us virtual function ko override karti hon.
- Call base class ke **pointer ya reference** ke through honi chahiye — base class ke object se by value nahi.

? COMMON CONFUSION

Agar in teen mein se koi bhi requirement missing hai, to call **statically** resolve ho jaati hai aur runtime polymorphism nahi hota — ye bahut common exam trap hai.

Minimal Example

C++

```cpp
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() const { cout << "Drawing a shape\n"; }
    virtual ~Shape() = default;   // virtual destructor — explained in 1.8
};

class Circle : public Shape {
public:
    void draw() const override { cout << "Drawing a circle\n"; }
};

class Square : public Shape {
public:
    void draw() const override { cout << "Drawing a square\n"; }
};

void render(const Shape& s) {   // reference to base class
    s.draw();                    // resolved at run time
}

int main() {
    Circle c;
    Square sq;
    render(c);    // Drawing a circle
    render(sq);   // Drawing a square
    Shape base;
    render(base); // Drawing a shape
}
```

OUTPUT

```
Drawing a circle
Drawing a square
Drawing a shape
```

Dekho ki function `render` ko `Circle` ya `Square` ke baare mein kuch pata nahi hai. Wo `Shape` ke reference ke through `s.draw()` call karta hai, aur phir bhi har baar sahi derived-class version chalta hai. Yehi runtime polymorphism hai.

Slicing Problem — Ek Classic Pitfall

Agar aap derived object ko *by value* aise function mein pass karte ho jo base object expect karta hai, to derived waala hissa silently kaat diya jaata hai — is phenomenon ko **object slicing** kehte hain.

C++

```cpp
void renderByValue(Shape s) { s.draw(); }  // Shape, not Shape&

Circle c;
renderByValue(c);     // prints "Drawing a shape"  — Circle part sliced off
```

! WARNING

Polymorphic objects ko **hamesha** pointer ya reference se pass karo. By value pass karne se wo slice ho jaate hain aur virtual functions ka poora maqsad khatam ho jaata hai.

Runtime Polymorphism at a Glance

Situation	Kaunsa function call hoga?
Base function virtual nahi hai, aur Derived object ko Base pointer se call kiya	Base::function() — *static dispatch*
Base function virtual hai, aur Derived object ko Base pointer se call kiya	Derived::function() — *dynamic dispatch*
Base function virtual hai, par Base object se (by value) call kiya	Base::function() — *slicing ho gayi*
Base function virtual hai, Derived object se directly call kiya	Derived::function() — direct call

≡ ONE-LINE REVISION

One-line revision. Runtime polymorphism = *virtual function + base pointer ya reference + overriding derived class*.

Ayush Jha

1.3 Virtual Functions

Definition. Virtual function ek non-static, non-constructor member function hota hai jise base class mein `virtual` keyword ke saath declare kiya jaata hai, taaki jab us function ko base-class pointer ya reference ke through kisi derived object par call kiya jaaye, to derived class ka override dynamically select ho jaaye.

1.3.1 Syntax

```C++
class Base {
public:
    virtual void f();               // declared virtual in the base
    virtual void g() const;
    virtual ~Base();                // (usually) virtual destructor
};

class Derived : public Base {
public:
    void f() override;              // override the base's virtual
    void g() const override;
    ~Derived() override;            // override the base's virtual destructor
};
```

Rules jo yaad rakhne hain.

• Sirf *member* function hi virtual ho sakta hai. Free functions aur static members virtual nahi ho sakte.

• `virtual` keyword **sirf base class mein** likha jaata hai. Derived class mein `override` use karo — ye compile-time check hai jo confirm karta hai ki aapne sach mein kuch override kiya hai.

• Overriding function ka signature base version se exactly match karna chahiye, `const`, reference qualifiers, aur (usually) return type including.

• Constructors virtual nahi ho sakte. Destructors ho sakte hain — aur jab class ko polymorphically use karna ho, tab unhe virtual banana hi chahiye.

1.3.2 Virtual Functions kaise kaam karte hain — Vtable

C++ standard koi particular implementation mandate nahi karta, par har mainstream compiler same technique use karta hai: **virtual function table**, ya *vtable*.

Aisi class ke liye jisme koi bhi virtual function ho:

• Compiler ek **vtable** banata hai — function-pointers ka array, har virtual function ke liye ek entry — *har class ke liye*.

• Us class ke har object mein ek hidden pointer hota hai, **vptr**, jo uski class ki vtable ko point karta hai.

• Jab base pointer ke through virtual function call hota hai, to generated code object ke vptr ko follow karke sahi vtable tak pahunchta hai aur wahan store kiye gaye function pointer ko call karta hai.

• Jab derived object construct hota hai, uska vptr *derived* class ki vtable par set ho jaata hai, isliye derived overrides automatically mil jaate hain.

> **IMPORTANT**
>
> **Virtual calls ki cost.** Har call par ek extra memory indirection. Modern CPUs par ye single-digit nanoseconds mein measure hoti hai aur tightest inner loops ko chhod kar har jagah bilkul negligible hai.

1.3.3 Ek Worked Example — Vehicles

C++

```cpp
#include <iostream>
#include <memory>
#include <vector>
using namespace std;

class Vehicle {
public:
    virtual void start() const { cout << "Vehicle starts\n"; }
    virtual double fuelEconomy() const { return 0.0; }
    virtual ~Vehicle() = default;
};

class Car : public Vehicle {
public:
    void   start() const override        { cout << "Car engine ignites\n"; }
    double fuelEconomy() const override  { return 18.5; }   // km/l
};

class Bike : public Vehicle {
public:
    void   start() const override        { cout << "Bike kick-starts\n"; }
    double fuelEconomy() const override  { return 55.0; }
};

int main() {
    vector<unique_ptr<Vehicle>> fleet;
    fleet.push_back(make_unique<Car>());
    fleet.push_back(make_unique<Bike>());

    for (const auto& v : fleet) {
        v->start();
        cout << " mileage = " << v->fuelEconomy() << " km/l\n";
    }
}
```

OUTPUT

```
Car engine ignites
 mileage = 18.5 km/l
Bike kick-starts
 mileage = 55 km/l
```

Dhyan do: same loop, same base-class pointer type `Vehicle*` — phir bhi har element ke liye alag code chalta hai. Virtual functions ke through runtime polymorphism ka matlab yahi hai.

1.3.4 Virtual vs Non-Virtual — Side-by-Side Demonstration

C++

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    void hello()          { cout << "Base::hello (non-virtual)\n"; }
    virtual void world()  { cout << "Base::world (virtual)\n"; }
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    void hello()          { cout << "Derived::hello\n"; }        // hides, not overrides
    void world() override { cout << "Derived::world\n"; }
};

int main() {
    Derived d;
    Base*  p = &d;
    p->hello();   // Base::hello  — static dispatch
    p->world();   // Derived::world — dynamic dispatch
}
```

OUTPUT

```
Base::hello (non-virtual)
Derived::world
```

? COMMON CONFUSION

`virtual` ke bina derived function override nahi karta — wo sirf base version ko *hide* karta hai. Hidden functions sirf derived static type ke through milte hain, base pointer ke through kabhi nahi.

1.3.5 `override` aur `final` Keywords

`override` (C++11). Kisi derived function par lagaya jaaye, to ye compiler ko instruct karta hai ki verify kare ki ye function sach mein base class ke kisi virtual function ko override karta hai. Agar nahi karta — maan lo signature galat type ho gaya — to code compile hi nahi hoga. Har jagah `override` lagane se silent bugs ki poori category khatam ho jaati hai.

`final` (C++11). Kisi virtual function par lagaya jaaye to aage override karna band kar deta hai. Kisi class par lagaya jaaye to aage inheritance band kar deta hai.

C++

```cpp
class Base   { public: virtual void f(); };
class Mid    : public Base { public: void f() override final; };  // no further override
class Leaf   : public Mid  { /* void f() override; would not compile */ };

class Sealed final {};                      // cannot be inherited from
```

1.3.6 Virtual Functions ke saath Common Mistakes

- Base class mein `virtual` keyword likhna bhool jaana — isse derived version sirf base version ko hide karega.

• Derived class mein signature badal dena (parameters alag, `const` missing) — derived function ek *naya* function ban jaata hai, override nahi karta. `override` ise compile time par pakad leta hai.

• Base-class constructor ya destructor ke andar se virtual function call karna — us moment object abhi *Base* hi hota hai, isliye base version chalta hai. Ye frequently tested trick question hai.

• Polymorphic objects ko by value pass karna — object slicing derived hissa kaat deti hai.

• Jab class ko base pointer ke through use karna ho tab destructor ko virtual banana bhool jaana — deletion undefined behaviour ban jaata hai. Dekho 1.8.

1.3.7 Exam-Level Points

✎ **EXAM POINT**

Q. Kya constructor virtual ho sakta hai?
A. Nahi. Virtual dispatch ke liye vptr chahiye, jo construction ke *dauraan* set hota hai. Ek hypothetical virtual constructor ko apne mechanism ke exist hone se pehle hi chalna padta.

✎ **EXAM POINT**

Q. Kya static member function virtual ho sakta hai?
A. Nahi. Static members kisi particular object ke nahi hote, isliye unke paas `this` pointer nahi hota aur na hi dispatch ke liye koi vptr hoti hai.

✎ **EXAM POINT**

Q. Kya friend function virtual ho sakta hai?
A. Nahi. Friend member nahi hota. Haan, friend function kisi virtual member function ko call kar sakta hai, jo phir dynamically dispatch hoga.

✎ **EXAM POINT**

Q. Kya function ko virtual banane se wo slow ho jaata hai?
A. Thoda bahut — har call par ek indirect memory access. Milne waali flexibility ke saamne ye trade-off normally dikhai bhi nahi deta.

≡ **ONE-LINE REVISION**

One-line revision. Virtual function base-class pointer ko object ki vtable ke through sahi derived-class version tak pahunchaata hai.

1.4 Pure Virtual Functions

Definition. Pure virtual function ek aisa virtual member function hota hai jiski base class mein koi definition nahi hoti, aur jise declaration ke end mein `= 0` laga kar declare kiya jaata hai; iski maujoodgi se hi har concrete (non-abstract) derived class par ye force padta hai ki wo us function ka apna definition de, tabhi us derived class ke objects ban sakte hain.

1.4.1 Syntax

```C++
class Base {
public:
    virtual void doWork() = 0;   // '= 0' makes the function pure
    virtual ~Base() = default;   // still recommended
};
```

Declaration ke end mein jo `= 0` likha hota hai, usse *pure-specifier* kehte hain. Iska matlab ye nahi ki function zero return karta hai; iska matlab hai ki "ye function yahan implement nahi hua — koi derived class uska implementation degi".

1.4.2 Pure Virtual Functions exist kyon karte hain

Kuch base classes aisa concept model karti hain jiska koi meaningful stand-alone behaviour nahi hota. Maan lo `Shape` class — wo batati hai ki har shape kya kar sakta hai — draw ho sakta hai, apni area bata sakta hai, scale ho sakta hai — par screen par aisa koi "generic shape" exist hi nahi karta jise aap actually draw kar sako. Base class sirf ek **interface** define karne ke liye exist karti hai. Relevant functions ko pure virtual banane se ye intention formally express ho jaati hai aur base class instantiate hone se ruk jaati hai.

1.4.3 Basic Example

```C++
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double area() const = 0;    // pure virtual
    virtual void   draw() const = 0;    // pure virtual
    virtual ~Shape() = default;
};

class Rectangle : public Shape {
    double w, h;
public:
    Rectangle(double w_, double h_) : w(w_), h(h_) {}
    double area() const override { return w * h; }
    void   draw() const override { cout << "[Rectangle " << w << "x" << h << "]\n"; }
};

class Triangle : public Shape {
    double b, h;
public:
    Triangle(double b_, double h_) : b(b_), h(h_) {}
    double area() const override { return 0.5 * b * h; }
    void   draw() const override { cout << "[Triangle b=" << b << ", h=" << h << "]\n"; }
};

int main() {
    // Shape s;                     // ERROR: cannot instantiate abstract class
    Rectangle r(3, 4);
    Triangle  t(6, 5);
    Shape* shapes[] = { &r, &t };
    for (Shape* s : shapes) { s->draw(); cout << " area = " << s->area() << "\n"; }
}
```

OUTPUT

```
[Rectangle 3x4]
 area = 12
[Triangle b=6, h=5]
 area = 15
```

1.4.4 Rules jo har student ko yaad rakhne hain

- Jis class mein **ek ya zyada** pure virtual functions hote hain, use **abstract class** kehte hain (§1.5).
- Abstract class ke **objects nahi ban sakte** — compiler ise forbid kar deta hai.
- Abstract class ke liye pointers ya references **banaye ja sakte hain**; polymorphism actually isi tarah use hota hai.
- Pure virtual function ki base class mein **definition ho sakti hai** (class body ke baahar define ki jaati hai). Derived classes ko phir bhi use override karna hi padta hai, par wo base definition ko explicitly `Base::function()` ke through call kar sakti hain.
- Agar derived class apne saare pure virtual functions override **nahi** karti, to wo derived class khud hi abstract reh jaati hai aur uske bhi objects nahi ban sakte.

1.4.5 Body ke saath Pure Virtual Function

Language ka ye corner aksar galat samjha jaata hai: pure virtual function ki definition allowed hai. Ye tab useful hota hai jab base class ek sensible default dena chahti hai jise derived class call kar sake, jabki derived class par override karna explicit roop se compulsory bhi rahe.

C++

```cpp
class Logger {
public:
    virtual void log(const std::string& msg) = 0;  // pure but with body below
    virtual ~Logger() = default;
};

// Definition of the pure virtual function itself
void Logger::log(const std::string& msg) {
    std::cout << "[BASE LOG] " << msg << '\n';
}

class FileLogger : public Logger {
public:
    void log(const std::string& msg) override {
        Logger::log(msg);                          // reuse the base default
        std::cout << "  (persisted to file)\n";
    }
};
```

1.4.6 Comparison: Virtual vs Pure Virtual

Aspect	Virtual function	Pure virtual function
Declaration	`virtual T f();`	`virtual T f() = 0;`
Base mein define karna zaroori?	Haan	Optional
Derived class ko override karna zaroori?	Nahi (base version inherit ho sakta hai)	Haan (warna derived class abstract hi rahegi)
Base class instantiate ho sakti hai?	Haan	Nahi
Purpose	Reusable default jise override kiya ja sakta hai	Interface / contract jo honour karna hi hai

≡ ONE-LINE REVISION

One-line revision. Pure virtual function = virtual function + `= 0` — ye apni class ko abstract bana deta hai aur har concrete derived class par apna implementation dena force karta hai.

Ayush Jha

1.5 Abstract Classes

Definition. Abstract class wo class hoti hai jisme kam se kam ek pure virtual function hota hai; kyunki kam se kam ek member function ka implementation nahi hai, compiler us class ke kisi bhi object ko banane ki permission nahi deta, aur class ko sirf uske pointers ya references ke through hi use kiya ja sakta hai.

1.5.1 Abstract Classes useful kyon hoti hain

Abstract class koi concrete cheez nahi, ek *concept* express karti hai. Wo kehti hai: “mere har instance se ye kaam ho sakta hai, par *kaise* hoga ye main nahi bataungi; har subclass ko ye faisla khud lena hoga.” Abstract classes isi tarah **Open/Closed Principle** (§1.9) ko support karti hain: nayi derived classes add ki ja sakti hain bina abstract interface ko use karne waale code ko badle.

Ayush Jha

1.5.2 Complete Example — Payment System

C++

```cpp
#include <iostream>
#include <string>
#include <memory>
#include <vector>
using namespace std;

class PaymentMethod {                    // Abstract interface
public:
    virtual bool pay(double amount) = 0;
    virtual string name() const     = 0;
    virtual ~PaymentMethod() = default;
};

class CreditCard : public PaymentMethod {
    string cardNo;
public:
    explicit CreditCard(string n) : cardNo(std::move(n)) {}
    bool pay(double amount) override {
        cout << "Charging " << amount << " to card " << cardNo << '\n';
        return true;
    }
    string name() const override { return "Credit Card"; }
};

class UPI : public PaymentMethod {
    string vpa;
public:
    explicit UPI(string v) : vpa(std::move(v)) {}
    bool pay(double amount) override {
        cout << "Requesting " << amount << " via UPI " << vpa << '\n';
        return true;
    }
    string name() const override { return "UPI"; }
};

void checkout(PaymentMethod& m, double amt) {
    cout << "Paying with " << m.name() << " ... ";
    if (m.pay(amt)) cout << "SUCCESS\n"; else cout << "FAILED\n";
}

int main() {
    CreditCard c("4111-1111-1111");
    UPI        u("ayush@iilm");
    checkout(c, 1499);
    checkout(u,  299);
}
```

OUTPUT

```
Paying with Credit Card ... Charging 1499 to card 4111-1111-1111
SUCCESS
Paying with UPI ... Requesting 299 via UPI ayush@iilm
SUCCESS
```

`checkout` function *koi bhi* `PaymentMethod` accept karta hai. Kal ko aap `NetBanking`, `Wallet` ya `Crypto` add kar do — `checkout` function mein ek bhi line change karne ki zaroorat nahi padegi.

1.5.3 Abstract Class vs Concrete Class

Aspect	Abstract class	Concrete class
Pure virtual hai?	Kam se kam ek	Ek bhi nahi
Instantiate ho sakti hai?	Nahi	Haan
Kaise use hoti hai?	Base pointer ya reference	Directly, ya base pointer ke through
Typical role	Interface define karti hai	Poora implementation deti hai
Example	`PaymentMethod`	`CreditCard`

1.5.4 C++ mein Interface Convention

C++ mein koi dedicated `interface` keyword nahi hota. Convention ke hisaab se, ek **interface** wo abstract class hoti hai jisme:

- Har function pure virtual hota hai.
- Koi data member nahi hota.
- Destructor virtual hota hai (aur usually `= default`).
- Implicit default constructor ke alawa koi constructor nahi hota.

C++

```cpp
class IPrintable {                      // convention: 'I' prefix = interface
public:
    virtual void print() const = 0;
    virtual ~IPrintable() = default;
};
```

1.5.5 Abstract Classes ke saath Common Mistakes

- Abstract class ko instantiate karne ki koshish karna: `Shape s;` — compiler mana kar dega.
- Derived class mein *har* pure virtual function ko override karna bhool jaana; isse derived class bhi abstract hi reh jaati hai aur instantiate nahi ho paati.
- Abstract base mein destructor ko non-virtual chhod dena — base pointer ke through delete karne par undefined behaviour ho jaata hai (§1.8).
- Jo cheez pure interface honi chahiye, usme data members daal dena — isse abstraction weak hoti hai aur multiple inheritance mushkil ho jaati hai.

✎ **EXAM POINT**

Exam pattern. "*Abstract class ko suitable example ke saath explain karo. Abstract class ka object kyon nahi ban sakta?*" — upar waala PaymentMethod example isi ka model answer hai.

≡ **ONE-LINE REVISION**

One-line revision. Abstract class mein kam se kam ek pure virtual function hota hai, isliye wo instantiate nahi ho sakti — wo sirf interface define karne ke liye exist karti hai.

1.6 Function Overriding

Definition. Function overriding wo mechanism hai jisse derived class apne base class se inherited virtual function ka apna definition deti hai — same function name, same parameter list aur (generally) same return type ke saath; overriding se dynamic dispatch ke dauraan derived-class waala version select hota hai.

1.6.1 Sahi Overriding ke liye Paanch Conditions

Kisi derived-class member function ke liye base-class function ko **override** karne ke liye — sirf hide ya overload karne ki jagah — neeche di gayi paanchon conditions satisfy honi chahiye:

1. Base-class function `virtual` declared hona chahiye.
2. Function ka **naam** bilkul same hona chahiye.
3. **Parameter list** identical honi chahiye (same number, same types, same order, aur matching `const`/reference qualifiers).
4. Member function ki **const-qualification** (yani end waala `const`) match karni chahiye.
5. **Return type** same hona chahiye, ya permitted *covariant* return type hona chahiye (base version ke return type se derived class ka pointer/reference).

◆ **REMEMBER**

Har derived-class override par `override` keyword lagao. Agar paanchon mein se koi bhi condition fail ho gayi, to compiler error de dega — silently unrelated function banane ki jagah.

1.6.2 Sahi Overriding ka Example

C++

```cpp
#include <iostream>
using namespace std;

class Employee {
public:
    virtual double bonus(double salary) const {   // base implementation
        return 0.05 * salary;
    }
    virtual ~Employee() = default;
};

class Manager : public Employee {
public:
    double bonus(double salary) const override {  // correct override
        return 0.15 * salary;
    }
};

class Intern : public Employee {
public:
    double bonus(double salary) const override {  // correct override
        return 0.02 * salary;
    }
};

void show(const Employee& e, double sal) {
    cout << "Bonus = " << e.bonus(sal) << '\n';
}

int main() {
    Manager m; Intern i; Employee base;
    show(m,    50000);   // 7500  (Manager::bonus)
    show(i,    50000);   // 1000  (Intern::bonus)
    show(base, 50000);   // 2500  (Employee::bonus)
}
```

OUTPUT

```
Bonus = 7500
Bonus = 1000
Bonus = 2500
```

1.6.3 Function Overloading vs Function Overriding

Aspect	Overloading	Overriding
Scope	Same class	Base class + derived class
Signature	Alag hona zaroori	Bilkul same hona zaroori
Inheritance chahiye?	Nahi	Haan
virtual chahiye?	Nahi	Haan (base par)
Kab resolve hota hai	Compile time par	Run time par
Polymorphism ka type	Compile-time (static)	Runtime (dynamic)
Return type	Alag ho sakta hai	Same hona chahiye (ya covariant)

1.6.4 Covariant Return Types

C++ “return type match karna chahiye” waali rule mein ek special relaxation deti hai. Agar base version type `B` ka pointer ya reference return karta hai, to overriding function `B` se derived kisi bhi class ka pointer ya reference return kar sakta hai. Ise *covariant return type* kehte hain, aur ye factory methods aur **Prototype** pattern (Unit IV) mein bahut useful hota hai.

C++

```
class Shape {
public:
    virtual Shape* clone() const { return new Shape(*this); }
    virtual ~Shape() = default;
};
class Circle : public Shape {
public:
    Circle* clone() const override { return new Circle(*this); }  // covariant
};
```

1.6.5 Overriding vs Hiding

Agar derived-class function ka naam same hai par signature *different* hai, ya base function `virtual` nahi hai, to overriding nahi hoti. Iske bajaay derived function same naam waale **saare** base-class functions ko **hide** kar deta hai. Ye bugs ka bahut common source hai.

C++

```cpp
class Base {
public:
    void greet()          { std::cout << "Hello from Base\n"; }
    void greet(int level) { std::cout << "Hello (level " << level << ")\n"; }
};

class Derived : public Base {
public:
    void greet() { std::cout << "Hi from Derived\n"; }   // HIDES both base greet()s
};

int main() {
    Derived d;
    d.greet();          // Hi from Derived
    // d.greet(3);      // ERROR: base version is hidden
    d.Base::greet(3);   // Explicit workaround
}
```

★ EXAM TIP

Hidden base-class overloads ko derived class ke scope mein waapas laane ke liye derived class body ke andar `using Base::greet;` likho.

1.6.6 Overriding mein Common Mistakes

- Base class mein `virtual` likhna bhool jaana — derived version sirf hide karega.
- `const`-qualification change kar dena — signatures match nahi karte.
- Parameter add ya remove kar dena — signatures match nahi karte.
- Return type different dena jo valid covariant return type bhi nahi hai.
- `override` likhna bhool jaana — code compile ho bhi sakta hai, par aap compile-time safety net kho dete ho.

≡ ONE-LINE REVISION

One-line revision. Overriding = same name + same signature + base function virtual hai + derived class apna body deti hai.

1.7 Dynamic Binding

Definition. Dynamic binding (jise late binding bhi kehte hain) wo process hai jisme kisi virtual call ke liye kaunsa specific function execute hoga, ye run time par decide hota hai — us object ke actual (dynamic) type ke basis par jis par call ho raha hai, na ki us pointer ya reference ke static type ke basis par jiske through call pahunchi.

1.7.1 Static Binding vs Dynamic Binding

Aspect	Static binding (early binding)	Dynamic binding (late binding)
Decision kab hota hai	Compile time par	Run time par
virtual chahiye?	Nahi	Haan
vtable use hoti hai?	Nahi	Haan (mainstream compilers mein)
Kis par apply hota hai	Normal functions, overloading, templates	Overridden virtual functions
Runtime cost	Kuch nahi (direct call)	Ek indirect memory access
Flexibility	Kam	Zyada

1.7.2 Dynamic Binding kaise kaam karta hai — Vtable dobara

Mainstream C++ compilers dynamic binding ko do invisible mechanisms se implement karte hain:

• Ek **vtable** — har class ke liye pointers ka ek array, har virtual function ke liye ek entry, jo us class ke current function definition ko point karta hai.

• Ek **vptr** — ek hidden pointer jo us class ke har object ke andar store hota hai jisme virtual functions hote hain, aur jo us class ki vtable ko point karta hai.

Jab base-class pointer ya reference ke through virtual function call hota hai:

```C++
base_ptr->virtualFunction();

// Roughly what the compiler generates:
(*(base_ptr->vptr[index_of_virtualFunction]))(base_ptr, args);
```

Object ke andar ka `vptr` object ki *asli* class ki vtable tak le jaata hai (sirf static base class ki nahi), isliye sahi override automatically select ho jaata hai.

1.7.3 Dynamic Binding in Action

C++

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound()      { cout << "generic sound\n"; }
    void         staticInfo() { cout << "I am an Animal (static)\n"; }
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void sound()      override { cout << "Woof!\n"; }
    void staticInfo()          { cout << "I am a Dog (static)\n"; }
};

int main() {
    Dog d;
    Animal* a = &d;

    a->sound();       // Woof!             -> dynamic binding
    a->staticInfo();  // I am an Animal    -> static  binding (not virtual)
}
```

OUTPUT

```
Woof!
I am an Animal (static)
```

Ek hi base pointer dono calls par do alag resolution strategies use karta hai — virtual function ke liye dynamic, aur ordinary function ke liye static.

1.7.4 Dynamic Binding ke liye Teeno Ingredients zaroori hain

Requirement	Agar missing hai to kya hoga
1. Base class mein virtual function	Static binding ho jaayegi
2. Derived class mein overriding function	Base version call hoga
3. Call base-class pointer ya reference ke through ho	Slicing ho sakti hai; base version call hoga

1.7.5 Kab Dynamic Binding nahi hoti

• Jab function virtual nahi hota.

• Jab call kisi object se hoti hai (pointer ya reference se nahi) — compiler ko exact type pata hota hai, isliye dispatch ki zaroorat hi nahi.

• Jab virtual call base-class constructor ya destructor ke andar se ki jaati hai — construction/destruction ke dauraan object ka dynamic type temporarily base class hi hota hai.

• Jab function explicit qualifier ke saath call ho, jaise `ptr->Base::f()` — ye static binding force kar deta hai.

! WARNING

Frequent viva trap. Constructor ke andar se ki gayi virtual call *base* version ko hi call karti hai, kyunki derived object abhi exist hi nahi karta. Construction ya destruction ke dauraan virtual dispatch par kabhi rely mat karo.

≡ ONE-LINE REVISION

One-line revision. Dynamic binding = run time par vtable ke through virtual function ka selection, object ki real class ke basis par.

Ayush Jha

1.8 Virtual Destructors

Definition. Virtual destructor base class ka wo destructor hota hai jise `virtual` keyword ke saath declare kiya jaata hai, taaki jab object ko base class ke pointer ke through delete kiya jaaye, to pehle object ke actual (dynamic) type ka destructor chale, phir sahi order mein har base subobject ke destructors.

1.8.1 Virtual Destructors essential kyon hain

Agar derived-class object ko aise base class ke pointer ke through delete kiya jaaye jiska destructor virtual **nahi** hai, to sirf base-class ka destructor chalta hai. Derived-class ka destructor — jo actually derived-class ke resources release karne ke liye responsible hota hai — kabhi call hi nahi hota. Result: **undefined behaviour**, jo sabse zyada memory leak ke roop mein dikhta hai.

1.8.2 Broken Example — Virtual Destructor missing hai

C++

```
#include <iostream>
using namespace std;

class Base {
public:
    Base()  { cout << "Base()\n"; }
    ~Base() { cout << "~Base()\n"; }         // NOT virtual  – BUG
};

class Derived : public Base {
    int* data;
public:
    Derived()  { data = new int[1000];   cout << "Derived()\n";  }
    ~Derived() { delete[] data;          cout << "~Derived()\n"; }
};

int main() {
    Base* p = new Derived();
    delete p;      // compiles, but undefined behaviour
                   // Prints:
                   // Base()
                   // Derived()
                   // ~Base()          <-- ~Derived() NEVER called => leak!
}
```

OUTPUT

```
Base()
Derived()
~Base()
```

! WARNING

Har `delete p` par 1000-integer ka buffer leak ho jaata hai. Kisi long-running server mein ye poora afternoon debugging mein lagega.

1.8.3 Correct Example — Virtual Destructor ke saath

C++

```cpp
class Base {
public:
    Base()          { cout << "Base()\n"; }
    virtual ~Base() { cout << "~Base()\n"; }   // virtual now
};

class Derived : public Base {
    int* data;
public:
    Derived()          { data = new int[1000]; cout << "Derived()\n"; }
    ~Derived() override{ delete[] data;        cout << "~Derived()\n"; }
};

int main() {
    Base* p = new Derived();
    delete p;    // Now prints Derived(), ~Derived(), ~Base() in the right order.
}
```

OUTPUT

```
Base()
Derived()
~Derived()
~Base()
```

1.8.4 Golden Rule

◆ REMEMBER

Agar kisi class mein koi bhi virtual function hai — ya class ko polymorphic deletion ke liye base class ki tarah use karna hai — to uska destructor `virtual` declare karo.

Ye rule itna important hai ki bahut saare static analyzers, style guides aur compiler warnings (GCC/Clang mein `-Wnon-virtual-dtor`) sirf isi rule ki violation pakadne ke liye bane hain.

1.8.5 Destructor Execution ka Order

Class `D` ke object ke liye, jo `B1` aur `B2` se derived hai:

- Sabse pehle `D` ka destructor chalta hai.
- Phir `D` ke data members ke destructors, reverse declaration order mein.
- Phir base classes ke destructors, inheritance list ke reverse order mein (yahan: `B2`, phir `B1`).

1.8.6 Pure Virtual Destructor

Destructor pure virtual ho sakta hai, par baaki pure virtual functions ke unlike ise *definition deni hi padti hai*, kyunki har derived-class destructor apne base destructor ko implicitly call karta hai.

```C++
class AbstractShape {
public:
    virtual ~AbstractShape() = 0;              // pure virtual destructor
};

AbstractShape::~AbstractShape() {              // required definition
    // usually empty
}
```

★ EXAM TIP

Jab aapke paas koi aur naturally pure-virtual function nahi hai, tab class ko abstract banane ke do idiomatic tareeqon mein se ye ek hai.

1.8.7 Virtual Destructors ke saath Common Mistakes

• Destructor ko sirf derived class mein virtual banana, base mein nahi — bekaar hai; base destructor phir bhi statically bind hota hai.

• Derived object ko raw base pointer se delete karna jab base destructor virtual nahi hai — undefined behaviour.

• Virtual destructor ki zaroorat se bachne ke liye base class ko `final` declare kar dena. Ye kaam to karta hai, par rarely yehi sahi design hota hai.

• Pure virtual destructor ki definition likhna bhool jaana — linker error.

≡ ONE-LINE REVISION

One-line revision. Base pointer + derived object + `delete` ⇒ base destructor ko `virtual` hona hi chahiye.

Ayush Jha

1.9 Introduction to SOLID Principles

SOLID ek acronym hai jise *Robert C. Martin* ne coin kiya tha — paanch closely related design principles ke liye jo object-oriented software ko samajhne, maintain karne aur extend karne mein aasaan banate hain. Ye paanch principles hain:

Letter	Full name	Ek line mein intent
S	Single Responsibility Principle	Ek class ka sirf ek hi reason to change hona chahiye.
O	Open/Closed Principle	Extension ke liye open, modification ke liye closed.
L	Liskov Substitution Principle	Subtypes ko wahan use kiya ja sakta hai jahan unka base type use hota hai.
I	Interface Segregation Principle	Clients ko un methods par depend nahi karna chahiye jo wo use nahi karte.
D	Dependency Inversion Principle	Concrete implementations par nahi, abstractions par depend karo.

★ MEMORY TRICK

Mnemonic. Koi bhi sentence yaad karo jiske words S-O-L-I-D se shuru hon — use apne memory hook ki tarah use karo.

1.9.1 S — Single Responsibility Principle (SRP)

Definition. Ek class ke paas change hone ka sirf ek, aur sirf ek hi reason hona chahiye; har responsibility ko apni alag class mein encapsulate karna chahiye.

Ye kaunsa problem solve karta hai. Jab ek hi class unrelated concerns sambhalti hai — maan lo business logic *aur* file I/O *aur* formatting — to kisi ek concern mein change karne par wahi file edit karni padti hai, jisse baaki concerns mein bugs aane ka risk badh jaata hai.

Bad design.

C++

```
class Invoice {
public:
    double totalAmount() const;         // business rule
    void   printAsPDF()  const;         // presentation
    void   saveToDB()    const;         // persistence
};    // three unrelated reasons to change  — violates SRP
```

Improved design.

C++

```
class Invoice   { public: double totalAmount() const; };
class InvoicePrinter { public: void print(const Invoice&) const; };
class InvoiceRepo    { public: void save (const Invoice&) const; };
```

◆ REMEMBER

Key takeaway. Responsibilities ko “reason to change” ke hisaab se split karo, sirf size ke hisaab se nahi.

1.9.2 O — Open/Closed Principle (OCP)

Definition. Software entities (classes, modules, functions) extension ke liye open honi chahiye par modification ke liye closed; naya behaviour existing code ko edit karke nahi, naya code likh kar add hona chahiye.

Bad design.

```C++
double area(const Shape& s) {
    if      (s.type == CIRCLE)   return 3.14 * s.r * s.r;
    else if (s.type == SQUARE)   return s.side * s.side;
    // adding TRIANGLE requires editing this function — not closed to modification
}
```

Improved design.

```C++
class Shape { public: virtual double area() const = 0; virtual ~Shape() = default; };
class Circle : public Shape { /*...*/ };
class Square : public Shape { /*...*/ };
double totalArea(const std::vector<Shape*>& v) {
    double s = 0; for (auto* sh : v) s += sh->area(); return s;
}     // adding Triangle now requires zero changes to totalArea()
```

1.9.3 L — Liskov Substitution Principle (LSP)

Definition. Agar `S` type `T` ka subtype hai, to program mein `T` type ke objects ko `S` type ke objects se replace karne par program ki correctness affect nahi honi chahiye.

Classic violation — Square/Rectangle trap.

```C++
class Rectangle {
public:
    virtual void setWidth (int w)  { width  = w; }
    virtual void setHeight(int h)  { height = h; }
    int area() const { return width * height; }
protected:
    int width{}, height{};
};

class Square : public Rectangle {          // "Square is a Rectangle" ?
public:
    void setWidth (int w) override { width = height = w; }
    void setHeight(int h) override { width = height = h; }
};

void process(Rectangle& r) {
    r.setWidth(5);
    r.setHeight(4);
    // Caller expects area() == 20, but if r is a Square it is 16.
}
```

? COMMON CONFUSION

Mathematically **Square** ek rectangle *hai*, par aisi `Square` class jo dono dimensions ko ek saath badal de, wo substitutable `Rectangle` **nahi** hai. Composition prefer karo, ya common base `Quadrilateral` introduce karo.

1.9.4 I — Interface Segregation Principle (ISP)

Definition. Ek bade, general interface se zyada better hai kayi chhote, focused interfaces; kisi bhi client ko un methods par depend karne par force nahi kiya jaana chahiye jo wo use karta hi nahi.

Bad design.

C++

```
class IMachine {                        // too fat
public:
    virtual void print()  = 0;
    virtual void scan()   = 0;
    virtual void fax()    = 0;
};

class OldPrinter : public IMachine {    // forced to implement scan() and fax()
    void print() override { /* ok */ }
    void scan()  override { throw "not supported"; }  // wrong
    void fax()   override { throw "not supported"; }  // wrong
};
```

Improved design.

C++

```
class IPrinter { public: virtual void print() = 0; virtual ~IPrinter() = default; };
class IScanner { public: virtual void scan () = 0; virtual ~IScanner() = default; };
class IFax     { public: virtual void fax  () = 0; virtual ~IFax()     = default; };

class OldPrinter    : public IPrinter                  { void print() override; };
class MultiFunction : public IPrinter, public IScanner, public IFax { /* all three */ };
```

1.9.5 D — Dependency Inversion Principle (DIP)

Definition. High-level modules ko low-level modules par depend nahi karna chahiye; dono ko abstractions par depend karna chahiye. Saath hi, abstractions ko details par depend nahi karna chahiye; details ko abstractions par depend karna chahiye.

Bad design.

C++

```
class MySQLDatabase { public: void save(const std::string& s); };

class OrderService {
    MySQLDatabase db;                   // hard-wired to MySQL
public:
    void placeOrder(const std::string& o) { db.save(o); }
};
```

Improved design.

C++

```cpp
class IDatabase {
public:
    virtual void save(const std::string& s) = 0;
    virtual ~IDatabase() = default;
};

class MySQLDatabase : public IDatabase { public: void save(const std::string& s) override; };
class MongoDatabase : public IDatabase { public: void save(const std::string& s) override; };

class OrderService {
    IDatabase& db;                          // depends on abstraction
public:
    explicit OrderService(IDatabase& d) : db(d) {}
    void placeOrder(const std::string& o) { db.save(o); }
};
```

★ EXAM TIP

DIP wahi principle hai jo unit testing ko sabse directly enable karta hai: ab `OrderService` ko ek fake `IDatabase` ke saath test kiya ja sakta hai jo record karti hai ki kya save hua.

1.9.6 SOLID at a Glance

Principle	Key idea	Sabse bada benefit
SRP	Har class ke liye change ka ek hi reason	Cohesion, easy debugging
OCP	Edit karke nahi, add karke extend karo	Kam regressions
LSP	Subclasses apne bases jaisa behave karein	Reliable polymorphism
ISP	Chhote, focused interfaces	Kam coupling
DIP	Abstractions par depend karo	Testability, flexibility

≡ ONE-LINE REVISION

One-line revision. SOLID = **S**ingle-responsibility + **O**pen-closed + **L**iskov + **I**nterface-segregation + **D**ependency-inversion.

Unit I — Revision Sheet

Ye sheet poore Unit I ko us material mein condense karti hai jo university examination mein aane ki sabse zyada sambhaavna rakhta hai. Exam se ek din pehle ise do baar padho, aur exam wali subah ek baar aur.

Unit I Summary

Unit I ne aapki object-oriented programming ko mechanical inheritance se genuine polymorphic design tak upgrade kiya. Aapne compile-time aur runtime polymorphism mein difference karna seekha, virtual aur pure-virtual functions likhe, abstract classes banayi, function overriding aur dynamic binding ko samjha, seekha ki base destructors virtual kyon hone chahiye, aur end mein sound object-oriented design guide karne waale paanch SOLID principles padhe.

Important Definitions

Term	Ek sentence mein definition
Encapsulation	Data aur us par kaam karne waale functions ko ek hi class mein bundle karna, aur baahar se direct access restrict karna.
Abstraction	Sirf essential features dikhana aur implementation details ko public interface ke peechhe chhupana.
Inheritance	Mechanism jisse derived class base class ke members ko reuse aur extend karti hai.
Polymorphism	Ek hi interface ka kai underlying behaviours ko represent kar paana.
Runtime polymorphism	Kaunsa function chalega ye decision run time par virtual call ke through hota hai.
Virtual function	Base mein virtual declared member function, taaki derived overrides dynamically select ho sakein.
Pure virtual function	Aisa virtual function jiska base implementation nahi hota (= 0), aur jo apni class ko abstract bana deta hai.
Abstract class	Aisi class jisme kam se kam ek pure virtual function ho; instantiate nahi ho sakti.
Function overriding	Derived class base ke virtual function ka apna implementation same signature ke saath deti hai.
Dynamic binding	Run time par, vtable ke through, object ke dynamic type ke basis par virtual function ka selection.
Virtual destructor	Base destructor ko virtual banana taaki base pointer ke through delete karne par poora object destroy ho.
SOLID	Paanch design principles: SRP, OCP, LSP, ISP, DIP.

Important Syntax

C++

```cpp
// Virtual function
class Base { public: virtual void f(); virtual ~Base() = default; };

// Pure virtual function -> abstract class
class Interface { public: virtual void g() = 0; virtual ~Interface() = default; };

// Correct override
class Derived : public Base { public: void f() override; };

// Final class or virtual
class Sealed final {};
class Base2 { public: virtual void h() final; };

// Pure virtual destructor (requires an out-of-class definition)
class AbstractBase { public: virtual ~AbstractBase() = 0; };
inline AbstractBase::~AbstractBase() {}
```

Important Comparisons

Comparison	Ek line mein answer
Overloading vs Overriding	Overloading = same class, different signatures, static; Overriding = base + derived, same signature, virtual, dynamic.
Virtual vs Pure Virtual	Virtual ka base body hota hai aur ise override karna optional hai; pure virtual mein = 0 hota hai aur override karna must hai.
Abstract vs Concrete class	Abstract mein ≥ 1 pure virtual hota hai aur instantiate nahi ho sakti; concrete mein koi nahi hota aur instantiate ho sakti hai.
Static vs Dynamic binding	Static compile time par resolve hoti hai; dynamic run time par vtable ke through resolve hoti hai.
Overriding vs Hiding	Overriding ke liye virtual + matching signature chahiye; warna derived function base waale ko simply hide kar deta hai.

Expected Exam Questions

- Runtime polymorphism define karo. Illustrate karne ke liye C++ program do.
- Virtual functions ki working ko vtable mechanism ke saath explain karo.
- Function overloading aur function overriding mein difference examples ke saath batao.
- Pure virtual function kya hota hai? Abstract class useful kyon hoti hai?
- Virtual destructor ka concept program ke saath explain karo.
- Paanch SOLID principles ko state aur explain karo, har ek ke liye C++ example ke saath.
- Dynamic binding kya hota hai? Ye static binding se kaise different hai?
- Example ke saath dikhao ki Square-Rectangle hierarchy se LSP kaise violate ho sakta hai.

Practice Questions (Graded)

Level	Task
★	Base class `Animal` banao jisme virtual method `speak()` ho, aur `Dog` aur `Cat` derive karo.
★★	Abstract class `Shape` design karo jisme pure virtual `area()` ho; Circle, Square aur Triangle derive karo.
★★★	Upar waale ko extend karo taaki `vector<unique_ptr<Shape>>` mixed shapes rakhe aur unki total area print kare.
★★★★	Wo memory-leak demonstrate karo jo tab hota hai jab base destructor virtual nahi hota; phir use fix karo.
★★★★★	DIP use karke payment system design karo: ek `IPayment` interface + kam se kam do implementations, jo `Order` class mein inject hon.

Self-Test (Two-Minute Recall)

- Runtime polymorphism ke liye zaroori teen ingredients batao.
- Jab constructor ke andar se virtual call hoti hai tab kya hota hai, explain karo.
- `IMachine` interface jisme `print, scan, fax` ho, wo SOLID ka kaunsa letter violate karta hai?
- Ek line mein likho ki non-virtual base destructor ke saath base pointer se `delete p` karna dangerous kyon hai.
- Covariant return type ka ek example do.

! DO NOT FORGET

Do not forget. Aapke paper ka virtual functions par aaya har question secretly in teen facts mein se kisi ek ko test karega: (1) virtual + base pointer + override; (2) abstract class instantiate nahi ho sakti; (3) base destructor ko virtual hona chahiye.

Ayush Jha

UNIT II

Standard Template Library (STL) and C++20 Ranges

Eleven containers · Five iterator categories · Seventeen algorithms

Unit II — Standard Template Library (STL) and C++20 Ranges

Ye unit kis baare mein hai

Unit II syllabus ka wo hissa hai jo aapke roz ke C++ likhne ka tareeqa sabse seedha badal deta hai. **Standard Template Library** ready-made containers (arrays, lists, maps, ...) ka ek bada collection hai, saath mein iterators (un containers mein uniformly ghoomne ka tareeqa) aur algorithms (sorting, searching, transforming, counting, ...). Har cheez haath se banane ki jagah sabse pehle STL ki taraf dekhna seekhna — ye programmer productivity ka sabse bada jumps mein se ek hai.

Ye unit kyon important hai

Aage ke har unit STL use karte hain. Unit III ke smart pointers STL objects ko own karte hain, Unit IV ke design patterns almost hamesha STL containers ke saath implement hote hain, aur Unit V ka concurrent code frequently shared data par STL algorithms chalaata hai. STL fluency optional nahi hai — ye modern C++ ki working vocabulary hai.

Prerequisites

- Aap C++ program likh kar compile kar sakte ho (F.3).
- Aapko classes, objects, references aur pointers aate hain (F.8 – F.10).
- Aapne templates ko kam se kam ek baar dekha hai (ek bahut chhota refresher §2.1 mein included hai).
- Aap range-based `for` loops se comfortable ho.

Learning objectives

- STL kya hai, ye explain karna aur uske teen principal components batana.
- Gyarah official containers mein se kisi problem ke liye sahi container choose karna.
- Iterators kya hain, exist kyon karte hain, aur kaunsa container kaunsi category support karta hai, ye samajhna.
- Satrah listed STL algorithms mein se har ek ko sahi situation mein apply karna.
- Examination mein chhote C++20 STL programs fluently padhna aur likhna.

Ayush Jha

2.1 STL kya hota hai?

Definition. Standard Template Library (STL) C++ Standard Library ka wo hissa hai jo generic data structures aur algorithms ka ek set provide karta hai — sab templates se implemented — taaki wahi code har element type par uniformly kaam kare.

2.1.1 STL ke Teen Components

Component	Ye kya hai	Examples
Containers	Class templates jo objects ke collections store karte hain.	`vector, list, map, set, ...`
Iterators	Uniform objects jo containers mein walk karte hain.	`begin(), end()`, pointers
Algorithms	Function templates jo iterators ke ranges par operate karte hain.	`sort, find, count, transform`

i IMPORTANT

Chautha ingredient. Kuch textbooks list mein **function objects** (functors, lambdas) aur **allocators** bhi add karte hain. Ye aapke syllabus mein officially listed nahi hain, par aap algorithms ke saath lambdas ka bahut use karoge.

2.1.2 Ek Minute ka Template Refresher

STL **templates** use karta hai, jisse wahi code different types ke saath kaam kar sakta hai. Ek bahut chhota example — Unit II mein aapko khud templates likhne nahi padenge, par unhe padhna zaroor aana chahiye:

C++

```
template <typename T>
T maxOf(T a, T b) { return (a > b) ? a : b; }

// use it:
maxOf<int>(3, 7);        // 7
maxOf<double>(1.4, 2.7); // 2.7
maxOf(std::string("hi"), std::string("hello")); // "hi"
```

2.1.3 STL exist kyon karta hai

• Har C++ project ko wahi data structures chahiye — dynamic arrays, linked lists, hash maps. Inhe dobara banane mein time waste hota hai aur galtiyaan aati hain.

• STL waale versions **battle-tested** aur **hand-optimised** hain — student ka apna implementation rarely unhe match kar paata hai.

• STL types ke interfaces consistent hote hain: har container ke paas `size()`, `empty()`, `begin()`, `end()` hote hain. Ek seekho, kai samajh jaao.

• STL algorithms containers se decoupled hote hain: wahi `sort` vector, deque ya plain array — teeno par kaam karta hai.

2.1.4 Container Categories

Gyarah official containers ko is tarah group kiya jaata hai:

Category	Members	Storage ka idea
Sequence	`array, vector, deque, list, forward_list`	Elements ek definite order mein

Category	Members	Storage ka idea
Container adaptor	`stack, queue, priority_queue`	Sequence container ka restricted view
Associative (ordered)	`set, map`	Sorted, tree-based
Associative (unordered)	`unordered_map`	Hash-table-based, average O(1)

≡ ONE-LINE REVISION

One-line revision. STL = *containers* + *iterators* + *algorithms*, sab templates ke roop mein likhe gaye.

2.1.5 Pehla Complete STL Program

C++

```cpp
#include <iostream>
#include <vector>
#include <algorithm>     // sort
#include <numeric>       // accumulate
using namespace std;

int main() {
    vector<int> v { 5, 2, 9, 1, 7, 3 };

    sort(v.begin(), v.end());            // algorithm on iterators
    for (int x : v) cout << x << ' ';    // range-based for uses iterators
    cout << "\nSum = " << accumulate(v.begin(), v.end(), 0) << '\n';
}
```

OUTPUT

```
1 2 3 5 7 9
Sum = 27
```

Sirf chhe lines ka code jo scratch se likhne mein teen page lag jaate. Isliye STL matter karta hai.

2.2 STL Containers

Ye section official syllabus mein listed gyarah ke gyarah containers ko syllabus order mein cover karta hai. Har container ke liye aapko milta hai: header, kab use karna hai, declaration syntax, common operations, ek worked example, aur exam-important points.

2.2.1 `std::array` — fixed-size array

Header: `<array>`
Complexity (typical): Access O(1), size compile time par fixed
Kab use karein: jab aapko aisa array chahiye jiska size code likhte waqt hi pata ho, aur STL-style iteration aur bounds checking bhi chahiye.

Definition. `std::array<T, N>` ek container hai jo `T` type ke `N` elements ke fixed-size C-style array ko encapsulate karta hai. Iska size ek compile-time constant hota hai.

C++

```cpp
#include <array>
#include <iostream>
using namespace std;

int main() {
    array<int, 5> a { 10, 20, 30, 40, 50 };

    cout << "size  = " << a.size()  << '\n';   // 5
    cout << "front = " << a.front() << '\n';   // 10
    cout << "back  = " << a.back()  << '\n';   // 50
    cout << "a[2]  = " << a.at(2)   << '\n';   // 30, throws on out-of-range

    for (int x : a) cout << x << ' ';
    cout << '\n';
}
```

OUTPUT

```
size  = 5
front = 10
back  = 50
a[2]  = 30
10 20 30 40 50
```

★ EXAM TIP

`a[i]` bounds check nahi karta; `a.at(i)` `std::out_of_range` throw karta hai. Jab index user input se aa raha ho tab `at()` prefer karo.

2.2.2 `std::vector` — dynamic array

Header: `<vector>`
Complexity (typical): Amortised O(1) push_back, O(1) indexed access
Kab use karein: jab aapko growable array chahiye — ye modern C++ ka default container hai.

Definition. `std::vector<T>` ek sequence container hai jo apne elements ko dynamically-allocated memory ke ek contiguous block mein store karta hai, aur elements add hote hi apne aap grow hota jaata hai.

```cpp
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v { 1, 2, 3 };

    v.push_back(4);         // add at the back
    v.push_back(5);
    v.pop_back();           // remove the last
    v.insert(v.begin(), 0); // insert at the front

    cout << "size = " << v.size() << ", capacity >= size = "
         << v.capacity() << '\n';
    for (auto x : v) cout << x << ' ';   // 0 1 2 3
}
```

OUTPUT

```
size = 4, capacity >= size = 6
0 1 2 3
```

Common member functions.

Function	Kya karta hai
`push_back(x)`	x ko end mein add karta hai.
`pop_back()`	Last element ko remove karta hai.
`insert(it, x)`	Position it se pehle x insert karta hai.
`erase(it)`	Position it ka element remove karta hai.
`size()`	Elements ki number.
`empty()`	true agar size() == 0.
`clear()`	Saare elements remove karta hai.
`reserve(n)`	n elements ke liye memory pehle se allocate karta hai.

◆ **REMEMBER**

`vector` elements ko **contiguously** store karta hai, isliye ye *cache-friendly* hota hai aur pure iteration ke liye sabse fast container hai. Jab tak koi specific reason na ho, isi ko choose karo.

2.2.3 `std::deque` — double-ended queue

Header: `<deque>`
Complexity (typical): O(1) push/pop dono ends par, O(1) indexed access
Kab use karein: jab aapko front aur back dono jagah fast insertion chahiye.

Definition. `std::deque<T>` (“double-ended queue”) ek sequence container hai jo dono ends par fast insertion aur removal support karta hai; internally ye usually elements ko fixed-size memory blocks ki ek sequence mein store karta hai.

C++

```cpp
#include <deque>
#include <iostream>
using namespace std;

int main() {
    deque<int> d { 2, 3, 4 };
    d.push_front(1);        // 1 2 3 4
    d.push_back (5);        // 1 2 3 4 5
    d.pop_front();          //   2 3 4 5
    for (auto x : d) cout << x << ' ';
}
```

OUTPUT

```
2 3 4 5
```

! WARNING

`vector` ke unlike, `deque` contiguous hone ki guarantee **nahi** deta. `&d[0]` ka pointer lekar use C-style array ki tarah treat mat karo.

Ayush Jha

2.2.4 `std::list` — doubly-linked list

Header: `<list>`
Complexity (typical): O(1) insert/erase kahin bhi, index se access O(n)
Kab use karein: jab aapko sequence ke middle mein frequent insertion ya removal chahiye.

Definition. `std::list<T>` ek sequence container hai jo doubly-linked list ke roop mein implemented hota hai; kisi position ka iterator mila ho to wahan constant-time insertion aur removal support karta hai, par index se random access support nahi karta.

C++

```cpp
#include <list>
#include <iostream>
using namespace std;

int main() {
    list<int> L { 10, 20, 30, 40 };

    auto it = L.begin();
    ++it; ++it;              // now points to 30
    L.insert(it, 25);        // 10 20 25 30 40
    L.remove(20);            // 10 25 30 40

    for (int x : L) cout << x << ' ';
}
```

OUTPUT

```
10 25 30 40
```

vector vs list.

Aspect	vector	list
Memory layout	Contiguous	Nodes heap par scattered
Index se random access	Haan, O(1)	Nahi (walk karna padta hai)
Middle mein insert/erase	O(n)	Iterator mila ho to O(1)
Cache-friendliness	Excellent	Poor
Insert se iterators invalidate	Ho sakte hain saare	Sirf erased waala

? COMMON CONFUSION

Practice mein, `vector` almost har cheez mein `list` ko beat karta hai — kai insertions mein bhi — kyunki cache effects bade hote hain. Sirf isliye `list` mat choose karo kyunki textbooks kehte hain ki linked lists insertion ke liye fast hote hain.

2.2.5 `std::forward_list` — singly-linked list

Header: `<forward_list>`
Complexity (typical): Iterator ke baad O(1) insert/erase; sirf forward traversal
Kab use karein: jab aapko sabse lean linked list chahiye — na back pointer, na size counter.

Definition. `std::forward_list<T>` ek container hai jo singly-linked list ke roop mein implemented hota hai; har node sirf agle node ka pointer store karta hai, isliye `std::list` ke comparison mein memory bachti hai.

C++

```cpp
#include <forward_list>
#include <iostream>
using namespace std;

int main() {
    forward_list<int> f { 1, 2, 3, 4 };

    f.push_front(0);                // 0 1 2 3 4
    f.insert_after(f.begin(), 99);  // 0 99 1 2 3 4
    for (int x : f) cout << x << ' ';
}
```

OUTPUT

```
0 99 1 2 3 4
```

i IMPORTANT

`forward_list` ke paas `size()` member **nahi** hota — size poochne ke liye poori list walk karni padti, aur designers ne us cost ko hide karne se mana kar diya.

2.2.6 `std::stack` — LIFO adaptor

Header: `<stack>`
Complexity (typical): O(1) push, pop, top
Kab use karein: jab aapko last-in-first-out data structure chahiye (parsing, undo, recursion elimination).

Definition. `std::stack<T>` ek container adaptor hai jo kisi underlying sequence container (by default `std::deque`) ke upar last-in-first-out (LIFO) interface provide karta hai.

C++

```cpp
#include <stack>
#include <iostream>
using namespace std;

int main() {
    stack<int> s;
    s.push(1); s.push(2); s.push(3);

    while (!s.empty()) {
        cout << s.top() << ' ';    // read the topmost
        s.pop();                   // remove it
    }
}
```

OUTPUT

```
3 2 1
```

? COMMON CONFUSION

`pop()` popped value return **nahi** karta. Pehle `top()` se value padho, phir `pop()` call karo. Ye design decision `pop()` ko exception-safe rakhta hai.

2.2.7 `std::queue` — FIFO adaptor

Header: `<queue>`
Complexity (typical): O(1) push (back), pop (front)
Kab use karein: jab aapko first-in-first-out order chahiye (task queues, BFS, buffering).

Definition. `std::queue<T>` ek container adaptor hai jo first-in-first-out (FIFO) interface provide karta hai: elements peechhe se add hote hain aur aage se remove hote hain.

C++

```cpp
#include <queue>
#include <iostream>
using namespace std;

int main() {
    queue<string> q;
    q.push("Alice"); q.push("Bob"); q.push("Charlie");

    while (!q.empty()) {
        cout << q.front() << ' ';
        q.pop();
    }
}
```

OUTPUT

```
Alice Bob Charlie
```

Ayush Jha

2.2.8 `std::priority_queue` — heap adaptor

Header: `<queue>`
Complexity (typical): O(log n) push aur pop, O(1) top
Kab use karein: jab aapko hamesha sabse bada (ya sabse chhota) element agla chahiye.

Definition. `std::priority_queue<T>` ek container adaptor hai jo apne elements ko aise ordered rakhta hai ki sabse bada element (by default) hamesha `top()` par available rahe. Internally ye usually ek binary heap hota hai.

C++

```cpp
#include <queue>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    priority_queue<int> pq;          // max-heap by default
    pq.push(3); pq.push(9); pq.push(1); pq.push(7);
    while (!pq.empty()) { cout << pq.top() << ' '; pq.pop(); }
    cout << '\n';

    // Min-heap: use greater<> comparator
    priority_queue<int, vector<int>, greater<int>> mn;
    mn.push(3); mn.push(9); mn.push(1); mn.push(7);
    while (!mn.empty()) { cout << mn.top() << ' '; mn.pop(); }
}
```

OUTPUT

```
9 7 3 1
1 3 7 9
```

2.2.9 `std::set` — unique, sorted keys

Header: `<set>`
Complexity (typical): O(log n) insert, find, erase
Kab use karein: jab aapko unique elements ka aisa collection chahiye jo hamesha sorted order mein rahe.

Definition. `std::set<T>` ek ordered associative container hai jo type `T` ke unique elements store karta hai; internally ye ek self-balancing binary search tree (typically red-black tree) ke roop mein implemented hota hai.

C++

```cpp
#include <set>
#include <iostream>
using namespace std;

int main() {
    set<int> s { 5, 1, 4, 1, 3, 5, 2 };   // duplicates ignored

    for (int x : s) cout << x << ' ';     // 1 2 3 4 5 (sorted, unique)
    cout << "\ncontains 3? " << s.count(3) << '\n';   // 1
    s.erase(3);
    cout << "contains 3? " << s.count(3) << '\n';     // 0
}
```

OUTPUT

```
1 2 3 4 5
contains 3? 1
contains 3? 0
```

2.2.10 `std::map` — sorted key → value dictionary

Header: `<map>`
Complexity (typical): O(log n) insert, find, erase
Kab use karein: jab aapko unique keys ke saath values associate karke sorted key order mein rakhne hain.

Definition. `std::map<K, V>` ek ordered associative container hai jo key-value pairs ko key ke hisaab se sorted store karta hai; har key unique honi chahiye.

C++

```cpp
#include <map>
#include <string>
#include <iostream>
using namespace std;

int main() {
    map<string, int> marks {
        {"Alice", 88}, {"Bob", 76}, {"Charlie", 91}
    };

    marks["Diana"] = 82;          // insert
    marks["Bob"]   = 80;          // update

    for (const auto& [name, m] : marks)    // structured bindings (C++17)
        cout << name << " -> " << m << '\n';

    if (marks.find("Eve") == marks.end())
        cout << "Eve not present\n";
}
```

OUTPUT

```
Alice -> 88
Bob -> 80
Charlie -> 91
Diana -> 82
Eve not present
```

! WARNING

`map["key"]` agar key absent hai to ek value-initialised entry **create** kar deta hai. Read-only lookup ke liye `find()` ya (C++20) `contains()` use karo.

2.2.11 `std::unordered_map` — hash-based key → value

Header: `<unordered_map>`
Complexity (typical): Average O(1) insert, find, erase
Kab use karein: jab aapko sabse fast lookup chahiye aur key order matter nahi karta.

Definition. `std::unordered_map<K, V>` ek unordered associative container hai jo key-value pairs ko hash table mein store karta hai, aur average constant-time insertion, lookup aur deletion deta hai.

C++

```cpp
#include <unordered_map>
#include <string>
#include <iostream>
using namespace std;

int main() {
    unordered_map<string, int> freq;
    string words[] = { "apple", "banana", "apple", "cherry",
                       "banana", "apple" };

    for (const string& w : words) ++freq[w];

    for (const auto& [w, n] : freq)
        cout << w << " occurs " << n << " times\n";
}
```

OUTPUT

```
apple occurs 3 times
banana occurs 2 times
cherry occurs 1 times   (order may vary)
```

map vs unordered_map.

Aspect	std::map	std::unordered_map
Backing structure	Red-black tree	Hash table
Iteration ka order	Key se sorted	Unspecified
Lookup complexity	O(log n)	O(1) average, O(n) worst
Iterator stability	Bahut stable	Rehash se iterators invalidate ho sakte hain
Custom order	Comparator template arg	Hash + equality template args

2.2.12 Sahi Container Chunna — Quick Guide

Agar aapko chahiye…	To choose karo
STL-style API waala fixed-size array	`std::array`
Growable array (default choice)	`std::vector`
Dono ends par fast insertion	`std::deque`
Frequent middle insertion, iterators stable rahein	`std::list`
Lean singly-linked list	`std::forward_list`
LIFO access	`std::stack`
FIFO access	`std::queue`
Hamesha sabse-bada-next access	`std::priority_queue`
Sorted unique keys	`std::set`
Sorted key-value pairs	`std::map`
Sabse fast key-value lookup, order matter nahi karta	`std::unordered_map`

! DO NOT FORGET

Do not forget. Default container `std::vector` hi hota hai. Koi doosra tabhi choose karo jab aap exactly bata sako ki `vector` kyon suitable nahi hai.

2.3 Iterators

Definition. Iterator pointer ka ek generalisation hota hai: ek aisa object jo container ke andar kisi position ko identify karta hai aur kam se kam ye operations support karta hai — dereferencing (`*it`) se us position ka element read ya write karna, aur increment (`++it`) se agli position par jaana.

2.3.1 Iterators exist kyon karte hain

STL algorithms **ranges** par operate karte hain, jo iterators ke pair ke roop mein diye jaate hain: `[begin, end)`. Kyunki har container iterators expose karta hai, isliye wahi algorithm kisi bhi container par kaam karta hai. Iterators containers aur algorithms ke beech ka glue hain.

```cpp
vector<int> v; list<int> L; array<int, 4> a;
sort(v.begin(), v.end());     // vector iterators
// sort(L.begin(), L.end());  // ERROR: list has bidirectional iterators, not random-access
sort(a.begin(), a.end());     // array iterators (random access)
```

2.3.2 Iterator ki Paanch Categories

C++ standard iterator capabilities ki ek hierarchy define karta hai. Har strong category apne se pehle waali ka superset hoti hai.

Category	Read kar sakta hai?	Write kar sakta hai?	Aage badh sakta hai?	Peeche aa sakta hai?	Random access?
Input	Haan (ek baar)	Nahi	Haan (ek baar)	Nahi	Nahi
Output	Nahi	Haan (ek baar)	Haan (ek baar)	Nahi	Nahi
Forward	Haan	Haan	Haan (kai baar)	Nahi	Nahi
Bidirectional	Haan	Haan	Haan	Haan	Nahi
Random access	Haan	Haan	Haan	Haan	Haan (it + n)

2.3.3 Kaunsa Container Kaunsa Iterator Deta Hai?

Container	Iterator category
`vector, deque, array`	Random access
`list`	Bidirectional
`forward_list`	Forward
`set, map, multiset, multimap`	Bidirectional
`unordered_map / unordered_set`	Forward
Input streams (`istream_iterator`)	Input
Output streams (`ostream_iterator`)	Output

2.3.4 Har Category ek Chhote Example ke saath

(a) Input iterator. Stream ko left se right ek baar read karta hai.

```cpp
#include <iterator>
#include <iostream>
using namespace std;

int main() {
    cout << "Enter numbers ending with ^D / ^Z: ";
    int sum = 0;
    istream_iterator<int> it(cin), endIt;   // input iterator
    while (it != endIt) { sum += *it; ++it; }
    cout << "Sum = " << sum << '\n';
}
```

(b) Output iterator. Stream mein left se right ek baar write karta hai.

```cpp
#include <vector>
#include <iterator>
#include <algorithm>
#include <iostream>
using namespace std;

int main() {
    vector<int> v { 1, 2, 3, 4 };
    copy(v.begin(), v.end(),
         ostream_iterator<int>(cout, " "));   // output iterator
}
```

```
OUTPUT
1 2 3 4
```

(c) Forward iterator. Re-read kiya ja sakta hai; sirf aage chalta hai. `forward_list` yehi deta hai.

```cpp
forward_list<int> fl { 1, 2, 3 };
for (auto it = fl.begin(); it != fl.end(); ++it) cout << *it << ' ';
```

(d) Bidirectional iterator. Aage aur peeche dono move kar sakta hai. `list`, `map` aur `set` yehi dete hain.

```cpp
list<int> L { 10, 20, 30 };
auto it = L.end();     // one past last
--it; --it;            // now points to 20
cout << *it;           // 20
```

(e) Random-access iterator. Full arithmetic support karta hai: `it + n`, `it - n`, `it[n]`, comparisons. Vectors, deques aur arrays yehi dete hain.

```cpp
vector<int> v { 10, 20, 30, 40 };
auto it = v.begin();
cout << it[2] << '\n';     // 30
cout << *(it + 3) << '\n';  // 40
```

2.3.5 Common Iterator Pitfalls

• `end()` ko dereference karna **undefined behaviour** hai — `end()` last element se ek aage point karta hai.

• Underlying container modify hone ke baad iterator use karna invalid ho sakta hai — vectors `push_back` par reallocate ho sakte hain.

• Do alag containers ke iterators compare karna undefined hai — iterators sirf apne hi container ke andar meaningful hote hain.

• List ya map ke iterator par random-access syntax (`it + 3`) use karne ki koshish karne par code compile nahi hoga.

≡ ONE-LINE REVISION

One-line revision. Iterator categories, weakest se strongest: *Input*, *Output*, *Forward*, *Bidirectional*, *Random-access*.

Ayush Jha

2.4 STL Algorithms

STL algorithms function templates hote hain jo `<algorithm>` aur `<numeric>` headers mein declared hote hain. Har ek iterators ke half-open **range** `[first, last)` par operate karta hai. Neeche waale sub-sections syllabus order mein satrah ke satrah listed algorithms cover karte hain.

★ **EXAM TIP**

Sab ke liye `<algorithm>` include karo, aur `accumulate` ke liye additionally `<numeric>` bhi.

2.4.1 `sort()`

Definition. `sort` kisi range ke elements ko non-descending order mein arrange karta hai — unstable comparison sort se (typically Introsort, average O(n log n)); iske liye random-access iterators chahiye.

Signature: `sort(first, last) | sort(first, last, cmp)`

C++

```cpp
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

int main() {
    vector<int> v { 4, 1, 3, 2, 5 };
    sort(v.begin(), v.end());
    for (int x : v) cout << x << ' ';
    cout << '\n';

    // Custom order: descending
    sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
    for (int x : v) cout << x << ' ';
}
```

OUTPUT

```
1 2 3 4 5
5 4 3 2 1
```

★ **EXAM TIP**

`sort()` equal elements ka relative order preserve nahi karta. Uske liye `stable_sort()` use karo.

2.4.2 `stable_sort()`

Definition. `stable_sort` range ko non-descending order mein sort karta hai, aur saath hi equal compare hone waale elements ka relative order bhi preserve karta hai; available memory ke hisaab se O(n log n) ya O(n log² n) guaranteed.

Signature: `stable_sort(first, last) | stable_sort(first, last, cmp)`

C++

```cpp
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

struct Student { string name; int marks; };

int main() {
    vector<Student> v { {"A", 80}, {"B", 90}, {"C", 80}, {"D", 90} };
    stable_sort(v.begin(), v.end(),
        [](const Student& a, const Student& b){ return a.marks < b.marks; });
    for (auto& s : v) cout << s.name << ' ' << s.marks << '\n';
}
```

OUTPUT

```
A 80
C 80
B 90
D 90
```

2.4.3 find()

Definition. `find` range ke pehle aise element ka iterator return karta hai jo diye gaye value ke equal compare ho; aisa koi element nahi mila to `last` return hota hai.

Signature: `auto it = find(first, last, value);`

C++

```cpp
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

int main() {
    vector<int> v { 10, 20, 30, 40 };
    auto it = find(v.begin(), v.end(), 30);
    if (it != v.end())
        cout << "Found at index " << (it - v.begin()) << '\n';
    else
        cout << "Not found\n";
}
```

OUTPUT

```
Found at index 2
```

2.4.4 find_if()

Definition. `find_if` range ke pehle aise element ka iterator return karta hai jiske liye diya gaya unary predicate `true` return karta hai; koi match nahi mila to `last` return hota hai.

Signature: `auto it = find_if(first, last, predicate);`

C++

```cpp
vector<int> v { 4, 7, 2, 9, 5, 12 };
auto it = find_if(v.begin(), v.end(), [](int n){ return n > 8; });
if (it != v.end()) cout << "First > 8 is " << *it;
```

OUTPUT

```
First > 8 is 9
```

Ayush Jha

2.4.5 count()

Definition. `count` range mein un elements ki ginti return karta hai jo diye gaye value ke equal compare hote hain.

Signature: `int n = count(first, last, value);`

C++

```
vector<int> v { 1, 2, 3, 2, 4, 2, 5 };
cout << "Number of 2s = " << count(v.begin(), v.end(), 2);
```

OUTPUT

```
Number of 2s = 3
```

2.4.6 transform()

Definition. `transform` input range ke har element par ek unary function apply karta hai aur result ko output range mein likhta hai (output range input range ke same bhi ho sakti hai).

Signature: `transform(first, last, dest, unaryOp);`

C++

```
vector<int> v { 1, 2, 3, 4 };
vector<int> squared(v.size());
transform(v.begin(), v.end(), squared.begin(),
          [](int n){ return n * n; });
for (int x : squared) cout << x << ' ';
```

OUTPUT

```
1 4 9 16
```

2.4.7 for_each()

Definition. `for_each` range ke har element par diya gaya callable object (function pointer, functor ya lambda) order mein apply karta hai.

Signature: `for_each(first, last, function);`

C++

```
vector<int> v { 10, 20, 30 };
int sum = 0;
for_each(v.begin(), v.end(), [&sum](int n){ sum += n; });
cout << "Sum = " << sum;
```

OUTPUT

```
Sum = 60
```

2.4.8 copy()

Definition. `copy` source range ke har element ko destination range mein copy karta hai, diye gaye iterator se shuru karke, aur last copied element se ek aage ka iterator return karta hai.

Signature: `copy(first, last, dest);`

```cpp
vector<int> src { 1, 2, 3, 4, 5 };
vector<int> dst(5);
copy(src.begin(), src.end(), dst.begin());
for (int x : dst) cout << x << ' ';
```

OUTPUT

```
1 2 3 4 5
```

2.4.9 `remove()`

Definition. `remove` un saare elements ko, jo diye gaye value ke equal *nahi* hain, range ke front mein move kar deta hai aur naye logical end ka iterator return karta hai. Ye container ka size change nahi karta; size chhota karne ke liye *erase-remove* idiom use karo.

Signature: `auto newEnd = remove(first, last, value);`

```cpp
vector<int> v { 1, 2, 3, 2, 4, 2, 5 };
v.erase(remove(v.begin(), v.end(), 2), v.end());
for (int x : v) cout << x << ' ';
```

OUTPUT

```
1 3 4 5
```

★ **EXAM TIP**

Erase-remove idiom bahut common exam question hai.

2.4.10 `replace()`

Definition. `replace` range mein diye gaye value ki har occurrence ko doosri value se replace kar deta hai.

Signature: `replace(first, last, oldValue, newValue);`

```cpp
vector<int> v { 1, 2, 3, 2, 4, 2 };
replace(v.begin(), v.end(), 2, 99);
for (int x : v) cout << x << ' ';
```

OUTPUT

```
1 99 3 99 4 99
```

2.4.11 `reverse()`

Definition. `reverse` range ke elements ka order in-place reverse kar deta hai.

Signature: `reverse(first, last);`

```cpp
vector<int> v { 1, 2, 3, 4, 5 };
reverse(v.begin(), v.end());
for (int x : v) cout << x << ' ';
```

OUTPUT

```
5 4 3 2 1
```

2.4.12 rotate()

Definition. `rotate` range ke elements ko aise reorder karta hai ki `middle` waala element first ban jaata hai aur usse pehle waale elements end mein chale jaate hain.

Signature: `rotate(first, middle, last);`

C++

```cpp
vector<int> v { 1, 2, 3, 4, 5 };
rotate(v.begin(), v.begin() + 2, v.end());
for (int x : v) cout << x << ' ';
```

OUTPUT

```
3 4 5 1 2
```

2.4.13 partition()

Definition. `partition` range ko aise rearrange karta hai ki jin elements ke liye predicate `true` deta hai wo pehle aate hain, aur jin ke liye `false` deta hai wo baad mein. Pehla “false” element ka iterator return karta hai.

Signature: `auto mid = partition(first, last, predicate);`

C++

```cpp
vector<int> v { 1, 2, 3, 4, 5, 6, 7, 8 };
auto mid = partition(v.begin(), v.end(),
                     [](int n){ return n % 2 == 0; });
for (int x : v) cout << x << ' ';
cout << "\nBoundary index = " << (mid - v.begin());
```

OUTPUT

```
2 4 6 8 1 3 5 7   (relative order not preserved)
Boundary index = 4
```

2.4.14 binary_search( )

Definition. `binary_search` test karta hai ki diya gaya value sorted range mein exist karta hai ya nahi — O(log n) time mein.

Signature: `bool present = binary_search(first, last, value);`

C++

```
vector<int> v { 1, 3, 5, 7, 9, 11 };
cout << boolalpha
     << binary_search(v.begin(), v.end(), 7)  << '\n'   // true
     << binary_search(v.begin(), v.end(), 8);            // false
```

OUTPUT

```
true
false
```

★ EXAM TIP

Input range **sorted honi hi chahiye**. Warna result unspecified hota hai.

2.4.15 lower_bound( )

Definition. `lower_bound` sorted range mein pehli aisi position ka iterator return karta hai jo diye gaye value se kam compare nahi hoti — doosre words mein, wo leftmost position jahan value insert karne par range sorted hi rahegi.

Signature: `auto it = lower_bound(first, last, value);`

C++

```
vector<int> v { 1, 3, 5, 5, 5, 7, 9 };
auto it = lower_bound(v.begin(), v.end(), 5);
cout << "lower_bound(5) at index " << (it - v.begin());
```

OUTPUT

```
lower_bound(5) at index 2
```

2.4.16 upper_bound( )

Definition. `upper_bound` sorted range mein pehli aisi position ka iterator return karta hai jo diye gaye value se strictly greater compare hoti hai.

Signature: `auto it = upper_bound(first, last, value);`

C++

```
vector<int> v { 1, 3, 5, 5, 5, 7, 9 };
auto it = upper_bound(v.begin(), v.end(), 5);
cout << "upper_bound(5) at index " << (it - v.begin());
```

OUTPUT

```
upper_bound(5) at index 5
```

★ EXAM TIP

lower_bound aur **upper_bound** mil kar us value ke equal saare elements ka half-open range dete hain: `[lower_bound, upper_bound)`.

2.4.17 `accumulate()`

Definition. `accumulate` — jo `<numeric>` mein declared hai — range ke elements ka running sum compute karta hai (ya custom binary operation ke saath koi bhi left-fold), ek initial value se shuru karke.

Signature: `T total = accumulate(first, last, init);`

C++

```cpp
#include <numeric>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v { 1, 2, 3, 4, 5 };
    cout << accumulate(v.begin(), v.end(), 0) << '\n';      // 15
    // Product using a custom binary op
    cout << accumulate(v.begin(), v.end(), 1,
                       [](int a, int b){ return a * b; });   // 120
}
```

OUTPUT

```
15
120
```

★ EXAM TIP

Initial value ka type hi accumulation ke type ko decide karta hai. `accumulate(v.begin(), v.end(), 0)` ko `vector<double>` ke saath dene se har element silently `int` mein truncate ho jaata hai. `double` force karne ke liye `0.0` pass karo.

2.4.18 Algorithm Comparison Cheat Sheet

Algorithm	Purpose (ek line mein)	Header
`sort`	Range ko in-place sort karta hai (unstable).	<algorithm>
`stable_sort`	Relative order preserve karte hue sort.	<algorithm>
`find`	Value ke equal pehla element dhoondhta hai.	<algorithm>
`find_if`	Predicate satisfy karne waala pehla element.	<algorithm>
`count`	Value ke equal kitne elements hain.	<algorithm>
`transform`	Har element par function apply karke dest mein likhta hai.	<algorithm>
`for_each`	Har element par callable invoke karta hai.	<algorithm>
`copy`	Range ko doosri range mein copy karta hai.	<algorithm>
`remove`	Non-matching elements ko aage move karta hai.	<algorithm>
`replace`	Old value ko new value se badalta hai.	<algorithm>
`reverse`	Order ko in-place reverse karta hai.	<algorithm>
`rotate`	Middle ko first banane ke liye rotate.	<algorithm>
`partition`	True-predicate elements ko pehle group karta hai.	<algorithm>
`binary_search`	Sorted range mein existence test.	<algorithm>
`lower_bound`	Value ke liye leftmost insertion point.	<algorithm>
`upper_bound`	Value se strictly greater pehli position.	<algorithm>
`accumulate`	Left-fold / running sum.	<numeric>

2.4.19 Frequently Confused Algorithms

Pair	Ek line mein difference
`sort` vs `stable_sort`	sort faster hai; stable_sort equal keys ka relative order preserve karta hai.
`find` vs `find_if`	find ek value match karta hai; find_if ek predicate match karta hai.
`lower_bound` vs `upper_bound`	lower_bound pehle ≥ value ko point karta hai; upper_bound pehle > value ko.
`remove` vs `erase`	remove reorder karke new-end return karta hai; erase actually delete karta hai. Dono ko combine karo.
`copy` vs `transform`	copy elements ko unchanged chhodta hai; transform copy karte hue function apply karta hai.

≡ ONE-LINE REVISION

One-line revision. STL algorithms iterators ka `[first, last)` range aur ek functor ya predicate lete hain — unhe container ke baare mein kuch pata nahi hota.

Unit II — Revision Sheet

Ye sheet Unit II ko us material mein compress karti hai jiske examination mein aane ki sambhaavna sabse zyada hai. Har internal se pehle ise padho, aur end-semester paper se ek raat pehle phir se.

Unit II Summary

Standard Template Library gyarah containers deti hai — sequence containers, container adaptors aur associative containers mein grouped; iterators ki paanch categories jo ek capability hierarchy banati hain; aur satrah algorithms jo containers par directly nahi, iterator ranges par operate karte hain. STL fluency haath se likhi gayi darjanon lines ko kuch saaf lines mein badal deti hai, aur ye professional C++ ki working vocabulary hai.

Important Definitions

Term	Ek sentence mein definition
STL	Standard library ka template-based hissa jo containers, iterators aur algorithms deta hai.
Container	Class template jo chosen type ke objects ka collection store karta hai.
Iterator	Pointer-jaisa object jo container ke andar kisi position ko identify karta hai.
Algorithm	Function template jo iterators ke range par operate karta hai.
Sequence container	Container jo elements ko definite order mein store karta hai (array, vector, deque, list, forward_list).
Associative container	Container jo elements ko value ke hisaab se keyed store karta hai (set, map, unordered_map).
Container adaptor	Sequence container ke upar bana hua restricted interface (stack, queue, priority_queue).
Range	Iterators ka pair `[first, last)` jo ek half-open sequence denote karta hai.

Important Syntax

```cpp
// Container declarations
std::array<int, 5>       a { 1, 2, 3, 4, 5 };
std::vector<int>         v;             v.push_back(10);
std::deque<int>          d;             d.push_front(0);
std::list<int>           L;             L.push_back(1);
std::forward_list<int>   f;             f.push_front(1);
std::stack<int>          s;             s.push(1);
std::queue<int>          q;             q.push(1);
std::priority_queue<int> pq;            pq.push(1);
std::set<int>            st;            st.insert(1);
std::map<std::string,int>       m;      m["Ayush"] = 90;
std::unordered_map<std::string,int> um; um["Ayush"] = 90;

// Iterator range
auto b = v.begin(); auto e = v.end();

// Algorithm calls
sort  (v.begin(), v.end());
find  (v.begin(), v.end(), 3);
count (v.begin(), v.end(), 3);
transform(v.begin(), v.end(), v.begin(), [](int n){ return n*n; });
accumulate(v.begin(), v.end(), 0);
```

Important Comparisons

Comparison	Ek line mein answer
vector vs list	vector: contiguous, O(1) index, cache-friendly. list: linked, O(1) middle insert, poor cache.
array vs vector	array: fixed compile-time size. vector: runtime par grow hota hai.
stack vs queue	stack: LIFO. queue: FIFO.
set vs unordered_set	set: sorted, O(log n). unordered_set: hash-based, average O(1), unordered.
map vs unordered_map	map: key se sorted, tree. unordered_map: hash table, average lookup faster.
sort vs stable_sort	stable_sort equal keys ka relative order preserve karta hai.
find vs find_if	find ek value match karta hai; find_if ek predicate match karta hai.
lower_bound vs upper_bound	lower: pehla ≥; upper: pehla >.

Important Programs jo Yaad Rakhne Hain

- Vector read karo, sort karo, print karo. (`sort`, range-based for use hota hai.)
- Count karo ki vector mein koi value kitni baar aati hai.
- Erase-remove idiom se kisi value ki saari occurrences erase karo.
- `unordered_map` se word-frequency counter.
- `priority_queue` se k sabse bade numbers ka selection.
- Sorted vector par `binary_search / lower_bound / upper_bound` ka use.
- `accumulate` se vector ka sum; `vector<double>` ka sum lete waqt initial value par dhyan.

Expected Exam Questions

- STL define karo. Uske main components examples ke saath explain karo.
- `vector` aur `list` mein difference batao.
- Iterators ki paanch categories un containers ke saath explain karo jo har ek ko support karte hain.
- `map` use karke student marks store aur display karne ka program likho.
- Erase-remove idiom illustrate karo.
- `lower_bound` aur `upper_bound` ko program ke saath explain karo.
- `priority_queue` ordinary queue se kaise different hai?
- `transform` ko aise program ke saath explain karo jo har element ka square karta ho.

Practice Questions (Graded)

Level	Task
★	Vector mein das integers read karo aur reverse order mein print karo.
★★	EOF tak words read karo; `unordered_map` se har word aur uski frequency print karo.
★★★	Student marks ka sorted vector diya ho, to `lower_bound / upper_bound` se report karo ki kitne students ne 60 se 80 ke beech score kiya.
★★★★	Custom comparator ke saath `priority_queue<Task>` use karke task-priority scheduler design karo.
★★★★★	Mini spell-checker implement karo: dictionary ko `set<string>` mein read karo aur har input word ke liye `lower_bound` se alphabetically sabse close dictionary word report karo.

! DO NOT FORGET

Do not forget. STL algorithms ko containers nahi, iterators chahiye. Classical STL mein `sort(v)` compile *nahi* hota — aapko `sort(v.begin(), v.end())` likhna padta hai. C++20 ranges (`std::ranges::sort(v)`) finally containers ko directly accept karte hain.

UNIT III

Modern C++20 Language Features

Type deduction · constexpr family · smart pointers · move semantics · utility library

Unit III — Modern C++20 Language Features

Ye unit kis baare mein hai

Unit III aapko wo features sikhaata hai jo *modern* C++ (C++11 onwards, especially C++20) ko puraane "C with classes" style se sabse zyada alag karte hain. Aap seekhoge ki **type deduction**, **compile-time computation**, **smart pointers**, **move semantics** aur modern utility library use karke safer, faster aur shorter code kaise likha jaata hai. Yahan cover kiya gaya har feature real-world modern C++ code mein bhi milta hai aur university examination mein bhi.

Ye unit kyon important hai

Agar Unit I ne aapko objects ke saath design karna sikhaaya aur Unit II ne standard library reuse karna, to Unit III aapko ek modern C++ programmer ki vocabulary sikhaata hai. Unit IV ka har design pattern smart pointers use karta hai; Unit V ka har concurrent program `std::thread` use karta hai jo move semantics par chalti hai; aur jo bhi C++20 codebase aap professionally padhoge, usme `auto`, `constexpr` aur structured bindings milegi hi.

Prerequisites

- Strong OOP (Unit I) — smart pointers ke andar aap phir se virtual functions dekhoge.
- STL fluency (Unit II) — utility library vectors, tuples aur containers ke upar layered hai.
- Aap references aur pointers (F.8) aur lambdas se comfortable ho.

Learning objectives

- `auto`, `decltype` aur `decltype(auto)` se types ko correctly deduce karna.
- `constexpr`, `consteval` aur `constinit` mein difference karna.
- Diye gaye ownership model ke liye sahi smart pointer choose karna.
- Move semantics, rvalue references aur perfect forwarding ko explain karke sahi tarah use karna.
- Modern utility library (`optional`, `variant`, `any`, `span`, `string_view`, `filesystem`, `chrono`) ko idiomatically use karna.
- Aisa code padhna aur likhna jo three-way comparison operator `<=>` aur standard attributes use karta ho.

Ayush Jha

3.1 Type Deduction — `auto`, `decltype`, `decltype(auto)`

3.1.1 `auto`

Definition. Jab `auto` keyword kisi variable ya function return type ke liye type specifier ki tarah use hota hai, to ye compiler ko instruct karta hai ki us entity ka type uske initialiser ya return expression ke type se khud deduce kar le.

C++

```
#include <vector>
#include <string>
#include <iostream>
using namespace std;

int main() {
    auto  n   = 42;              // int
    auto  pi  = 3.14;             // double
    auto  s   = std::string("hi"); // std::string
    auto  v   = std::vector<int>{1,2,3};
    auto  it  = v.begin();         // std::vector<int>::iterator

    // Range-based for using auto - a modern C++ idiom
    for (auto x : v) cout << x << ' ';
    cout << '\n';
    // Use auto& when you may want to modify:
    for (auto& x : v) x *= 2;
    for (auto x : v) cout << x << ' ';
}
```

OUTPUT

```
1 2 3
2 4 6
```

Important rules.

- `auto` top-level `const` aur references ko drop kar deta hai. `const int& r = x; auto y = r;` likhne par `y` ka type `int` banta hai, `const int&` nahi.
- Unhe preserve karne ke liye explicitly likho: `auto&`, `const auto&`, `auto&&`.
- `auto` declared variable ko initialise karna **must** hai.
- Function return type `auto` ho sakta hai (C++14 onwards).

★ EXAM TIP

Non-trivial elements ke containers par range-based for loops mein `const auto&` prefer karo. Isse har element ki copy bachti hai aur accidental modification bhi nahi hoti.

3.1.2 `decltype`

Definition. `decltype(expr)` ek compile-time operator hai jo expression `expr` ka exact type deta hai — uski `const` qualification aur reference-ness including — bina us expression ko evaluate kiye.

```cpp
int  x  = 10;
const int& r = x;

decltype(x)  a = x;   // int
decltype(r)  b = x;   // const int&  (r is a reference)
decltype((x)) c = x;  // int&  (parenthesised => lvalue expression)

// Use case: mirror the exact type of another entity
template <typename T, typename U>
auto add(T a, U b) -> decltype(a + b) { return a + b; }
```

? COMMON CONFUSION

`decltype((x))` mein extra parentheses matter karte hain. `decltype(x)` `x` ka declared type deta hai (`int`); jabki `decltype((x))` `(x)` ko ek lvalue expression ki tarah treat karta hai aur `int&` deta hai.

3.1.3 `decltype(auto)`

Definition. `decltype(auto)` `auto` ki placeholder syntax ko `decltype` ki exact type-preservation ke saath combine karta hai: `decltype(auto)` declared kisi variable ya return value ka type bilkul waise deduce hota hai jaise `decltype(initialiser)` likha ho.

```cpp
int  x = 5;
int& getRef() { return x; }

auto           a = getRef();   // int   (auto drops reference)
decltype(auto) b = getRef();   // int&  (reference preserved)
b = 42;                        // modifies x itself

// Common use: forwarding return type through a wrapper
template <typename F, typename... A>
decltype(auto) invoke_ret(F f, A&&... args) {
    return f(std::forward<A>(args)...);   // returns exactly what f returns
}
```

3.1.4 Teeno ka Comparison

Feature	Reference preserved?	Const preserved?	Kahan use hota hai
`auto`	Nahi	Nahi	Variables, return types, lambdas
`decltype(e)`	Haan	Haan	Type queries, trailing return types
`decltype(auto)`	Haan	Haan	Return types ka perfect forwarding

≡ ONE-LINE REVISION

One-line revision. `auto` = deduced par stripped; `decltype(e)` = *e* ka exact type; `decltype(auto)` = deduced *aur* exact.

3.2 Compile-Time Programming — `constexpr`, `consteval`, `constinit`

3.2.1 `constexpr`

Definition. `constexpr` declare karta hai ki koi variable ya function compile time par evaluate ho *sakta* hai; use run time par bhi usable hona chahiye, aur jab arguments constant expressions hote hain tab compiler computation compile time par hi kar deta hai.

C++

```
constexpr int square(int n) { return n * n; }

constexpr int SIZE = square(5);          // computed at compile time -> 25
int a[SIZE];                             // OK: SIZE is a compile-time constant

int input;
std::cin >> input;
int r = square(input);                   // still legal: runtime evaluation
```

3.2.2 `consteval` (C++20)

Definition. `consteval` ek *immediate* function declare karta hai: use compile time par evaluate hona **hi** chahiye. Kisi `consteval` function ko aise arguments ke saath call karna jo constant expressions nahi hain, ek compile-time error hai.

C++

```
consteval int cube(int n) { return n * n * n; }

constexpr int a = cube(4);   // OK - compile-time
// int x;  std::cin >> x;
// int b = cube(x);          // ERROR: x is not a constant expression
```

★ **EXAM TIP**

`consteval` tab use karo jab function *kabhi bhi* runtime code mein leak nahi hona chahiye — maan lo jab wo koi lookup table generate karta ho ya koi compile-time invariant verify karta ho.

3.2.3 `constinit` (C++20)

Definition. `constinit` kisi static ya thread storage duration waale variable par apply hota hai; ye guarantee deta hai ki variable compile time par *constant-initialised* hua hai, jisse *static initialisation order fiasco* se bacha jaata hai. `constexpr` ke unlike, variable ka khud `const` hona zaroori nahi — aap use run time par modify kar sakte ho.

C++

```
constinit int counter = 0;   // guaranteed compile-time init
// ... later, at run time:
++counter;                    // legal - constinit does not imply const

constinit const double PI = 3.14159;   // both compile-time init AND immutable
```

3.2.4 Comparison Table

Specifier	Compile-time?	Runtime allowed?	Introduced in	Typical use
`const`	Kabhi-kabhi (const-init)	Haan	C++98	Immutable variables
`constexpr`	Ho sakta hai (agar inputs const hon)	Haan	C++11	Functions/values dono jagah usable
`consteval`	Hona hi chahiye	Nahi	C++20	Immediate functions, code generators
`constinit`	Hona hi chahiye (sirf init)	Haan (variable mutable reh sakta hai)	C++20	Statics ke liye SIOF se bachna

? COMMON CONFUSION

Common confusion. `constinit` sirf *initialisation* ke baare mein hai; ye variable ko immutable *nahi* banata. Dono chahiye to combine karo: `constinit const`.

≡ ONE-LINE REVISION

One-line revision. *const* = shayad compile-time, *constexpr* = dono, *consteval* = sirf compile-time, *constinit* = sirf compile-time init.

Ayush Jha

3.3 Modern Initialisation

3.3.1 Uniform Initialisation — Brace Syntax

Definition. Uniform (ya brace) initialisation C++11 ki wo rule hai jiske hisaab se kisi bhi type ko braces mein di gayi values ki list se initialise kiya ja sakta hai — ye built-in types, aggregates, constructors waali classes aur standard containers — sab ke liye consistent syntax deta hai.

C++

```
int         x   { 42 };
double      pi  { 3.14 };
std::string s   { "hi" };
std::vector<int> v  { 1, 2, 3 };

struct Point { int x, y; };
Point p { 10, 20 };                     // aggregate init

class Box {
    int w, h;
public:
    Box(int w_, int h_) : w(w_), h(h_) {}
};
Box b { 5, 6 };                         // constructor init
```

Benefits.

- Har jagah same syntax — padhane mein aur padhne mein dono easy.
- **Narrowing conversions** reject hoti hain. `int a { 3.7 };` ek compile-time error hai, jabki puraana `int a = 3.7;` silently chal jaata tha.
- *Most vexing parse* se bachata hai: `Widget w();` ek function declaration hai, par `Widget w{};` unambiguously ek object hai.

> **! WARNING**
>
> Agar class ke paas `std::initializer_list` constructor *aur* normal constructor dono hain, to brace initialisation `initializer_list` version ko prefer karta hai: `std::vector<int> v(10, 5)` das 5s banata hai, jabki `std::vector<int> v{10, 5}` do-element vector `{10, 5}` banata hai.

3.3.2 Designated Initialisers (C++20)

Definition. Designated initialisers (C++ mein C++20 se, C mein bahut pehle se available) aapko initialisation ke waqt har aggregate member ka *naam* likhne dete hain. Naam usi order mein hone chahiye jis order mein members aggregate mein declared hain.

C++

```
struct Config {
    std::string host;
    int         port;
    bool        secure;
};

Config c { .host = "iilm.edu", .port = 443, .secure = true };
Config d { .port = 8080 };   // only port set; host = "", secure = false
```

Rules jo yaad rakhne hain.

• Sirf **aggregates** par apply hota hai — aise structs/classes jisme sirf public data ho, koi user-declared constructor nahi, koi virtuals nahi.

• Named initialisers **declaration order** mein hone chahiye (C ke unlike).

• Missing fields value-initialised ho jaate hain (ints ke liye zero, strings ke liye empty, booleans ke liye false).

• Same braces ke andar designated aur positional mix nahi kar sakte.

★ **EXAM TIP**

Designated initialisers configuration structs ko call site par self-documenting bana dete hain — reader ko struct definition dekhe bina hi pata chal jaata hai ki har argument kya hai.

Ayush Jha

3.4 Memory Management

Modern C++ raw `new`/`delete` ki jagah do closely related ideas use karti hai: (1) **smart pointers** jo memory ko own karte hain aur use automatically release karte hain, aur (2) **move semantics** jo bina copy kiye ownership transfer karti hai. Dono mil kar beginner C++ ki almost har memory-management bug ko khatam kar dete hain.

3.4.1 Raw Pointer se Smart Pointer tak — Ownership ki Kahani

Raw pointer sirf ye batata hai ki object *kahan* hai; ye kuch nahi batata ki *use delete karne ki zimmedaari kiski hai*. Yehi ambiguity leaks aur double-frees ka source hai. Smart pointer pointer ke saath ek ownership policy bundle kar deta hai, taaki cleanup automatic aur provably correct ho jaaye.

3.4.2 `std::unique_ptr`

Definition. `std::unique_ptr<T>` ek smart pointer hai jo dynamically allocated object ki **exclusive ownership** express karta hai; jab `unique_ptr` destroy ya reset hota hai, uska owned object bhi destroy ho jaata hai. `unique_ptr` copyable nahi hota, sirf movable hota hai.

C++

```cpp
#include <memory>
#include <iostream>
using namespace std;

struct File {
    File()  { cout << "File opened\n"; }
    ~File() { cout << "File closed\n"; }
    void read() { cout << "reading...\n"; }
};

int main() {
    unique_ptr<File> f = make_unique<File>();
    f->read();
    // No delete needed - destructor of unique_ptr closes the file automatically

    unique_ptr<File> g = std::move(f);   // OK: move
    // unique_ptr<File> h = f;            // ERROR: cannot copy
    if (!f) cout << "f is now empty\n";
}
```

OUTPUT

```
File opened
reading...
f is now empty
File closed
```

3.4.3 `std::shared_ptr`

Definition. `std::shared_ptr<T>` ek smart pointer hai jo **shared ownership** express karta hai: kai `shared_ptrs` ek hi object ko point kar sakte hain, aur object sirf tab destroy hota hai jab uska aakhri `shared_ptr` scope se baahar jaata hai. Internally ye ek reference count maintain karta hai.

C++

```cpp
#include <memory>
#include <iostream>
using namespace std;

int main() {
    shared_ptr<int> a = make_shared<int>(42);
    cout << "count = " << a.use_count() << '\n';   // 1
    {
        shared_ptr<int> b = a;                     // copy - both share
        cout << "count = " << a.use_count() << '\n'; // 2
    }
    cout << "count = " << a.use_count() << '\n';   // 1 again
}
```

OUTPUT

```
count = 1
count = 2
count = 1
```

★ EXAM TIP

`shared_ptr<T>(new T(args...))` ki jagah `make_shared<T>(args...)` prefer karo. `make_shared` object aur control block dono ke liye ek hi allocation use karta hai — faster aur exception-safe.

3.4.4 `std::weak_ptr`

Definition. `std::weak_ptr<T>` ek non-owning smart pointer hai jo kisi `shared_ptr` ke manage kiye gaye object ko observe karta hai bina uski reference count badhaaye; jab access chahiye hota hai tab ye `lock()` ke through temporary `shared_ptr` mein upgrade ho jaata hai.

C++

```cpp
shared_ptr<int> sp = make_shared<int>(100);
weak_ptr<int>   wp = sp;                 // does NOT extend lifetime

if (auto locked = wp.lock()) {           // upgrade to shared_ptr temporarily
    cout << *locked << '\n';             // 100
}
sp.reset();                              // last shared_ptr gone -> object destroyed
if (wp.expired()) cout << "expired\n";  // yes
```

`weak_ptr` ki zaroorat kyon hai.

Do `shared_ptr`s jo ek-doosre ko point karein, ek **reference cycle** bana dete hain: unki counts kabhi zero tak nahi girte, aur na hi koi object kabhi destroy hota hai. Ek direction ko `weak_ptr` se replace karke cycle ko todo. Classic case hai ek `Parent` jo apne `Child`s ko own karta hai (`shared_ptr`) jabki har `Child` apne `Parent` ko waapas refer karta hai (`weak_ptr`).

3.4.5 Smart-Pointer Comparison

Property	unique_ptr	shared_ptr	weak_ptr
Ownership	Exclusive	Shared	Kuch nahi (observer)
Copyable?	Nahi	Haan	Haan
Movable?	Haan	Haan	Haan
Reference count?	Nahi	Haan	shared_ptr wali use karta hai
Overhead	Zero	Ek control block	shared jaisa hi
Typical use	Single-owner resource	Multi-owner resource	Cycles todna / observe karna

Ayush Jha

3.4.6 Rvalue References aur Move Semantics

Rvalue reference. Double ampersand `T&&` se declared ek reference jo sirf *rvalues* ko bind hota hai — temporaries aur `std::move` se explicitly cast kiye gaye objects. Rvalue references kisi function ko ye detect karne dete hain ki uska argument ek temporary hai, taaki wo copy karne ki jagah uske internal resources chura (steal) le.

Move semantics. Language rules aur library conventions ka wo set (move constructor aur move assignment operator) jisse dynamically allocated resources ki ownership ek jald-destroy-hone-waale object se doosre object mein transfer ho jaati hai, expensive copy se bach kar.

C++

```
#include <iostream>
#include <string>
using namespace std;

class Buffer {
    size_t n; int* data;
public:
    explicit Buffer(size_t n_) : n(n_), data(new int[n_]) {
        cout << "constructed (" << n << ")\n";
    }
    ~Buffer() { delete[] data; }

    // COPY constructor - expensive
    Buffer(const Buffer& o) : n(o.n), data(new int[o.n]) {
        std::copy(o.data, o.data + n, data);
        cout << "copied (" << n << ")\n";
    }
    // MOVE constructor - steals resources
    Buffer(Buffer&& o) noexcept : n(o.n), data(o.data) {
        o.n = 0; o.data = nullptr;          // leave source valid but empty
        cout << "moved\n";
    }
};

Buffer make(size_t n) { return Buffer(n); }   // returns rvalue

int main() {
    Buffer a = make(1'000'000);   // move constructor used - no big copy
}
```

OUTPUT

```
constructed (1000000)
moved
```

? COMMON CONFUSION

`std::move(x)` khud se kuch move **nahi** karta. Ye sirf `x` ko rvalue mein *cast* karta hai taaki rvalue reference lene waala overload select ho jaaye.

3.4.7 Perfect Forwarding

Definition. Perfect forwarding wo technique hai jisse function template apne arguments ko doosre function tak forward karta hai, har argument ki *value category* (lvalue ya rvalue) aur `const` qualification ko preserve karte hue — ye forwarding reference (`T&&`) aur `std::forward<T>` ko combine karke achieve hota hai.

```cpp
#include <utility>
#include <memory>

template <typename T, typename... Args>
std::unique_ptr<T> makeUnique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}

// A caller may pass lvalues, rvalues, or const references - all are forwarded
// to T's constructor with their original value category intact.
```

Rules jo yaad rakhne hain.

- `template <typename T> void f(T&& x);` mein parameter `x` ek **forwarding reference** hai, rvalue reference nahi — ye lvalues aur rvalues dono ko bind hota hai.
- Aise parameter ko relay karte waqt `std::move` nahi, `std::forward<T>(x)` use karo.
- Perfect forwarding `std::make_unique`, `std::make_shared`, `emplace_back` aur generally factories ke andar use hota hai.

3.4.8 Copy vs Move Semantics — Quick Table

Aspect	Copy semantics	Move semantics
Introduced	C++98 se	C++11
Constructor	`T(const T&)`	`T(T&&)`
Behaviour	Resources ki deep duplicate	Source se resources steal karta hai
Source object	Untouched rehta hai	Valid par unspecified state mein chhoda jaata hai
Cost	Data size ke proportional	Usually O(1)
Kab trigger hota hai	Lvalue argument se	Rvalue / `std::move` se

≡ ONE-LINE REVISION

One-line revision. *unique_ptr* = ek owner, *shared_ptr* = kai owners, *weak_ptr* = observer; *move* rvalue se steal karta hai, *forward* template arg ko unchanged relay karta hai.

3.5 Utility Library

Modern C++ ke saath ek extensive utility library aati hai. Ye section official syllabus mein listed har utility ko cover karta hai.

3.5.1 Structured Bindings (C++17)

Definition. Structured binding declaration `auto` initialisation ka wo form hai jo kisi tuple-like ya aggregate object ko ek hi statement mein individually named variables mein decompose kar deta hai.

C++

```cpp
#include <map>
#include <string>
#include <iostream>
using namespace std;

map<string, int> marks { {"Ayush", 90}, {"Bob", 82} };

for (const auto& [name, m] : marks)
    cout << name << " -> " << m << '\n';

// Also works with std::pair, std::tuple, and aggregates:
struct Point { int x, y; };
Point p{3, 4};
auto [px, py] = p;
```

OUTPUT

```
Ayush -> 90
Bob -> 82
```

3.5.2 `std::optional`

Definition. `std::optional<T>` ek wrapper hai jo ya to type `T` ki ek value rakhta hai ya bilkul koi value nahi rakhta (`std::nullopt`); isse value ke na hone ko signal karne ke liye special sentinel ya output pointer ki zaroorat khatam ho jaati hai.

C++

```cpp
#include <optional>
#include <iostream>
using namespace std;

optional<int> safeDivide(int a, int b) {
    if (b == 0) return nullopt;
    return a / b;
}

int main() {
    if (auto r = safeDivide(10, 2)) cout << *r << '\n';   // 5
    if (auto r = safeDivide(10, 0)) cout << *r << '\n';
    else                            cout << "no result\n";
}
```

OUTPUT

```
5
no result
```

3.5.3 `std::variant`

Definition. `std::variant<T1, T2, ...>` ek type-safe tagged union hai jo kisi bhi moment listed alternative types mein se exactly ek type ki ek value rakhta hai.

C++

```cpp
#include <variant>
#include <string>
#include <iostream>
using namespace std;

int main() {
    variant<int, double, string> v;
    v = 42;                       cout << get<int>(v)    << '\n';   // 42
    v = 3.14;                     cout << get<double>(v) << '\n';   // 3.14
    v = string("hello");
    // visit picks the correct handler at run time
    visit([](auto&& x){ cout << x << '\n'; }, v);                  // hello
}
```

OUTPUT

```
42
3.14
hello
```

3.5.4 `std::any`

Definition. `std::any` **kisi bhi** copyable type ki value rakhta hai, apna runtime type yaad rakhte hue; value nikaalne ke liye `std::any_cast<T>()` call karna padta hai jo `T` match na hone par exception ke saath fail hota hai.

C++

```cpp
#include <any>
#include <iostream>
#include <string>
using namespace std;

int main() {
    any a = 10;
    cout << any_cast<int>(a) << '\n';       // 10
    a = string("hi");
    cout << any_cast<string>(a) << '\n';    // hi
    try { any_cast<int>(a); }
    catch (bad_any_cast&) { cout << "wrong type\n"; }
}
```

OUTPUT

```
10
hi
wrong type
```

? COMMON CONFUSION

optional vs variant vs any. *optional* = ek type ya kuch nahi; *variant* = fixed types ke set mein se ek; *any* = kisi bhi type ka ek. Problem ke hisaab se sabse specific tool prefer karo.

3.5.5 `std::pair`

Definition. `std::pair<A, B>` ek class template hai jo possibly-different types ke exactly do elements rakhta hai, jinhe `.first` aur `.second` se access kiya jaata hai.

C++

```cpp
#include <utility>
#include <string>
#include <iostream>
using namespace std;

pair<string, int> best {"Ayush", 98};
cout << best.first << " -> " << best.second << '\n';

auto [name, score] = best;              // structured binding
cout << name << ' ' << score << '\n';
```

3.5.6 `std::tuple`

Definition. `std::tuple<T1, T2, ..., Tn>` ek class template hai jo `std::pair` ko kisi bhi fixed number ke heterogeneous elements tak generalise karta hai.

C++

```cpp
#include <tuple>
#include <string>
#include <iostream>
using namespace std;

tuple<string, int, double> t {"Ayush", 20, 8.9};
cout << get<0>(t) << ' ' << get<1>(t) << ' ' << get<2>(t) << '\n';

auto [name, age, cgpa] = t;
cout << name << ' ' << age << ' ' << cgpa << '\n';
```

3.5.7 `std::span` (C++20)

Definition. `std::span<T>` kisi existing contiguous sequence of `T` ke upar ek non-owning view hota hai — array, vector ya doosra span. Ye ek pointer aur ek size carry karta hai aur bina data copy kiye uniform STL-style interface deta hai.

```cpp
#include <span>
#include <vector>
#include <array>
#include <iostream>
using namespace std;

void printAll(span<const int> s) {
    for (int x : s) cout << x << ' ';
    cout << '\n';
}

int main() {
    vector<int>   v { 1, 2, 3, 4, 5 };
    array<int, 3> a { 6, 7, 8 };
    int raw[] = { 9, 10 };

    printAll(v);          // 1 2 3 4 5
    printAll(a);          // 6 7 8
    printAll(raw);        // 9 10
}
```

3.5.8 `std::string_view`

Definition. `std::string_view` characters ke contiguous sequence ke upar ek lightweight, non-owning view hota hai, jo `std::string` API ka read-only hissa bina allocate ya copy kiye deta hai.

```cpp
#include <string_view>
#include <iostream>
using namespace std;

void greet(string_view name) {        // no copy, no allocation
    cout << "Hello, " << name << "!\n";
}

int main() {
    greet("Ayush");                   // literal
    string s = "Genspark";
    greet(s);                         // std::string
    greet(s.substr(0, 3));            // still a view
}
```

> **! WARNING**
>
> Kabhi bhi aisa `string_view` return mat karo jo kisi temporary `std::string` ke andar point kar raha ho — underlying storage tab tak ja chuki ho sakti hai.

3.5.9 `std::filesystem`

Definition. `<filesystem>` header file system ke saath interact karne ka ek portable API deta hai: paths banana, directories par iterate karna, file status query karna, aur files aur directories create, copy ya remove karna.

```cpp
#include <filesystem>
#include <iostream>
namespace fs = std::filesystem;
using namespace std;

int main() {
    fs::path p = "./notes";
    fs::create_directories(p);           // makes directory (if needed)

    for (const auto& e : fs::directory_iterator(".")) {
        cout << (e.is_directory() ? "[D] " : "[F] ") << e.path() << '\n';
    }
    cout << "cwd = " << fs::current_path() << '\n';
}
```

3.5.10 `std::chrono`

Definition. `<chrono>` library durations, time points aur clocks ko strongly-typed tarike se represent karti hai, taaki compiler aapko galti se milliseconds ko seconds mein add karne se rok sake.

```cpp
#include <chrono>
#include <thread>
#include <iostream>
using namespace std;
using namespace std::chrono;

int main() {
    auto start = steady_clock::now();
    this_thread::sleep_for(500ms);       // literal 500 milliseconds
    auto end   = steady_clock::now();

    auto elapsed = duration_cast<milliseconds>(end - start);
    cout << "Slept about " << elapsed.count() << " ms\n";
}
```

OUTPUT

```
Slept about 500 ms
```

★ **EXAM TIP**

Durations measure karne ke liye `steady_clock` use karo; `system_clock` wall-clock timestamps ke liye hai aur OS clock adjust hone par peeche jump kar sakta hai.

3.6 Other C++20 Features

3.6.1 Three-Way Comparison Operator <=> (Spaceship, C++20)

Definition. Three-way comparison operator `<=>` — jise informally *spaceship operator* kehte hain — ek hi call mein apne dono operands ki ordering relationship evaluate karta hai aur ek comparison-category object return karta hai (`strong_ordering`, `weak_ordering` ya `partial_ordering`), jisse `<`, `<=`, `==`, `!=`, `>`, aur `>=` — ye chhah ke chhah operators automatically derive ho jaate hain.

C++

```cpp
#include <compare>
#include <iostream>
using namespace std;

struct Version {
    int major, minor, patch;
    auto operator<=>(const Version&) const = default;   // all six ops generated
};

int main() {
    Version a{1, 4, 0}, b{1, 5, 2};
    cout << boolalpha
         << (a  < b) << ' '   // true
         << (a == b) << ' '   // false
         << (b  > a);         // true
}
```

OUTPUT

```
true false true
```

◆ REMEMBER

C++20 se pehle aapko chhah overloaded operators likhne padte the. `<=>` par `= default` likhne se ek hi line mein saare mil jaate hain.

3.6.2 [[nodiscard]]

Definition. Jab `[[nodiscard]]` kisi function ya return type par apply hota hai, to caller agar returned value ko discard karta hai to compiler warning deta hai.

C++

```cpp
[[nodiscard]] int compute();

int main() {
    compute();          // warning: return value of 'compute()' discarded
    int x = compute();  // OK
}
```

3.6.3 [[likely]] aur [[unlikely]]

Definition. Attributes `[[likely]]` aur `[[unlikely]]` kisi statement ya branch ko expected ya unexpected path ke roop mein mark karte hain, jisse compiler apni branch layout ko bias kar sakta hai aur processor ko hint de sakta hai.

```cpp
int process(int n) {
    if (n < 0) [[unlikely]] return -1;   // error path
    /* ... normal work ... */             [[likely]];
    return 0;
}
```

★ **EXAM TIP**

In annotations ko har jagah mat chhido — ye sirf hints hain. Sirf wahan lagao jahan imbalance sach mein bada ho aur measurement se benefit dikh raha ho.

3.6.4 `[[deprecated]]`

Definition. `[[deprecated("reason")]]` kisi entity ko deprecated mark karta hai: us entity ke uses ab bhi compile hote hain par compiler warning aati hai jisme diya gaya reason hota hai.

```cpp
[[deprecated("use hashPassword() instead")]]
std::string md5(const std::string&);
```

3.6.5 `enum class` (Scoped Enumeration)

Definition. `enum class` — jise *scoped enumeration* bhi kehte hain — ek strongly-typed enumeration hai jiske enumerators integers mein implicitly convert nahi hote aur jinke names enumeration ke andar hi scoped rehte hain.

```cpp
enum class Color { Red, Green, Blue };
enum class Traffic { Red, Yellow, Green };   // no clash with Color::Red

Color c = Color::Red;
// int  n = c;                 // ERROR: no implicit conversion
int    n = static_cast<int>(c);

if (c == Color::Red) { /* ... */ }
// if (c == Traffic::Red) { }  // ERROR: different enum types
```

Classic `enum` ke muqaable advantages.

- `int` mein koi implicit conversion nahi — type errors jaldi pakad mein aate hain.
- Enumerator names **scoped** hote hain — `Color::Red` aur `Traffic::Red` takraate nahi.
- Underlying type specify kar sakte ho: `enum class Flag : uint8_t { On, Off };`

3.6.6 Standard Attributes Cheat Sheet

Attribute	Purpose
`[[nodiscard]]`	Return value throw away hone par warning.
`[[maybe_unused]]`	Conditionally used entities ke liye “unused” warnings chhupata hai.
`[[likely]]`	Compiler ko hint ki branch usually li jaati hai.
`[[unlikely]]`	Compiler ko hint ki branch rarely li jaati hai.
`[[deprecated("msg")]]`	Kisi entity ke future use ko discourage karta hai.
`[[fallthrough]]`	Switch case ke andar fall-through warning chhupata hai.
`[[noreturn]]`	Aise function ko mark karta hai jo kabhi return nahi karta (e.g. `std::exit` call karta ho).

≡ ONE-LINE REVISION

One-line revision. <=> chhah comparison operators ki jagah leta hai; *enum class* unsafe C-style enums ki jagah leta hai; attributes double square brackets mein likhe jaane waale hints hain.

Ayush Jha

Unit III — Revision Sheet

Ye sheet Unit III ko us material mein compress karti hai jiske examination mein aane ki sambhaavna sabse zyada hai. Har C++ programmer ko neeche ke har question ka answer do minute ke andar aana chahiye.

Unit III Summary

Modern C++20 language ke har corner ko refine karta hai: types `auto` aur `decltype` se deduce hote hain, computations `constexpr` family ke saath compile time par chali jaati hain, memory smart pointers se automatically manage hoti hai, ownership move semantics se sasti tarah transfer hoti hai, aur generic code apne arguments ko perfectly forward karta hai. Core language ke upar, utility library `optional`, `variant`, `any`, `span`, `string_view`, `filesystem` aur `chrono` deti hai; spaceship operator chhah ke chhah comparison operators generate karta hai; scoped `enum class` enumerations ko type-safe banata hai; aur standard attributes compiler ko intent communicate karte hain.

Ayush Jha

Important Definitions

Term	Ek sentence mein definition
auto	Placeholder jo compiler ko variable ka type uske initialiser se deduce karne ko kehta hai.
decltype	Compile-time operator jo expression ka exact type deta hai, references aur const including.
decltype(auto)	Placeholder jo decltype rules se type deduce karta hai — references aur const preserve karta hai.
constexpr	Specifier jo compile-time evaluation allow karta hai, run time par bhi usable rehte hue.
consteval	Specifier jo compile-time evaluation require karta hai — immediate function.
constinit	Specifier jo compile-time initialisation require karta hai (variable run time par mutable reh sakta hai).
Uniform init.	Brace-enclosed initialisation syntax jo saare types par uniformly kaam karta hai aur narrowing forbid karta hai.
Designated init.	C++20 aggregate initialisation jisme har member ka naam likha jaata hai (declaration order mein).
unique_ptr	Smart pointer jo exclusive ownership express karta hai; movable hai par copyable nahi.
shared_ptr	Smart pointer jo reference count ke through shared ownership express karta hai.
weak_ptr	Non-owning observer jo lock() ke through shared_ptr mein upgrade hota hai.
Rvalue reference	Reference (T&&) jo sirf rvalues ko bind hota hai; move semantics enable karta hai.
Move semantics	Resources ko jald-destroy-hone-waale object se doosre mein transfer karna, copy se bach kar.
Perfect forwarding	Template technique jo har argument ko uski original value category ke saath relay karti hai.
optional	Aisa T ka wrapper jisme value ho bhi sakti hai aur nahi bhi.
variant	Fixed alternative types ki list ka type-safe tagged union.
any	Container jo kisi bhi copyable type ki value rakhta hai.
span	Kisi sequence ke upar non-owning contiguous view.
string_view	Character data ke upar non-owning read-only view.
chrono	Strongly-typed duration/time-point library.
Spaceship	<=> — three-way comparison jo chhah ke chhah comparison operators generate karta hai.
enum class	Strongly-typed, scoped enumeration.

Important Syntax

```C++
// Type deduction
auto           n = 42;
decltype(n)    m = n;
decltype(auto) r = getRef();

// constexpr family
constexpr int  sq(int n) { return n*n; }
consteval int  cube(int n){ return n*n*n; }
constinit int  counter = 0;

// Uniform / designated
std::vector<int> v { 1, 2, 3 };
struct P{int x,y;}; P p{ .x=1, .y=2 };

// Smart pointers
auto up = std::make_unique<T>(args...);
auto sp = std::make_shared<T>(args...);
std::weak_ptr<T> wp = sp;

// Move / forward
Buf b = std::move(a);
template<class F,class...A> decltype(auto)
call(F f, A&&... a){ return f(std::forward<A>(a)...); }

// Utility library
std::optional<int> o = 5;
std::variant<int,std::string> v = "hi";
std::any a = 3.14;
std::span<const int> s(vec);
std::string_view sv = "hello";

// C++20 extras
struct V{int x; auto operator<=>(const V&) const = default;};
[[nodiscard]] int compute();
enum class Color { Red, Green };
```

Important Comparisons

Comparison	Ek line mein answer
auto vs decltype vs decltype(auto)	auto ref/const strip karta hai; decltype expression se exactly match karta hai; decltype(auto) = auto with decltype rules.
constexpr vs consteval vs constinit	constexpr = compile time par ho sakta hai; consteval = hona hi chahiye; constinit = sirf compile-time init.
unique_ptr vs shared_ptr vs weak_ptr	Exclusive vs shared vs observing ownership.
Copy vs Move semantics	Copy duplicate karta hai; move rvalue source se ownership transfer karta hai.
optional vs variant vs any	Zero-ya-ek; fixed-set mein se ek; kisi bhi type ka ek.
string vs string_view	string storage own karta hai; string_view sirf observe karta hai.
enum vs enum class	Classic enum names leak karta hai aur int mein convert ho jaata hai; enum class scoped aur strongly typed hai.

Expected Exam Questions

- `auto`, `decltype` aur `decltype(auto)` mein difference examples ke saath explain karo.
- `constexpr`, `consteval` aur `constinit` compare karo. Har ek ka ek example do.
- Smart pointer kya hota hai? `unique_ptr`, `shared_ptr` aur `weak_ptr` ko programs ke saath explain karo.
- Move semantics aur rvalue references ko aisi class ke saath explain karo jo dynamically allocated buffer own karti ho.
- `makeUnique` template ke saath perfect forwarding illustrate karo.
- `std::optional`, `std::variant` aur `std::any` ko programs ke saath explain karo.
- Spaceship operator comparison operator overloading ko kaise simplify karta hai? Example do.
- Classical `enum` aur `enum class` mein difference batao.
- Koi teen standard attributes syntax ke saath explain karo.

Practice Questions (Graded)

Level	Task
★	Ek program likho jo `auto` aur range-based `for` se vector ke har element ko print kare.
★★	`std::optional<double>` return karne waala `safeDivide` function implement karo.
★★★	Ek chhoti resource class design karo jo move constructor aur move assignment operator demonstrate kare; dikhao ki rvalues ke liye copy version select nahi hota.
★★★★	Ek `Config` aggregate implement karo jo designated initialisers se initialise ho aur structured bindings se print ho.
★★★★★	Ek simple `Node`/`Tree` banao jisme har node apne children ko `shared_ptr` se own kare aur parent ko `weak_ptr` se refer kare; dikhao ki tree correctly destroy hota hai.

! DO NOT FORGET

Do not forget. `make_unique` / `make_shared` prefer karo; application code mein kabhi raw `new` mat call karo. Range-based fors mein `const auto&` prefer karo. Har nayi enumeration ke liye `enum class` prefer karo. Narrowing-safety ke liye brace initialisation prefer karo.

UNIT IV

Object-Oriented Design Patterns

Creational · Structural · Behavioural

Unit IV — Object-Oriented Design Patterns

Ye unit kis baare mein hai

Unit IV us reusable design vocabulary ko catalogue karta hai jo professional object-oriented programmers aapas mein share karte hain. **Design pattern** kisi baar-baar aane wali problem ke baar-baar use hone waale solution ko ek naam deta hai, taaki engineers use har baar reinvent kiye bina discuss aur apply kar sakein. Syllabus mein classic *Gang of Four* ke das patterns teen families mein divide karke cover hote hain: **Creational** patterns (objects kaise bante hain), **Structural** patterns (objects kaise compose hote hain) aur **Behavioural** patterns (objects aapas mein kaise communicate karte hain).

Ye unit kyon important hai

Design patterns kisi bhi technical interview ya viva mein engineering maturity ka sabse reliable indicator hote hain. Pattern ka naam batana, uska intent ek sentence mein samjhana, aur ek chhota sahi C++ example likhna — yehi "theek-thaak" aur "selected" ke beech ka difference hai. Aage jo bhi professional codebase aap padhoge, wo inhi patterns se bana hoga — explicitly ya implicitly.

Prerequisites

- Solid Unit I — virtual functions, abstract classes, dynamic binding aur SOLID.
- Solid Unit III — smart pointers, move semantics, structured bindings.
- Base pointers aur references ke through polymorphism se comfort.

Learning objectives

- *Design pattern* define karna, aur diye gaye pattern ko creational, structural ya behavioural mein classify karna.
- Chaar Creational patterns (Factory, Singleton, Builder, Prototype) ko C++ mein reproduce karna.
- Teen Structural patterns (Adapter, Decorator, Facade) ko C++ mein reproduce karna.
- Teen Behavioural patterns (Strategy, Observer, State) ko C++ mein reproduce karna.
- Batana ki koi pattern kab appropriate hai aur kab *nahi*.

Ayush Jha

4.1 Design Pattern kya hota hai?

Definition. Design pattern software design ki kisi recurring problem ka ek named, generalised, reusable solution hota hai; ye koi finished code ka tukda nahi hota, balki ye description hota hai ki thodi si classes aur objects ko kaise arrange kiya jaaye taaki problem wide range of situations mein cleanly solve ho.

4.1.1 Design Patterns use kyon kiye jaate hain

• Ye ek **common vocabulary** dete hain: “singleton” bolte hi poori class structure turant communicate ho jaati hai.

• Ye **proven solutions** document karte hain: patterns real projects ke dashkon ke trial-and-error se aate hain; unhe use karne se wahi galtiyaan dobara dhoondhne se bache rehte hain.

• Ye **extensibility** badhaate hain: achhi tarah chuna gaya pattern nayi requirements ko existing code badle bina add karne deta hai.

• Ye code ko doosre engineers ke liye **padhna aasaan** bana dete hain, kyunki sab wahi vocabulary share karte hain.

4.1.2 Common Misconceptions

• **Patterns libraries nahi hote.** Pattern ek design idea hai; code aapko khud likhna padta hai.

• **Patterns ready-made copy-and-paste code nahi hote.** Har use pattern ko apne domain ke hisaab se adapt karta hai.

• **Zyada patterns ka matlab better design nahi hota.** Patterns over-use karne se code unnecessarily complicated ban jaata hai.

• **Har problem ko pattern ki zaroorat nahi hoti.** Agar plain class problem solve kar rahi hai, to plain class hi use karo.

4.1.3 Patterns ki Teen Categories

Category	Focus	Is syllabus ke patterns
Creational	Objects **kaise bante** hain	Factory, Singleton, Builder, Prototype
Structural	Objects **kaise compose** hote hain	Adapter, Decorator, Facade
Behavioural	Objects **kaise collaborate aur communicate** karte hain	Strategy, Observer, State

4.1.4 Programmer ki Tarah Patterns ke Baare Mein Kaise Sochein

Har problem ke saamne teen questions poochho:

• **Pain kis layer mein hai?** Object creation mein, object structure mein, ya object communication mein?

• **Change ka axis kya hai?** Kya naye products aane waale hain (Factory)? Kya behaviours swap honge (Strategy)? Kya system ke ek hisse ko kai doosron ko notify karna hoga (Observer)?

• **Kya ye pattern baad mein remove karna bhi easy rahega?** Pattern ka achha use code ko change karna aasaan banata hai; bura use use impossible bana deta hai.

★ **MEMORY TRICK**

Memory trick. Teeno categories mil kar **C-S-B** banati hain — *Create, Structure, Behave*.

≡ **ONE-LINE REVISION**

One-line revision. Design pattern ek named, reusable design idea hota hai — na code, na library, na silver bullet.

4.2 Creational Design Patterns

Creational patterns ka concern hota hai ki objects instantiate kaise hote hain. Ye construction ki details ko chhupa dete hain, taaki client code concrete classes par nahi, abstractions par depend kare.

4.2.1 Factory Pattern

Category. Creational

> **Definition.** Factory pattern object banane ke liye ek interface (ek static ya virtual function) define karta hai, par ye subclasses ya ek central function par chhod deta hai ki kaunsi concrete class instantiate hogi — isse client code sirf abstract product interface par depend karta hai.

Problem. Jo client code har jagah `new ConcreteType(...)` likhta hai, wo `ConcreteType` se tightly bound ho jaata hai. Koi naya type add karne par poori codebase mein scattered edits karni padti hain.

Intent. Object creation ko ek hi point of choice ke peechhe encapsulate karna, taaki naya product add karne par sirf wahi point edit karna pade.

Analogy. Pizza restaurant: customer “pepperoni” order karta hai aur kitchen (factory) decide karti hai ki kaunsi recipe use hogi. Customer ko ovens kabhi dikhte hi nahi.

Participants.

- **Product** — wo abstract interface jo client ko return hota hai (e.g. Shape).
- **ConcreteProduct** — har concrete class jo Product implement karti hai (Circle, Square...).
- **Factory** — wo static ya member function jo Product* / unique_ptr return karta hai.

C++ implementation.

Ayush Jha

C++

```cpp
#include <iostream>
#include <memory>
#include <string>
using namespace std;

class Shape {
public:
    virtual void draw() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
    void draw() const override { cout << "Drawing Circle\n"; }
};
class Square : public Shape {
    void draw() const override { cout << "Drawing Square\n"; }
};

class ShapeFactory {
public:
    static unique_ptr<Shape> create(const string& kind) {
        if (kind == "circle") return make_unique<Circle>();
        if (kind == "square") return make_unique<Square>();
        return nullptr;
    }
};

int main() {
    auto s1 = ShapeFactory::create("circle");
    auto s2 = ShapeFactory::create("square");
    s1->draw();
    s2->draw();
}
```

OUTPUT

```
Drawing Circle
Drawing Square
```

Advantages.

- Client sirf **Product** interface par depend karta hai, concrete classes par nahi.
- Naya product add karne par sirf factory ko edit karna padta hai.
- Unit I ke **Open/Closed Principle** ko support karta hai.

Disadvantages.

- Factory ko har product ke baare mein pata hona chahiye; ye ek natural bottleneck ban jaata hai.
- Client aur product ke beech ek extra indirection add hoti hai.

Kab use karein. Jab client code ko objects banane hain par use ye nahi pata hona chahiye ki kaunsi concrete class choose karni hai; aur products ka set grow karne waala ho.

Kab use na karein. Jab sirf ek hi product class ho; direct construction simpler hai.

✎ **EXAM POINT**

Exam-style answer. Factory object creation ko ek single function ke peechhe encapsulate karta hai, taaki clients sirf abstract Product interface par depend karein. Shape hierarchy aur `ShapeFactory::create(kind)` static method ke saath illustrate karo.

? **COMMON CONFUSION**

Comparison. Factory vs Prototype. Factory har product ko scratch se banata hai; Prototype kisi existing configured object ko clone karta hai.

Ayush Jha

4.2.2 Singleton Pattern

Category. Creational

> **Definition.** Singleton pattern ensure karta hai ki kisi class ka program ki poori lifetime mein exactly ek hi instance ho, aur us instance tak pahunchne ka ek global access point provide karta hai.

Problem. Kuch resources — logger, configuration store, OS-level device handle — exactly ek hi object ke roop mein exist karni chahiye; multiple instances ya to state corrupt kar denge ya resources waste karenge.

Intent. Uniqueness guarantee karna aur controlled access dena.

Analogy. Kisi desh ka Prime Minister: kisi bhi time par exactly ek hi hota hai, aur jo bhi government se baat karna chahta hai wo usi ek office ke through jaata hai.

Participants.

- **Singleton** — khud class, jiska constructor private hota hai.
- **instance()** — static member function jo sole instance ko reference se return karta hai.

C++ implementation.

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

class Logger {
    Logger() = default;                    // private constructor
    Logger(const Logger&)            = delete;   // no copy
    Logger& operator=(const Logger&) = delete;
public:
    static Logger& instance() {            // Meyers singleton (thread-safe since C++11)
        static Logger inst;
        return inst;
    }
    void log(const string& msg) { cout << "[LOG] " << msg << '\n'; }
};

int main() {
    Logger::instance().log("system started");
    Logger::instance().log("shutting down");
}
```

OUTPUT

```
[LOG] system started
[LOG] shutting down
```

Advantages.

- Single instance guarantee karta hai aur global access point deta hai.
- C++11 ke baad se function-local `static` thread-safe tarah se initialise hota hai — classic “Meyers singleton” by default safe hai.

Disadvantages.

- Hidden global state introduce karta hai, jisse unit testing mushkil ho jaati hai.
- Ek specific concrete class ke saath coupling encourage karta hai; **Dependency Inversion Principle** ke saath clash karta hai.
- Tests mein mocks se substitute karna ya tests ke beech reset karna mushkil hota hai.

Kab use karein. Jab exactly ek instance hone ki koi genuine hardware ya system wajah ho (device driver, logging framework, resource cache).

Kab use na karein. Jab aap Singleton ko sirf convenient global ki tarah use karne ka soch rahe ho; uski jagah dependency injection consider karo.

✎ **EXAM POINT**

Exam-style answer. Singleton private constructor aur static `instance()` method ke through class ke exactly ek instance ki guarantee deta hai. Logger class ke saath illustrate karo. Batana ki Meyers singleton (function-local static) C++11 se thread-safe hai.

? **COMMON CONFUSION**

Comparison. Singleton vs Global variable. Singleton lazy-initialise hota hai, class ke andar encapsulated hota hai, aur subclass se replace kiya ja sakta hai; global variable inmein se kuch nahi hoti.

Ayush Jha

4.2.3 Builder Pattern

Category. Creational

> **Definition.** Builder pattern kisi complex object ki construction ko uski final representation se alag karta hai, taaki wahi construction process chhote, well-named steps ko chain karke different representations produce kar sake.

Problem. Jis class mein bahut saare optional parameters hote hain, use ya to bahut lamba constructor chahiye (call site par unreadable) ya setter calls ki ek series (jismein se koi bhi bhool sakta hai). Dono error-prone hain.

Intent. Object ko step-by-step build karna ek fluent interface ke through, jo `build()` call ke saath end hota hai.

Analogy. Burger restaurant par order karna: aap counter ko batate ho *bun*, phir *patty*, phir *cheese*, phir *sauces*. Counter burger assemble kar deta hai. Aap raw ingredients ko khud kabhi haath nahi lagaate.

Participants.

- **Product** — wo complex object jo build karna hai (e.g. Pizza).
- **Builder** — wo class jiske methods parts add karte hain aur finally Product return karte hain.
- **Director** — (optional) wo class jo builder calls ke sequence ko orchestrate karti hai.

C++ implementation.

C++

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Pizza {
public:
    string base;
    vector<string> toppings;
    void print() const {
        cout << "Pizza base=" << base << ", toppings=";
        for (const auto& t : toppings) cout << t << ' ';
        cout << '\n';
    }
};

class PizzaBuilder {
    Pizza p;
public:
    PizzaBuilder& setBase(const string& b)    { p.base = b;                return *this; }
    PizzaBuilder& addTopping(const string& t) { p.toppings.push_back(t);   return *this; }
    Pizza build()                             { return std::move(p); }
};

int main() {
    Pizza pizza = PizzaBuilder()
        .setBase("thin crust")
        .addTopping("mozzarella")
        .addTopping("mushrooms")
        .addTopping("olives")
        .build();
    pizza.print();
}
```

OUTPUT

```
Pizza base=thin crust, toppings=mozzarella mushrooms olives
```

Advantages.

- Lambe constructor ko self-documenting steps ki series mein badal deta hai.
- End product immutable rehta hai: client koi step bhool nahi sakta kyunki `build()` finished object ko validate karta hai.
- Wahi builder interface extend karne par multiple representations produce kar sakta hai.

Disadvantages.

- Simple objects ke liye plain constructor se zyada code likhna padta hai.
- Har product ke liye ek extra class add hoti hai.

Kab use karein. Jab product ke paas kayi optional parts hon, ya construction ka ek specific order ho jise enforce karne ki zimmedaari clients par nahi honi chahiye.

Kab use na karein. Jab product ke ek-do parameters hon aur construction straightforward ho.

✎ **EXAM POINT**

Exam-style answer. Builder complex object ki step-by-step construction ko uski representation se alag karta hai — fluent methods ki chain jo `build()` par end hoti hai, use karke. Pizza / PizzaBuilder ke saath illustrate karo.

? **COMMON CONFUSION**

Comparison. Builder vs Factory. Factory poora product ek hi call mein banata hai; Builder use kai named steps mein banata hai.

Ayush Jha

4.2.4 Prototype Pattern

Category. Creational

> **Definition.** Prototype pattern naye objects ko constructors call karne ki jagah kisi existing, already-configured instance ko **clone** karke banata hai — isse client code objects ko bina unke concrete types jaane duplicate kar sakta hai.

Problem. Kuch objects scratch se banane mein expensive hote hain, ya unki exact class client ko pata hi nahi hoti (sirf base pointer available hota hai). Already configured prototype ko copy karna faster aur simpler hota hai.

Intent. Polymorphic hierarchy par ek `clone()` operation provide karna, taaki clients base pointer ke through objects duplicate kar sakein.

Analogy. Rubber stamp: har baar wahi logo haath se banane ki jagah, aap pehle se inked stamp ko bas press kar dete ho.

Participants.

- **Prototype** — abstract base class jo `clone()` declare karti hai.
- **ConcretePrototype** — har concrete class jo apni copy return karti hai.
- **Client** — wo code jo prototypes ko bina unke real types jaane clone karta hai.

C++ implementation.

C++

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

class Shape {
public:
    virtual unique_ptr<Shape> clone() const = 0;
    virtual void draw() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
    int radius;
public:
    explicit Circle(int r) : radius(r) {}
    unique_ptr<Shape> clone() const override { return make_unique<Circle>(*this); }
    void draw() const override { cout << "Circle r=" << radius << '\n'; }
};

class Square : public Shape {
    int side;
public:
    explicit Square(int s) : side(s) {}
    unique_ptr<Shape> clone() const override { return make_unique<Square>(*this); }
    void draw() const override { cout << "Square s=" << side << '\n'; }
};

int main() {
    vector<unique_ptr<Shape>> proto;
    proto.push_back(make_unique<Circle>(5));
    proto.push_back(make_unique<Square>(3));

    // Clone every prototype - client does NOT know the real class
    for (const auto& p : proto) p->clone()->draw();
}
```

OUTPUT

```
Circle r=5
Square s=3
```

Advantages.

- Objects ko bina unki concrete class jaane duplicate karta hai.
- Expensive initialisation ko bypass karta hai.
- Polymorphic hierarchies aur covariant return types ke saath achha kaam karta hai (dekho §1.6.4).

Disadvantages.

- Har concrete class ko `clone()` implement karna hi padta hai.
- Deep vs shallow copy ek subtle decision hai jo har class ke liye document karni padti hai.

Kab use karein. Jab objects construct karne mein expensive hon, ya client ko base pointer ke through copy karni ho bina derived type jaane.

Kab use na karein. Jab objects construct karne mein cheap hon aur unke concrete types client ko pata hon.

✎ EXAM POINT

Exam-style answer. Prototype virtual `clone()` method ke through kisi existing prototype ko clone karke naye objects banata hai. Covariant / unique_ptr return types use karne waali Shape hierarchy ke saath illustrate karo.

? COMMON CONFUSION

Comparison. Factory vs Prototype. Factory: *naya banao*. Prototype: *isi ki copy banao*.

4.3 Structural Design Patterns

Structural patterns ka concern hota hai ki classes aur objects ko kaise compose kiya jaaye taaki bade structures banein, aur composition flexible aur efficient bhi rahe.

4.3.1 Adapter Pattern

Category. Structural

> **Definition.** Adapter pattern kisi existing class (jise *Adaptee* kehte hain) ke interface ko us interface mein convert karta hai jo client expect karta hai (jise *Target* kehte hain), taaki aisi classes jinke interfaces otherwise incompatible hain, wo saath milkar kaam kar sakein.

Problem. Koi existing class sahi kaam to karti hai par galat function names ya parameter shapes ke saath; aur client code ko change nahi kiya ja sakta.

Intent. Adaptee ko ek nayi class ke andar wrap karna jo expected interface present karti hai aur internally delegate karti hai.

Analogy. Power-plug adapter: Indian device round pin ke liye bana hota hai; UK socket ke pins rectangular hote hain. Adapter device ki taraf round-pin face dikhata hai aur apni inner side se UK wall mein plug hota hai.

Participants.

- **Target** — wo interface jo client use karta hai.
- **Adaptee** — wo existing incompatible class.
- **Adapter** — wo wrapper jo Target implement karta hai aur ek Adaptee rakhta hai.
- **Client** — wo code jo Target se baat karta hai.

C++ implementation.

Ayush Jha

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

// The interface the client wants to use.
class MediaPlayer {
public:
    virtual void play(const string& file) = 0;
    virtual ~MediaPlayer() = default;
};

// An existing (incompatible) library we cannot modify.
class LegacyAudio {
public:
    void playMP3(const string& path) { cout << "playing MP3 " << path << '\n'; }
};

// Adapter: presents MediaPlayer face, delegates to LegacyAudio.
class LegacyAudioAdapter : public MediaPlayer {
    LegacyAudio legacy;
public:
    void play(const string& file) override { legacy.playMP3(file); }
};

int main() {
    LegacyAudioAdapter adapter;
    MediaPlayer* player = &adapter;
    player->play("song.mp3");
}
```

OUTPUT

```
playing MP3 song.mp3
```

Advantages.

- Legacy ya third-party code ko bina modify kiye re-use karne deta hai.
- Client code ko specific existing class se decouple karta hai.

Disadvantages.

- Indirection ki ek extra layer add hoti hai.
- Proliferate ho sakta hai: har incompatible pair ke liye ek adapter.

Kab use karein. Jab aapko kisi library ya legacy class ko integrate karna ho jiska interface aapke client ki expectations se match nahi karta.

Kab use na karein. Jab interface ke dono sides aapke control mein hon — mismatch ko directly fix karo.

✎ EXAM POINT

Exam-style answer. Adapter existing class ke interface ko us interface mein convert karta hai jo client expect karta hai — existing class ko ek nayi class ke andar wrap karke jo target interface implement karti hai. MediaPlayer / LegacyAudio wrapper ke saath illustrate karo.

? COMMON CONFUSION

Comparison. Adapter vs Decorator. Adapter interface *badalta* hai; Decorator interface *waisa hi rakhta* hai par behaviour add karta hai.

4.3.2 Decorator Pattern

Category. Structural

> **Definition.** Decorator pattern kisi object ke saath additional responsibilities ko dynamically attach karta hai — use ek doosre object ke andar wrap karke jo wahi interface implement karta hai; behaviour extend karne ke liye ye subclassing ka ek flexible alternative hai.

Problem. Aap individual objects mein responsibilities add karna chahte ho (extra logging, compression, encryption…) par har combination ke liye subclasses se class hierarchy explode nahi karna chahte.

Intent. Objects ko layered decorators mein wrap karke behaviour compose karna — har decorator wahi interface implement karta hai aur calls ko apne wrapped object ko forward karta hai.

Analogy. Coffee order karna: *Espresso* se shuru karo, use *Milk* mein wrap karo, phir *Sugar* mein. Har wrapper cost aur description add karta hai — bina koi nayi class "MilkAndSugarEspresso" banaye.

Participants.

- **Component** — abstract interface.
- **ConcreteComponent** — base implementation.
- **Decorator** — abstract wrapper jo Component implement karta hai aur ek Component rakhta hai.
- **ConcreteDecorator** — wo classes jo specific responsibilities add karti hain.

C++ implementation.

Ayush Jha

C++

```cpp
#include <iostream>
#include <memory>
#include <string>
using namespace std;

class Beverage {
public:
    virtual string describe() const = 0;
    virtual double cost() const     = 0;
    virtual ~Beverage() = default;
};

class Espresso : public Beverage {
public:
    string describe() const override { return "Espresso"; }
    double cost() const     override { return 50; }
};

// Abstract decorator: holds and forwards to a Beverage.
class BeverageDecorator : public Beverage {
protected:
    unique_ptr<Beverage> inner;
public:
    explicit BeverageDecorator(unique_ptr<Beverage> b) : inner(std::move(b)) {}
};

class Milk : public BeverageDecorator {
public:
    using BeverageDecorator::BeverageDecorator;
    string describe() const override { return inner->describe() + " + milk"; }
    double cost() const     override { return inner->cost() + 10; }
};
class Sugar : public BeverageDecorator {
public:
    using BeverageDecorator::BeverageDecorator;
    string describe() const override { return inner->describe() + " + sugar"; }
    double cost() const     override { return inner->cost() + 5; }
};

int main() {
    unique_ptr<Beverage> order = make_unique<Espresso>();
    order = make_unique<Milk>(std::move(order));
    order = make_unique<Sugar>(std::move(order));
    cout << order->describe() << " - " << order->cost() << " Rs\n";
}
```

OUTPUT

```
Espresso + milk + sugar - 65 Rs
```

Advantages.

- Responsibilities ko run time par combine karta hai; jo combinations 2^n subclasses maangti, wo linear ho jaati hain.
- **Open/Closed Principle** ke saath fully compliant.

Disadvantages.

- Bahut saare chhote objects ban jaate hain, jisse object graph padhna mushkil ho jaata hai.
- Decorator layers ke across equality checks aur identity semantics subtle ho jaati hain.

Kab use karein. Jab aapko individual objects mein responsibilities dynamically add karni hon, base class hierarchy ko modify kiye bina.

Kab use na karein. Jab sirf ek hi decoration chahiye — simple inheritance clearer hai.

✎ **EXAM POINT**

Exam-style answer. Decorator object ke saath nayi responsibilities dynamically attach karta hai — use wahi interface share karne waale doosre object mein wrap karke. Beverage / Espresso / Milk / Sugar hierarchy ke saath illustrate karo.

? **COMMON CONFUSION**

Comparison. Decorator vs Adapter. Adapter interface badalta hai; Decorator interface preserve karta hai aur behaviour add karta hai.

Ayush Jha

4.3.3 Facade Pattern

Category. Structural

> **Definition.** Facade pattern kisi subsystem ke interfaces ke set ke upar ek unified, simplified interface deta hai, taaki clients subsystem ki poori complexity samjhe bina hi common tasks perform kar sakein.

Problem. Subsystem mein bahut si chhoti classes hoti hain, har ek ki apni responsibility; unhe directly use karne ke liye client ko har ek ko samajhna padta hai — jo fragile aur error-prone hai.

Intent. High-level operations ka ek chhota set expose karna jo internally subsystem ko coordinate karta hai.

Analogy. Hotel ka reception desk: aap bolte ho “mujhe check-in karna hai.” Receptionist aapke liye housekeeping, billing, room service aur security ko orchestrate kar deta hai.

Participants.

- **Facade** — wo class jo simplified high-level interface deti hai.
- **Subsystem** — wo kayi chhoti classes jo actual kaam karti hain.
- **Client** — wo code jo sirf Facade se baat karta hai.

C++ implementation.

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

// --- Subsystem components ---
class CPU       { public: void start()   { cout << "CPU started\n"; } };
class Memory    { public: void load()    { cout << "Memory loaded\n"; } };
class HardDrive { public: void read()    { cout << "HardDrive read\n"; } };
class OS        { public: void boot()    { cout << "OS booted\n"; } };

// --- Facade ---
class Computer {
    CPU cpu; Memory mem; HardDrive hdd; OS os;
public:
    void powerOn() {
        cpu.start();
        mem.load();
        hdd.read();
        os.boot();
        cout << "Computer ready\n";
    }
};

int main() {
    Computer pc;
    pc.powerOn();      // single simple call
}
```

OUTPUT

```
CPU started
Memory loaded
HardDrive read
OS booted
Computer ready
```

Advantages.

- Client code ko subsystem complexity se shield karta hai.
- Client aur subsystem ke beech coupling kam karta hai.
- Ordering ya invariants enforce karne ki ek natural jagah deta hai.

Disadvantages.

- Facade ek *god object* ban sakta hai agar wo bahut saari responsibilities jama kar le.
- Jin clients ko fine-grained control chahiye, unhe facade ko bypass karna hi padta hai.

Kab use karein. Jab subsystem mein kayi chhoti classes hon aur common use cases mein unmein se kai ko ek saath orchestrate karna hota ho.

Kab use na karein. Jab subsystem ka direct use pehle se hi straightforward ho; facade unnecessary indirection add karega.

✎ **EXAM POINT**

Exam-style answer. Facade ek single high-level interface deta hai jo subsystem ki complexity ko hide karta hai. Computer facade ke saath illustrate karo jo power on ke liye CPU, Memory, HardDrive aur OS ko call karta hai.

? **COMMON CONFUSION**

Comparison. Facade vs Adapter. Adapter ek interface convert karta hai; Facade kayi interfaces ko simplify karta hai.

Ayush Jha

4.4 Behavioural Design Patterns

Behavioural patterns ka concern algorithms aur objects ke beech responsibilities ki assignment hota hai. Ye sirf classes ke patterns hi nahi, unke beech communication ke patterns bhi describe karte hain.

4.4.1 Strategy Pattern

Category. Behavioural

> **Definition.** Strategy pattern interchangeable algorithms ki ek family define karta hai, har ek ko common interface waale object ke roop mein encapsulate karta hai, aur algorithm ko us client se independently vary karne deta hai jo use use karta hai.

Problem. Kisi class ko ek task karna hai jo kai alag tareeqon se ho sakta hai; `if/else` chains hard-code karne se client har algorithm ke saath couple ho jaata hai.

Intent. Har algorithm ko uski apni class mein nikaalna, taaki context object ek ko hold kare aur run time par swap kar sake.

Analogy. Google Maps route planning: *walking*, *cycling*, *driving*, *public transport* — ye alag-alag algorithms hain jo ek hi sawaal ka jawaab dete hain — “main A se B tak kaise jaaun?”. Aap ek choose karte ho; baaki kaam map kar deta hai.

Participants.

- **Strategy** — algorithm ka abstract interface.
- **ConcreteStrategy** — alag-alag algorithm implementations.
- **Context** — wo class jo ek Strategy rakhti hai aur use delegate karti hai.

C++ implementation.

Ayush Jha

C++

```cpp
#include <iostream>
#include <memory>
#include <vector>
using namespace std;

class SortStrategy {
public:
    virtual void sort(vector<int>& v) = 0;
    virtual ~SortStrategy() = default;
};

class BubbleSort : public SortStrategy {
    void sort(vector<int>& v) override {
        cout << "using bubble sort\n";
        for (size_t i = 0; i < v.size(); ++i)
            for (size_t j = 0; j + 1 < v.size() - i; ++j)
                if (v[j] > v[j+1]) std::swap(v[j], v[j+1]);
    }
};
class QuickSort : public SortStrategy {
    void sort(vector<int>& v) override {
        cout << "using quick sort\n";
        std::sort(v.begin(), v.end());   // reuse std::sort as "quick sort"
    }
};

class Sorter {                       // Context
    unique_ptr<SortStrategy> algo;
public:
    void setStrategy(unique_ptr<SortStrategy> s) { algo = std::move(s); }
    void sort(vector<int>& v)                    { algo->sort(v); }
};

int main() {
    vector<int> data { 5, 2, 8, 1, 3 };
    Sorter s;
    s.setStrategy(make_unique<BubbleSort>());
    s.sort(data);
    s.setStrategy(make_unique<QuickSort>());
    s.sort(data);
    for (int x : data) cout << x << ' ';
}
```

OUTPUT

```
using bubble sort
using quick sort
1 2 3 5 8
```

Advantages.

- Algorithms ko run time par swap kiya ja sakta hai bina context change kiye.
- Context ke andar lambi `if/else` chains khatam ho jaati hain.
- **Open/Closed Principle** ke saath compliant.

Disadvantages.

- Har algorithm ke liye ek class introduce hoti hai.
- Ek choose karne ke liye client ko concrete strategies ke baare mein pata hona chahiye.

Kab use karein. Jab ek hi task ke liye multiple algorithms hon aur choice run time par ya per instance ki ja sakti ho.

Kab use na karein. Jab sirf ek hi algorithm ho aur aur add karne ka koi plan na ho.

✎ **EXAM POINT**

Exam-style answer. Strategy interchangeable algorithms ki family define karta hai, har ek apni class mein, aur Context ko run time par ek select karne deta hai. BubbleSort aur QuickSort ke beech swap karne waale Sorter ke saath illustrate karo.

? **COMMON CONFUSION**

Comparison. Strategy vs State. Strategy: *client* algorithm choose karta hai. State: *object khud* apni strategy badalta hai jab uski internal state change hoti hai.

Ayush Jha

4.4.2 Observer Pattern

Category. Behavioural

> **Definition.** Observer pattern objects ke beech one-to-many dependency define karta hai, taaki jab ek object (*subject*) ki state change ho, uske saare dependents (*observers*) ko automatically notify aur update kar diya jaaye.

Problem. Program ke kai hisson ko tab react karna hota hai jab koi state change hoti hai; state-holder ka har ek ko directly call karna tight coupling create karta hai.

Intent. Observers ko subject ke saath subscribe karne dena; subject har subscriber ko notify karta hai bina ye jaane ki wo concretely kaun hain.

Analogy. YouTube channel: subscribers hi observers hain. Jab channel video upload karta hai (state change), har subscriber ko notification mil jaata hai. Channel ko har subscriber ko personally jaanne ki zaroorat nahi hoti.

Participants.

- **Subject** — observers ka track rakhta hai aur unhe notify karta hai.
- **Observer** — abstract interface jisme `update()` method hota hai.
- **ConcreteObserver** — har subscriber jo update implement karta hai.

C++ implementation.

C++

```cpp
#include <iostream>
#include <memory>
#include <vector>
#include <string>
using namespace std;

class Observer {
public:
    virtual void update(const string& event) = 0;
    virtual ~Observer() = default;
};

class NewsAgency {                              // Subject
    vector<Observer*> observers;
public:
    void subscribe (Observer* o)   { observers.push_back(o); }
    void unsubscribe(Observer* o)  { observers.erase(remove(observers.begin(),
                                                            observers.end(), o),
                                                     observers.end()); }
    void publish  (const string& e){ for (auto* o : observers) o->update(e); }
};

class NewsChannel : public Observer {
    string name;
public:
    explicit NewsChannel(string n) : name(std::move(n)) {}
    void update(const string& e) override {
        cout << name << " received: " << e << '\n';
    }
};

int main() {
    NewsAgency agency;
    NewsChannel c1{"CNN"}, c2{"BBC"}, c3{"NDTV"};
    agency.subscribe(&c1); agency.subscribe(&c2); agency.subscribe(&c3);
    agency.publish("Election results announced");
}
```

OUTPUT

```
CNN received: Election results announced
BBC received: Election results announced
NDTV received: Election results announced
```

Advantages.

- Loose coupling: subject sirf Observer interface jaanta hai, concrete subscribers nahi.
- Naye observers bina subject change kiye add kiye ja sakte hain.
- SRP ke saath compliant: subject observers ka kaam khud nahi karta.

Disadvantages.

- Update order guaranteed nahi hoti jab tak explicitly enforce na ki jaaye.
- Subscriptions ka careless management dangling pointers bana sakta hai.
- Bade systems mein zyada notifications performance ko degrade kar sakti hain.

Kab use karein. Jab ek object mein change unknown number of doosron mein reflect hona chahiye (UI listeners, event buses, live dashboards).

Kab use na karein. Jab sirf ek hi dependent ho — direct method call simpler hai.

✎ EXAM POINT

Exam-style answer. Observer one-to-many relationship set up karta hai, taaki jab subject ki state change ho, uske saare observers automatically notify ho jaayein. NewsAgency aur kai NewsChannel subscribers ke saath illustrate karo.

? COMMON CONFUSION

Comparison. Observer vs Strategy. Observer kayi ko *notify* karta hai; Strategy ek ko *select* karta hai.

4.4.3 State Pattern

Category. Behavioural

> **Definition.** State pattern kisi object ko apni internal state change hone par apna behaviour alter karne deta hai; object apni class change karta hua lagta hai, kyunki har state ek alag class se represent hoti hai jo common interface implement karti hai.

Problem. Jis class ka behaviour uski state par depend karta hai, uske har method mein bada `switch(state)` aa jaata hai — unreadable aur extend karna mushkil.

Intent. Har state ko uski apni class mein nikaalna; context behaviour ko current state object ko delegate karta hai aur transitions par us object ko swap kar deta hai.

Analogy. Traffic light: *Red*, *Yellow* aur *Green* alag-alag states hain, har ek ka apna behaviour ("ab cars kya kar sakti hain") aur agle state ke liye apna rule.

Participants.

- **Context** — wo object jiska behaviour state par depend karta hai.
- **State** — abstract interface jisme wo operations hain jo Context ko chahiye.
- **ConcreteState** — har state class jo State implement karti hai aur transition define karti hai.

C++ implementation.

Ayush Jha

C++

```cpp
#include <iostream>
#include <memory>
using namespace std;

class TrafficLight;             // forward declaration
class State {
public:
    virtual void handle(TrafficLight& ctx) = 0;
    virtual ~State() = default;
};

class TrafficLight {
    unique_ptr<State> current;
public:
    explicit TrafficLight(unique_ptr<State> s) : current(std::move(s)) {}
    void setState(unique_ptr<State> s) { current = std::move(s); }
    void request()                     { current->handle(*this); }
};

class GreenState;
class RedState;
class YellowState;

class RedState    : public State { public: void handle(TrafficLight& ctx) override; };
class GreenState  : public State { public: void handle(TrafficLight& ctx) override; };
class YellowState : public State { public: void handle(TrafficLight& ctx) override; };

void RedState::handle(TrafficLight& ctx) {
    cout << "RED  -> STOP; next GREEN\n";
    ctx.setState(make_unique<GreenState>());
}
void GreenState::handle(TrafficLight& ctx) {
    cout << "GREEN-> GO;   next YELLOW\n";
    ctx.setState(make_unique<YellowState>());
}
void YellowState::handle(TrafficLight& ctx) {
    cout << "YEL  -> SLOW; next RED\n";
    ctx.setState(make_unique<RedState>());
}

int main() {
    TrafficLight light(make_unique<RedState>());
    for (int i = 0; i < 4; ++i) light.request();
}
```

OUTPUT

```
RED  -> STOP; next GREEN
GREEN-> GO;   next YELLOW
YEL  -> SLOW; next RED
RED  -> STOP; next GREEN
```

Advantages.

- Context ke andar giant `switch` statements khatam ho jaate hain.
- Naya state add karne ke liye sirf ek nayi class chahiye.
- State transitions state classes ke andar hi localised rehte hain.

Disadvantages.

- Classes ki sankhya badh jaati hai.

• Bahut kam states ke liye switch clearer ho sakta hai.

Kab use karein. Jab class ka behaviour uski state ke saath significantly change hona ho, aur state machine grow karne waali ho.

Kab use na karein. Jab sirf do-teen simple states hon jinka behaviour trivial ho — enum use karo.

✎ **EXAM POINT**

Exam-style answer. State object ko apni internal state change hone par apna behaviour badalne deta hai — current State object ko delegate karke. Red, Green aur Yellow states mein cycle karne waali TrafficLight ke saath illustrate karo.

? **COMMON CONFUSION**

Comparison. State vs Strategy. Dono delegate object use karte hain, par State pattern ke delegates operation ke dauraan *khud ko replace* karte hain, jabki Strategy ke delegates client select karta hai aur wo waise hi rehte hain.

Ayush Jha

Unit IV — Revision Sheet

Ye sheet Unit IV ko us material mein compress karti hai jiske examination mein aane ki sambhaavna sabse zyada hai. Har pattern ka naam, uski category, uska one-sentence intent, aur ek participants list yaad karo.

Unit IV Summary

Design patterns recurring design problems ke named, reusable solutions hote hain, jo **Creational** (Factory, Singleton, Builder, Prototype), **Structural** (Adapter, Decorator, Facade) aur **Behavioural** (Strategy, Observer, State) mein grouped hain. Har pattern Unit I mein padhe SOLID principles ka ek application express karta hai, aur Units I aur III mein padhe polymorphism aur smart-pointer tools par depend karta hai.

Important Definitions (One-Line Recall)

Pattern	Category	Ek line mein intent
Factory	Creational	Object creation ko ek single function ke peechhe encapsulate karna.
Singleton	Creational	Class ke exactly ek instance ki guarantee dena.
Builder	Creational	Complex object ko fluent API se step-by-step build karna.
Prototype	Creational	Existing prototype ko clone karke naye objects banana.
Adapter	Structural	Existing class ke interface ko usme convert karna jo client expect karta hai.
Decorator	Structural	Individual objects ke saath responsibilities dynamically attach karna.
Facade	Structural	Complex subsystem ke upar ek single high-level interface dena.
Strategy	Behavioural	Interchangeable algorithms encapsulate karna; run time par select karna.
Observer	Behavioural	Subject ki state change hone par one-to-many notification.
State	Behavioural	State objects ko delegate karke behaviour badalna, jo ek-doosre ko replace karte hain.

Frequently Confused Patterns

Pair	Ek line mein difference
Factory vs Prototype	Factory fresh product banata hai; Prototype existing waali ki copy karta hai.
Factory vs Builder	Factory: ek call. Builder: kai named steps.
Adapter vs Decorator	Adapter interface badalta hai; Decorator waisa rakhta hai par behaviour add karta hai.
Decorator vs Facade	Decorator ek object wrap karta hai; Facade poora subsystem wrap karta hai.
Strategy vs State	Strategy = client algorithm pick karta hai; State = object evolve hote hi apna algorithm khud badalta hai.
Observer vs Strategy	Observer kayi ko notify karta hai; Strategy ek ko delegate karta hai.

Expected Exam Questions

- *Design pattern* define karo. Teen categories ko examples ke saath list karo.

- Factory design pattern ko C++ example ke saath explain karo.
- C++ mein thread-safe Singleton (Meyers singleton) implement karo.
- Builder pattern kya hai? Pizza construct karne ka C++ example do.
- `clone()` waali Shape hierarchy se Prototype pattern illustrate karo.
- Adapter aur Decorator patterns mein difference C++ examples ke saath batao.
- Computer subsystem example ke saath Facade pattern explain karo.
- C++ mein Strategy pattern implement karo (sorting algorithms).
- NewsAgency example ke saath Observer pattern explain karo.
- C++ mein TrafficLight se State pattern illustrate karo.

Practice Questions (Graded)

Level	Task
★	Singleton `Config` class likho jo ek single application setting store kare.
★★	`ShapeFactory` likho jo string se Circle, Rectangle aur Triangle produce kare.
★★★	`OrderBuilder` design karo jo optional discounts aur gift-wrap ke saath complex Order assemble kare.
★★★★	Espresso aur Milk, Sugar, Cream ke decorators waali Beverage hierarchy design karo, aur teen alag combinations demonstrate karo.
★★★★★	Mini stock-price system design karo: `Stock` subject aur `Screen` / `Log` observers; har observer notification par naya price print kare.

! DO NOT FORGET

Do not forget. Har design-pattern exam question ke liye apna answer is structure mein likho: (1) one-sentence intent, (2) participants list, (3) chhota C++ example, (4) ek advantage aur ek disadvantage, (5) kab use karna hai aur kab nahi. Ye checklist time pressure mein bhi full marks dilaati hai.

UNIT V

Concurrent Programming and Modern Applications

Threads · Mutexes · Atomics · Futures · C++20 Coroutines

Unit V — Concurrent Programming and Modern Applications

Ye unit kis baare mein hai

Unit V aapko aise programs likhna sikhaata hai jo ek hi samay par ek se zyada cheezon par progress karte hain. Modern machines ke paas kayi CPU cores hote hain; concurrent programming ke bina wo saara hardware idle hi baitha rehta hai. Aap threads launch karna, mutexes se shared data protect karna, atomics se lock-free operations karna, `std::async` aur `std::future` se asynchronous kaam chalaana, aur aakhir mein C++20 **coroutines** — modern C++ ke sabse zyada discussed features mein se ek — use karna seekhoge.

Ye unit kyon important hai

Aap jis bhi professional C++ codebase par kaam karoge, usme kahin na kahin concurrency hogi hi — ek web browser mein bhi sauon threads hote hain. Is unit ke concepts operating systems, database engines, game engines aur scientific computing par bhi apply hote hain. Concurrency is language ka sabse mushkil chapters mein se ek bhi hai: wahi code aapke laptop par sahi ho sakta hai aur production mein galat. Sahi primitives seekhna optional nahi hai.

Prerequisites

- Modern C++ se comfort (Unit III) — lambdas, smart pointers, move semantics.
- References aur value / reference se arguments pass karna samajhna (F.6, F.8).
- Aap C++20 toolchain se compile karte ho: GCC/Clang par `-std=c++20 -pthread`.

Learning objectives

- Concurrency, parallelism, process aur thread explain karna.
- `std::thread` aur `std::jthread` se threads launch karna.
- Mutex, `lock_guard` aur `scoped_lock` se shared data protect karna.
- Simple lock-free updates ke liye atomic operations use karna.
- `std::future` aur `std::async` se background task ka result lena.
- Coroutine concept explain karna aur `co_await`, `co_yield`, `co_return` use karna.

Ayush Jha

5.1 Modern Concurrency

5.1.1 Concurrency kya hai?

Definition. Concurrency kisi program ki wo property hai jo batati hai ki program mein multiple logically independent activities overlapping time intervals mein progress kar rahi hain, taaki baahar se dekhne par program ek saath kai cheezein kar raha ho aisa lage.

Concurrency ek **structural** property hai. Ye is baare mein kuch nahi kehti ki program kitne CPU cores use kar raha hai; ek single-core machine bhi activities ke beech rapidly switch karke concurrent ho sakti hai.

5.1.2 Concurrency vs Parallelism

Definition. Parallelism ka matlab hai multiple activities ka separate execution units (CPU cores) par *simultaneous execution*. Concurrency program ki wo design hai jo aisi execution allow karti hai — chahe wo actually simultaneously ho ya na ho.

Aspect	Concurrency	Parallelism
Matlab	Program ki structure	Hardware par simultaneous execution
Chahiye	Interleaved progress	≥ 2 execution units
Focus	Correctness aur coordination	Speed aur throughput
1 core par possible?	Haan	Nahi
Analogy	Ek chef do dishes ke beech switch kar raha ho	Do chefs, har ek apni dish bana raha ho

5.1.3 Process vs Thread

Process. Process kisi chalte hue program ka ek instance hota hai jiska apna private virtual address space, file descriptors aur baaki operating-system resources hote hain.

Thread. Thread kisi process ke *andhar* execution ka ek independent path hota hai, jo us process ke baaki threads ke saath process ka address space share karta hai.

Aspect	Process	Thread
Address space	Har process ka alag	Process ke baaki threads ke saath shared
Creation cost	Heavy (fork / CreateProcess)	Light (kuch kilobytes ki stack)
Communication	Pipes, sockets, files se	Shared memory se (fast, par synchronisation chahiye)
Failure isolation	Strong — ek crash baakiyon ko rarely affect karta hai	Weak — ek thread crash hone par usually poora process mar jaata hai
Typical use	Isolated services	Ek program ke andar parallel kaam

5.1.4 Multithreading useful kyon hai

- **Responsiveness.** UI thread responsive reh sakta hai jabki worker thread lambi computation kar rahi ho.
- **Throughput.** Multi-core CPU sirf usi program ko accelerate karta hai jo pehle se kai threads par chalta hai.
- **I/O overlap.** Jab ek thread network reply ka wait kar rahi ho, doosri thread kaam ka kaam kar sakti hai.

• **Modularity.** Complex programs (browsers, IDEs, servers) naturally cooperating threads mein decompose hote hain.

5.1.5 Shared Data se Aane Waale Problems

Jaise hi do threads kisi ek variable ko bina coordination ke read ya write karte hain, teen classic problems possible ho jaate hain.

Race condition. Aisi situation jismein program ki correctness do ya zyada threads ke memory accesses ke exact order par depend karti hai.

Data race. Race condition ka ek specific form jismein kam se kam do threads bina synchronisation ke ek hi memory location access karte hain aur unmein se kam se kam ek access write hota hai. C++ mein data race **undefined behaviour** hota hai.

Deadlock. Do ya zyada threads mein se har ek aisi lock hold karke baitha hai jiski doosre ko zaroorat hai, isliye koi bhi aage nahi badh sakta.

C++

```
#include <thread>
#include <iostream>
using namespace std;

int counter = 0;                          // shared, unsynchronised
void addTenThousand() {
    for (int i = 0; i < 10000; ++i) ++counter;   // data race!
}

int main() {
    thread t1(addTenThousand);
    thread t2(addTenThousand);
    t1.join(); t2.join();
    cout << counter << '\n';              // may print 12873, 17842, ... - NOT 20000
}
```

! WARNING

Upar waala program **undefined behaviour** hai. Iska output runs, compilers, aur ek hi binary ke repeat executions mein bhi badalta rehta hai. §5.4 aur §5.7 dono supported fixes dikhate hain: mutex aur atomic.

≡ ONE-LINE REVISION

One-line revision. Concurrency = *structure*, parallelism = *execution*; do threads + shared writable data bina synchronisation ke = **undefined behaviour**.

5.2 std::thread

Definition. `std::thread`, jo `<thread>` header mein declared hai, ek C++11 class hai jo execution ke ek single thread ko represent karti hai; kisi callable aur uske arguments ke saath `std::thread` construct karte hi wo callable ek naye operating-system thread par launch ho jaata hai.

5.2.1 Basic Syntax

C++

```cpp
#include <thread>
#include <iostream>
using namespace std;

void hello(int id) {
    cout << "hello from thread " << id << '\n';
}

int main() {
    thread t1(hello, 1);         // launches on a new OS thread
    thread t2([]{ cout << "hello from lambda\n"; });

    t1.join();                    // wait for t1 to finish
    t2.join();                    // wait for t2 to finish
    cout << "main is done\n";
}
```

OUTPUT

```
hello from thread 1
hello from lambda
main is done   (order between t1 & t2 may vary)
```

5.2.2 join() vs detach()

Method	Matlab	Typical use
`join()`	Jab tak thread finish na ho tab tak block karo, phir resources release karo.	Almost hamesha.
`detach()`	Thread ko independently chalne ke liye chhod do; aap handle kho dete ho.	“Fire and forget” background tasks.

! WARNING

Jo `std::thread` apne destructor chalne se pehle na joined ho na detached, wo poore program ko `std::terminate` ke through terminate kar deta hai. Ye do sabse common beginner concurrency bugs mein se ek hai.

5.2.3 Arguments Sahi Tarah Pass Karna

Thread constructor ko diye gaye arguments by default nayi thread mein **copy** hote hain. Reference se pass karne ke liye explicitly `std::ref` ya `std::cref` use karo.

```C++
void inc(int& n) { ++n; }

int main() {
    int x = 10;
    // std::thread t(inc, x);         // ERROR: 'x' would be copied, inc needs a reference
    std::thread t(inc, std::ref(x));  // OK
    t.join();
    std::cout << x << '\n';           // 11
}
```

5.2.4 `hardware_concurrency()`

Static function `std::thread::hardware_concurrency()` ek hint return karta hai ki hardware ek saath kitne threads parallel chala sakta hai — thread pools ka size decide karne ke liye useful.

```C++
std::cout << "cores = "
          << std::thread::hardware_concurrency() << '\n';
```

Ayush Jha

5.3 `std::jthread` (C++20)

Definition. `std::jthread` `std::thread` ka C++20 replacement hai jo apne destructor mein apni worker thread ko automatically join kar leta hai aur cooperative cancellation mechanism `std::stop_token` ke saath integrated hota hai.

5.3.1 Destructor mein Automatic Join

C++

```cpp
#include <thread>
#include <iostream>
using namespace std;

int main() {
    jthread j([]{
        for (int i = 0; i < 3; ++i) cout << "work " << i << '\n';
    });
    // No explicit j.join() needed - the destructor joins for us.
}
```

OUTPUT

```
work 0
work 1
work 2
```

5.3.2 `stop_token` ke through Cooperative Cancellation

Koi `jthread` apne first argument ke roop mein `std::stop_token` accept kar sakta hai. Main thread baad mein `request_stop()` call kar sakta hai; worker token check karta hai aur cleanly exit kar jaata hai.

C++

```cpp
#include <thread>
#include <chrono>
#include <iostream>
using namespace std;
using namespace std::chrono_literals;

int main() {
    jthread worker([](stop_token st){
        while (!st.stop_requested()) {
            cout << "working...\n";
            this_thread::sleep_for(200ms);
        }
        cout << "worker stopping cleanly\n";
    });

    this_thread::sleep_for(600ms);
    worker.request_stop();      // ask the worker to finish
    // destructor joins automatically
}
```

OUTPUT

```
working...
working...
working...
worker stopping cleanly
```

5.3.3 `thread` vs `jthread`

Aspect	std::thread	std::jthread
Introduced	C++11	C++20
Destructor mein auto-join?	Nahi — terminate call hota hai	Haan
Built-in cancellation?	Nahi	Haan (`stop_token`)
Recommended default?	Sirf legacy code	Haan (modern code)

◆ **REMEMBER**

Har naye C++20 program mein `std::jthread` prefer karo. Ye `std::thread` hi hai — bas footguns ke bina.

Ayush Jha

5.4 Mutex

Definition. Mutex (*mutual exclusion* ka short form) ek synchronisation primitive hai jo ensure karta hai ki ek time par sirf ek hi thread designated critical section ke andar execute kar sake. C++ mein ye `<mutex>` header ke `std::mutex` se milta hai.

5.4.1 Pehle Race Condition, Phir Fix

§5.1.5 ka toota hua counter yaad karo. Ab hum use fix kar sakte hain:

C++

```
#include <mutex>
#include <thread>
#include <iostream>
using namespace std;

int   counter = 0;
mutex m;

void addTenThousand() {
    for (int i = 0; i < 10000; ++i) {
        m.lock();
        ++counter;
        m.unlock();
    }
}

int main() {
    thread t1(addTenThousand), t2(addTenThousand);
    t1.join(); t2.join();
    cout << counter << '\n';   // ALWAYS 20000
}
```

OUTPUT

```
20000
```

5.4.2 Mutexes ke Rules

• Shared data ka har read *aur* har write ek hi critical section ke andar hona chahiye.

• Critical section ko jitna ho sake utna chhota rakho — mutex sirf us code ke around hold karo jo shared data ko touch karta hai.

• `lock()` aur `unlock()` ke beech kabhi exception mat throw karo jab tak aap `lock_guard` (§5.5) jaisa RAII helper use na kar rahe ho; uncaught exception mutex ko permanently locked chhod deta hai.

• Ek hi `std::mutex` par ek hi thread se do baar `lock()` call mat karo — yehi deadlock hai. Recursive locking chahiye to `std::recursive_mutex` use karo.

! WARNING

Manual `lock() / unlock()` yahan sirf samajhne ke liye dikhaya gaya hai. Real code mein **hamesha** RAII wrappers `lock_guard`, `scoped_lock` ya `unique_lock` use karo.

5.5 std::lock_guard

Definition. `std::lock_guard<Mutex>` ek class template hai jo apne constructor ko diye gaye mutex ko lock karti hai aur apne destructor mein use unlock kar deti hai, isliye mutex release hone ki guarantee rehti hai chahe critical section ke andar exception throw bhi ho jaaye. Ye mutex ka sabse simple RAII wrapper hai.

C++

```
#include <mutex>
#include <thread>
#include <vector>
#include <iostream>
using namespace std;

vector<int> data;
mutex m;

void add(int x) {
    lock_guard<mutex> lock(m);        // locks m
    data.push_back(x);
}                                     // destructor unlocks m automatically

int main() {
    thread t1(add, 1), t2(add, 2), t3(add, 3);
    t1.join(); t2.join(); t3.join();
    for (int x : data) cout << x << ' ';
}
```

OUTPUT

```
1 2 3   (order may vary)
```

Rules jo yaad rakhne hain.

- Non-copyable, non-movable — lock us block tak *scoped* hota hai jisme declare hua hai.
- Mutex destructor mein unlock hota hai — exceptions use locked nahi chhod sakti.
- Ek se zyada mutexes lock nahi kar sakta; uske liye `scoped_lock` (§5.6) use karo.

5.6 std::scoped_lock

Definition. `std::scoped_lock<Mutexes...>`, jo C++17 mein aaya, ek RAII wrapper hai jo ek deadlock-avoidance algorithm use karke ek ya zyada mutexes ko ek saath lock kar sakta hai aur apne destructor mein sabko unlock kar deta hai.

5.6.1 Ek se Zyada Locks kyon matter karte hain — Deadlock ka Trap

C++

```
// Two accounts, each with its own mutex.
// A wrong transfer implementation:
void wrongTransfer(Account& from, Account& to, int amt) {
    lock_guard g1(from.m);
    lock_guard g2(to.m);          // if another thread transfers the other way,
                                  // both threads deadlock.
    from.balance -= amt;
    to.balance   += amt;
}
```

`scoped_lock` se fix:

C++

```cpp
#include <mutex>
#include <thread>
#include <iostream>
using namespace std;

struct Account { int balance; mutex m; };

void transfer(Account& from, Account& to, int amt) {
    scoped_lock lock(from.m, to.m);   // locks BOTH atomically, no deadlock
    from.balance -= amt;
    to.balance   += amt;
}

int main() {
    Account a{100, {}}, b{100, {}};
    thread t1(transfer, ref(a), ref(b), 30);
    thread t2(transfer, ref(b), ref(a), 20);
    t1.join(); t2.join();
    cout << "a=" << a.balance << ", b=" << b.balance << '\n';   // a=90, b=110
}
```

OUTPUT

```
a=90, b=110
```

5.6.2 `lock_guard` vs `scoped_lock`

Aspect	lock_guard	scoped_lock
Introduced	C++11	C++17
Mutexes ki sankhya	Exactly one	Ek ya zyada
Multiple locks ke liye deadlock-avoidance?	N/A	Haan (internally `std::lock` use karta hai)
Recommended default?	Single-mutex code	Naya code, hamesha

★ EXAM TIP

C++17 ke baad se aap kisi bhi mutex ke liye — single mutex ke liye bhi — simply `scoped_lock lock(m);` likh sakte ho aur future ke liye safe reh sakte ho.

5.7 Atomic Operations

Definition. `std::atomic<T>`, jo `<atomic>` header mein declared hai, ek template hai jo type `T` ki value ko wrap karta hai aur read, write aur read-modify-write operations deta hai jinki guarantee hoti hai ki multiple threads ek hi object par concurrently access kar rahe hon tab bhi wo ek single indivisible step ki tarah execute honge.

5.7.1 Ek Lock-Free Counter

C++

```cpp
#include <atomic>
#include <thread>
#include <iostream>
using namespace std;

atomic<int> counter{0};

void addTenThousand() {
    for (int i = 0; i < 10000; ++i) ++counter;   // atomic increment
}

int main() {
    thread t1(addTenThousand), t2(addTenThousand);
    t1.join(); t2.join();
    cout << counter << '\n';         // always 20000, without any mutex
}
```

OUTPUT

```
20000
```

5.7.2 Atomics kya dete hain

• **Indivisibility.** Koi thread kisi atomic write ko half-done observe nahi kar sakta.

• **Memory ordering.** Atomics ek memory-order tag carry karte hain (default `seq_cst`) jo unke aas-paas ke non-atomic accesses ka order bhi discipline karta hai.

• **Koi blocking nahi.** Zyaadatar atomic operations `lock xadd` jaise CPU instructions se implement hote hain aur kabhi thread ko sleep par nahi daalte.

5.7.3 Atomics vs Mutexes — Kab kaunsa choose karein

Situation	Prefer
Threads ke beech shared ek single integer ya boolean	`std::atomic`
Multiple related variables ko saath move karna ho	`std::mutex`
Rare, uncontended updates	`std::atomic`
Lambe critical sections	`std::mutex`
Kisi condition par blocking chahiye	`std::condition_variable`

! WARNING

Atomics mutexes ka magic replacement **nahi** hain. Ye sirf single-object operations ko indivisible banate hain; do ya zyada variables par phaila kuch bhi abhi bhi mutex maangta hai.

5.8 Futures and Async (Introduction)

Definition. `std::future<T>` ek handle hai jo type `T` ki kisi asynchronous computation ke eventual result ko represent karta hai; `std::async` ek factory hai jo aisi computation launch karta hai aur uska corresponding `future` return karta hai.

5.8.1 Synchronous vs Asynchronous Execution

Aspect	Synchronous	Asynchronous
Caller ka behaviour	Task finish hone ka wait karta hai	Turant aage badh jaata hai
Result	Function call se milta hai	`future` ke through milta hai
Blocking?	Haan (call ke dauraan)	Sirf jab future query ho
Typical use	Cheap, fast computations	Slow ya I/O-bound kaam

5.8.2 Sabse Simple Example

C++

```cpp
#include <future>
#include <iostream>
#include <thread>
#include <chrono>
using namespace std;
using namespace std::chrono_literals;

int slowSquare(int n) {
    this_thread::sleep_for(500ms);   // pretend expensive
    return n * n;
}

int main() {
    future<int> f = async(launch::async, slowSquare, 12);

    cout << "main continues while slowSquare runs\n";
    int result = f.get();            // waits here for the answer
    cout << "12*12 = " << result << '\n';
}
```

OUTPUT

```
main continues while slowSquare runs
12*12 = 144
```

5.8.3 `launch::async` vs `launch::deferred`

Policy	Task actually kab chalta hai?
`launch::async`	Turant, ek nayi thread par.
`launch::deferred`	Lazily, calling thread par — jab pehli baar `get()` invoke ho.
(default — koi policy nahi)	Implementation-defined; upar waalon mein se koi bhi.

★ EXAM TIP

Agar aap chahte ho ki kaam alag thread par chale to hamesha `launch::async` explicitly pass karo. Default policy implementation-defined hai aur isse subtle bugs aaye hain.

5.8.4 Futures mein Exceptions

Agar asynchronous function throw karta hai, to exception future ke andar store ho jaata hai aur jab `get()` call hota hai tab re-throw ho jaata hai.

C++

```
auto f = async(launch::async, []{ throw runtime_error("boom"); });
try { f.get(); }
catch (const exception& e) { cout << "caught: " << e.what() << '\n'; }
```

OUTPUT

```
caught: boom
```

≡ ONE-LINE REVISION

One-line revision. `async` kaam start karta hai; `future` result carry karta hai; `get()` wait karke result deta hai (ya exception).

5.9 Coroutines (Introduction)

Definition. Coroutine ek special function hota hai jo apni execution ko well-defined points par **suspend** kar sakta hai, apne caller ko control waapas de sakta hai (shayad ek value ke saath), aur baad mein bilkul wahin se **resume** ho sakta hai jahan wo ruka tha — apni local state intact ke saath. C++20 coroutines ko ek language feature banata hai: jis bhi function ke body mein `co_await`, `co_yield` ya `co_return` hota hai, wo ek coroutine hai.

5.9.1 Ordinary Function se Coroutine tak

Ek ordinary function top se bottom tak ek uninterrupted burst mein chalta hai; jab wo return karta hai, uske local variables destroy ho jaate hain aur uska stack frame chala jaata hai.

Coroutine bhi top se bottom chalna shuru karta hai, par `co_await`, `co_yield` ya `co_return` mein se koi bhi use temporarily suspend kar sakta hai. Uski local state destroy nahi hoti; wo heap par allocate hui **coroutine frame** mein save ho jaati hai aur baad mein resume hoti hai.

Coroutine call karne ka result koi plain value nahi hota; wo ek **coroutine return object** hota hai jiska type define karta hai ki har suspension aur resumption point par kya karna hai.

i IMPORTANT

Coroutines C++20 ke sabse powerful features mein se ek hain par sabse zyada machinery-waale bhi. Ye introduction *ideas* par focus karta hai aur minimal par technically sahi examples dikhata hai. Practice mein aap `cppcoro`, `Boost.Asio` ya network libraries ke saath aane waale coroutines jaisi library use karte ho — coroutine machinery khud haath se rarely likhte ho.

5.10 Coroutine Basics

Teen keywords kisi function ko coroutine banate hain:

- `co_await` — coroutine ko suspend karta hai jab tak *awaitable* ye signal na de de ki uska result ready hai.
- `co_yield` — coroutine ko suspend karta hai aur caller ko ek value haath mein deta hai (generators ke liye use hota hai).
- `co_return` — coroutine ko complete karta hai, chahe to ek final value ke saath.

Coroutine ka return type aisi class honi chahiye jo nested `promise_type` provide kare. Promise wahi type hai jo compiler actually tab banata hai jab coroutine call hota hai, aur ye runtime ko batata hai ki suspension par, return par, aur exceptions par kya karna hai.

C++

```cpp
// The minimal boiler-plate every C++20 coroutine return type needs.
#include <coroutine>

struct Task {
    struct promise_type {
        Task get_return_object()             { return {}; }
        std::suspend_never initial_suspend() { return {}; }   // start eagerly
        std::suspend_never final_suspend()   noexcept { return {}; }
        void return_void()                   {}
        void unhandled_exception()           {}
    };
};
```

5.11 co_await

Definition. `co_await expr` enclosing coroutine ko suspend karta hai, uske caller ko control waapas de deta hai, jab tak awaitable expression `expr` ye report na kar de ki uska result available ho gaya hai.

Upar waale `Task` ko return type maan kar, `co_await` use karne waala coroutine aisa dikhta hai:

C++

```
#include <coroutine>
#include <iostream>

// Task type from 5.10 assumed available.
Task demo() {
    std::cout << "before suspend\n";
    co_await std::suspend_always{};   // suspends here
    std::cout << "resumed and finishing\n";
}

int main() {
    demo();                           // starts, prints, suspends
    std::cout << "back in main\n";
    // In a real program the coroutine would be resumed by a scheduler.
}
```

OUTPUT

```
before suspend
back in main
```

Upar waala example essence dikhata hai: `co_await std::suspend_always{}` control `main` ko waapas de deta hai aur coroutine memory mein baitha rehta hai, kisi scheduler ke resume karne ka wait karta hua.

5.12 co_yield

Definition. `co_yield expr` `co_await promise.yield_value(expr)` ke equivalent hota hai: ye coroutine ko suspend karta hai aur `expr` ko caller tak deliver karta hai. Isse **generators** banaye jaate hain — aise sequences jo demand par ek-ek value produce karte hain.

Yahan pehle *n* integers yield karne waala ek minimal par complete C++20 generator hai:

C++

```
#include <coroutine>
#include <iostream>
#include <optional>
using namespace std;

template <typename T>
struct Generator {
    struct promise_type {
        T current{};
        Generator get_return_object() {
            return Generator{coroutine_handle<promise_type>::from_promise(*this)};
        }
        suspend_always initial_suspend() { return {}; }
        suspend_always final_suspend() noexcept { return {}; }
        suspend_always yield_value(T v)  { current = v; return {}; }
        void return_void()               {}
        void unhandled_exception()       { std::terminate(); }
    };
    coroutine_handle<promise_type> h;
    Generator(coroutine_handle<promise_type> hh) : h(hh) {}
    ~Generator() { if (h) h.destroy(); }
    optional<T> next() {
        if (h.done()) return nullopt;
        h.resume();
        if (h.done()) return nullopt;
        return h.promise().current;
    }
};

Generator<int> range(int n) {
    for (int i = 0; i < n; ++i) co_yield i;
}

int main() {
    auto g = range(5);
    while (auto v = g.next()) cout << *v << ' ';
}
```

OUTPUT

```
0 1 2 3 4
```

5.13 co_return

Definition. `co_return` coroutine ka `return`-analogue hai: ye coroutine ko end karta hai. Agar value di gayi ho (`co_return expr;`) to wo promise ko final result ke roop mein forward hoti hai; bare `co_return;` us coroutine ko end karta hai jiska promise `return_void()` define karta hai.

```cpp
// Uses the Task type from 5.10.
Task greet() {
    std::cout << "hello\n";
    co_return;             // ends the coroutine
}
```

! WARNING

Coroutine ke andar kabhi plain `return` use mat karo. `co_return` use karo — compiler ko pata hona chahiye ki function ek coroutine hai.

5.14 Coroutines ke Applications

Coroutines tab useful hote hain jab code mein “yahan ruko aur baad mein jaari rakho” ka natural notion hota hai. Practice mein ye sabse common applications hain:

Application	Coroutines kaise help karte hain
Generators	Sequences ko ek-ek value produce karna bina pehle se poori list banaye.
Asynchronous I/O	Network / file code jo sequentially padha jaata ho par wait karte waqt thread block na kare.
Task-based concurrency	Asynchronous tasks ko `co_await` se ordinary calls jaisa compose karna.
State machines	State machine ko straight-line code ke roop mein likhna, transitions par suspensions ke saath.
Game engines	In-game scripts, animations aur NPC AI jo kai frames tak chalte hain.
Event-driven UIs	Callback-based spaghetti ke bina user events par react karna.

Common misuses.

• Bahut chhote synchronous kaam ke liye coroutines use karna — wo frame allocate karte hain aur suspension cost dete hain.

• Production code ke liye coroutine machinery haath se likhna — `cppcoro` ya `Boost.Asio` ke coroutine support jaisi well-tested library prefer karo.

• Coroutines aur raw threads ko carelessly mix karna — coroutine kis thread par resume hogi ye matter karta hai aur iske baare mein sochna padta hai.

≡ ONE-LINE REVISION

One-line revision. Coroutine ek function hai jo *suspend* aur *resume* ho sakta hai; `co_await` wait karta hai, `co_yield` value baahar deta hai, `co_return` coroutine ko end karta hai.

Unit V — Revision Sheet

Ye sheet Unit V ko us material mein compress karti hai jiske examination mein aane ki sambhaavna sabse zyada hai. Concurrency syllabus ka sabse mushkil chapter hai — definitions ko verbatim yaad karo.

Unit V Summary

Modern C++ concurrent programming ko first-class citizen bana deta hai. Threads `std::thread` se launch hoti hain ya better, `std::jthread` se; shared data `std::mutex` aur ek RAII wrapper (`lock_guard`, `scoped_lock`) se protect hota hai; simple counters ke liye `std::atomic` best hai; asynchronous kaam `std::future` return karta hai. C++20 aage **coroutines** introduce karta hai — functions jo `co_await`, `co_yield` aur `co_return` se suspend aur resume hote hain.

Important Definitions

Term	Ek sentence mein definition
Concurrency	Program ki aisi structure jisme multiple activities overlapping intervals mein progress karti hain.
Parallelism	Separate hardware units par multiple activities ka simultaneous execution.
Process	Chalte hue program ka instance jiska apna address space hota hai.
Thread	Process ke andar execution ka independent path, jo uska address space share karta hai.
Race condition	Aisi situation jisme program ki correctness concurrent operations ke exact ordering par depend karti hai.
Data race	Do threads ka ek hi memory ko unsynchronised access karna jisme kam se kam ek write ho — undefined behaviour.
Deadlock	Do ya zyada threads ka ek-doosre ki hold ki hui lock ka wait karna.
Mutex	Primitive jo ek time par sirf ek thread ko critical section execute karne deta hai.
lock_guard	RAII wrapper jo ek mutex lock karta hai aur destructor mein unlock karta hai.
scoped_lock	C++17 RAII wrapper jo bina deadlock ke kitne bhi mutexes lock karta hai.
Atomic	Wrapper (`std::atomic<T>`) jo type T ki value par indivisible operations deta hai.
Future	Kisi asynchronous computation ke eventual result ka handle.
async	Factory jo asynchronous computation start karta hai aur future return karta hai.
Coroutine	Function jo suspend aur baad mein resume ho sakta hai, apni local state preserve karte hue.
co_await	Coroutine ko suspend karta hai jab tak awaited object ready na ho.
co_yield	Coroutine ko suspend karta hai aur caller ko value deta hai (generators).
co_return	Coroutine ko terminate karta hai, chahe to final value ke saath.

Important Syntax

C++

```cpp
// Threads
std::thread  t (fn, arg);        t.join();
std::jthread j (fn, arg);        // auto-joins in destructor

// Mutex + RAII
std::mutex m;
{ std::lock_guard  g(m);          /* single lock */ }
{ std::scoped_lock g(m1, m2);     /* multi-lock, deadlock-free */ }

// Atomic
std::atomic<int> n{0}; ++n; n.load(); n.store(5);

// Async / Future
auto f = std::async(std::launch::async, work, arg);
int  r = f.get();

// Coroutines (C++20)
Task    demo(){ co_await std::suspend_always{}; co_return; }
Generator<int> range(int n){ for(int i=0;i<n;++i) co_yield i; }
```

Important Comparisons

Comparison	Ek line mein answer
Concurrency vs Parallelism	Concurrency = structure; parallelism = simultaneous execution.
Process vs Thread	Process = apna address space; thread = process ke andar shared address space.
thread vs jthread	jthread auto-join karta hai aur `stop_token` se cooperative cancellation support karta hai.
lock_guard vs scoped_lock	lock_guard = ek mutex; scoped_lock = kitne bhi, deadlock-free.
mutex vs atomic	Mutex arbitrary critical sections protect karta hai; atomic single indivisible reads/writes protect karta hai.
sync vs async execution	Sync: caller wait karta hai. Async: caller aage badhta hai aur baad mein future query karta hai.
co_await vs co_yield vs co_return	await = result ka wait; yield = value baahar dena; return = coroutine end karna.

Expected Exam Questions

- Concurrency aur parallelism ko examples ke saath differentiate karo.
- Process aur thread explain karo. Teen differences batao.
- Race condition kya hota hai? Ek aisa C++ program do jo ise exhibit kare, phir use fix karo.
- `std::jthread` aur cooperative cancellation use karne waala C++20 program likho.
- `lock_guard` aur `scoped_lock` ka difference explain karo.
- Atomic operations kya hote hain? Mutex ki jagah inhe kab use karte ho?
- `std::future` aur `std::async` ko example ke saath explain karo.
- Coroutine kya hota hai? `co_await`, `co_yield`, `co_return` explain karo.
- Coroutines ke chaar applications list karo.

Practice Questions (Graded)

Level	Task
★	Do threads launch karo jo apni-apni id das baar print karein.
★★	Do threads ke beech counter share karo, race condition demonstrate karo, phir mutex se fix karo.
★★★	`std::jthread` aur `stop_token` se aisa worker banao jo demand par rukta ho.
★★★★	`std::async` use karke ek bade vector ka sum do halves mein parallel compute karo.
★★★★★	`co_yield` use karke coroutine-based Fibonacci generator implement karo.

! DO NOT FORGET

Do not forget. Jo `std::thread` na joined ho na detached, wo program terminate kar deta hai. `std::jthread` prefer karo. Mutexes ke around hamesha RAII (`lock_guard` / `scoped_lock`) use karo. Coroutine ke andar se kabhi `return` mat call karo — `co_return` use karo.

Ayush Jha

BACK MATTER

CPP Master Revision Guide

Programming Practice Set · Glossary · Final Exam Checklist

All five units condensed for the last day · the last hour · the last five minutes

CPP Master Revision Guide

Book ka ye aakhri section **Programming for Problem Solving – II (C++)** ke poore syllabus ko aisi form mein collapse karta hai jise aap paper se pehle wale din start se finish tak padh sakte ho, exam hall jaane se pehle wale ghante mein padh sakte ho, aur apna pehla answer likhne se pehle ke last five minutes mein bhi padh sakte ho. Har unit represented hai; official syllabus ka har named topic kahin na kahin is checklist mein zaroor maujood hai.

> **IMPORTANT**
>
> **Is guide ko kaise use karein.** Sabse pehle checklist padho taaki verify ho jaaye ki aapne syllabus ka har topic padh liya hai. Phir unit-wise concept dump padho. Last day par exam-trap list aur last-hour sheet ko dohrao. Paper wali subah sirf five-minute sheet padho.

1. Complete Syllabus Checklist

Har item ko revise karte hi tick karo. Paper se ek din pehle agar koi item unchecked hai, to use sabse pehle revise karo.

Unit I — Advanced Object-Oriented Programming

☐	Topic	Kahan cover hua
☐	Review of C++ fundamentals & OOP concepts	§1.1
☐	Runtime polymorphism	§1.2
☐	Virtual functions	§1.3
☐	Pure virtual functions	§1.4
☐	Abstract classes	§1.5
☐	Function overriding	§1.6
☐	Dynamic binding	§1.7
☐	Virtual destructors	§1.8
☐	SOLID principles (S, O, L, I, D)	§1.9

Unit II — STL and C++20 Ranges

☐	Topic	Kahan cover hua
☐	STL kya hai? Containers, iterators, algorithms	§2.1
☐	`std::array, vector, deque, list, forward_list`	§2.2.1–2.2.5
☐	`std::stack, queue, priority_queue`	§2.2.6–2.2.8
☐	`std::set, map, unordered_map`	§2.2.9–2.2.11
☐	Iterator categories (Input, Output, Forward, Bidirectional, Random-Access)	§2.3
☐	Algorithms: sort, stable_sort, find, find_if, count, transform, for_each, copy	§2.4.1–2.4.8
☐	Algorithms: remove, replace, reverse, rotate, partition	§2.4.9–2.4.13
☐	Algorithms: binary_search, lower_bound, upper_bound, accumulate	§2.4.14–2.4.17

Unit III — Modern C++20 Language Features

☐	Topic	Kahan cover hua
☐	Type deduction: `auto`, `decltype`, `decltype(auto)`	§3.1
☐	Compile-time: `constexpr`, `consteval`, `constinit`	§3.2
☐	Uniform & designated initialisation	§3.3
☐	Smart pointers: `unique_ptr, shared_ptr, weak_ptr`	§3.4.1–3.4.5
☐	Rvalue references, move semantics, perfect forwarding	§3.4.6–3.4.8
☐	Structured bindings, optional, variant, any, pair, tuple	§3.5.1–3.5.6
☐	span, string_view, filesystem, chrono	§3.5.7–3.5.10
☐	Spaceship operator <=>	§3.6.1
☐	Attributes: nodiscard, likely, unlikely, deprecated	§3.6.2–3.6.4
☐	`enum class`	§3.6.5

Unit IV — Object-Oriented Design Patterns

☐	Topic	Kahan cover hua
☐	Design pattern kya hota hai?	§4.1
☐	Creational: Factory, Singleton, Builder, Prototype	§4.2
☐	Structural: Adapter, Decorator, Facade	§4.3
☐	Behavioural: Strategy, Observer, State	§4.4

Unit V — Concurrent Programming and Modern Applications

☐	Topic	Kahan cover hua
☐	Concurrency vs parallelism; process vs thread; shared data ke problems	§5.1
☐	`std::thread` and `std::jthread`	§5.2–5.3
☐	Mutex, lock_guard, scoped_lock	§5.4–5.6
☐	Atomic operations	§5.7
☐	Futures and async	§5.8
☐	Coroutines & teen keywords (`co_await, co_yield, co_return`)	§5.9–5.13
☐	Coroutines ke applications	§5.14

2. Unit-Wise Key Concepts

Har unit ko un ideas mein condense kiya gaya hai jinki exam mein sabse zyada weight hoti hai.

Unit I — Advanced OOP

OOP **encapsulation, abstraction, inheritance aur polymorphism** par bani hai. Polymorphism ya to **compile-time** hoti hai (overloading, templates) ya **runtime** (base pointer ya reference ke through virtual functions). **Virtual function** run time par har-class **vtable** aur har-object **vptr** ke through select hota hai. **Pure virtual function** (`= 0`) apni class ko **abstract** bana deta hai aur har concrete derived class par override karna force karta hai. Function overriding ke liye base function `virtual` hona chahiye aur derived signature exactly match karni chahiye — compile time par mistakes pakadne ke liye `override` use karo. Jab class base pointer ke through use hoti hai tab base destructor **virtual hona hi chahiye**, warna deletion undefined behaviour hai. **SOLID** principles sound object-oriented design guide karte hain: Single-responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion.

Unit II — STL and C++20 Ranges

STL **containers + iterators + algorithms** hai — sab templates ke roop mein likhe gaye. Default container `std::vector` hai; baaki sequence containers (array, deque, list, forward_list), adaptors (stack, queue, priority_queue) aur associative containers (set, map, unordered_map) sirf tab choose hote hain jab `vector` suitable na ho. Iterators paanch categories mein aate hain — Input, Output, Forward, Bidirectional, Random-Access — aur har container wahi sabse strong category publish karta hai jo wo support kar sakta hai. STL algorithms half-open ranges `[first, last)` par operate karte hain aur container ke baare mein kuch nahi jaante. Syllabus ke ye satrah algorithms master karo: sort, stable_sort, find, find_if, count, transform, for_each, copy, remove, replace, reverse, rotate, partition, binary_search, lower_bound, upper_bound, accumulate.

Unit III — Modern C++20

Modern C++ manual bookkeeping ki jagah language features deta hai. Types deduce karne ke liye `auto` use karo, exact-type mirroring ke liye `decltype`, aur return types perfectly forward karne ke liye `decltype(auto)`. Computation ko compile time par le jaao — `constexpr` / `consteval` / `constinit` trio ke saath. Memory ko smart pointers se manage karo: exclusive ownership ke liye `unique_ptr`, shared ownership ke liye `shared_ptr`, observe karne ya cycles todne ke liye `weak_ptr`. `make_unique` aur `make_shared` prefer karo. Rvalues se steal karne ke liye move semantics aur template parameters relay karne ke liye perfect forwarding use karo. Utility library deti hai `optional` (nothing ya T), `variant` (fixed set mein se ek), `any` (kisi bhi type ka ek), `span` aur `string_view` (non-owning views), plus `filesystem` aur `chrono`. Spaceship operator `<=>` chhah ke chhah comparison operators generate karta hai; `enum class` type-safe, scoped enumerations deta hai.

Unit IV — Design Patterns

Design patterns named, reusable design ideas hote hain — na code, na libraries. **Creational** patterns object creation ki details chhupate hain (*Factory, Singleton, Builder, Prototype*). **Structural** patterns classes aur objects ko compose karte hain (*Adapter, Decorator, Facade*). **Behavioural** patterns describe karte hain ki objects aapas mein kaise communicate karte hain (*Strategy, Observer, State*). Har pattern Unit I ka polymorphism aur Unit III ke smart pointers use karta hai; har pattern kisi ek SOLID principle ko bhi implement karta hai.

Ayush Jha

Unit V — Concurrency & Coroutines

Concurrency ek *structural* property hai; parallelism *simultaneous execution* hai. Process apna address space own karta hai; threads apna share karte hain aur shared data ke access ko synchronise karna padta hai. Unsynchronised sharing se **data race** banta hai, jo undefined behaviour hai. Threads ke liye `std::jthread` use karo (auto-join, cooperative stop), single mutex ke liye `lock_guard`, kai mutexes ke liye `scoped_lock`, indivisible single-value operations ke liye `std::atomic`, aur asynchronous kaam ke liye `std::async` + `std::future`. C++20 coroutines **suspend** aur **resume** ho sakte hain: `co_await` awaitable ka wait karta hai, `co_yield` value baahar deta hai (generator), aur `co_return` coroutine ko terminate karta hai.

Ayush Jha

3. All Important Definitions (Consolidated)

Definitions ka single alphabetical dump jo aapko syllabus ke kisi bhi unit se, ek sentence mein dene aane chahiye.

Term	Ek sentence mein definition
Abstract class	Aisi class jisme kam se kam ek pure virtual function ho; instantiate nahi ho sakti.
Abstraction	Sirf essential features expose karna aur implementation ko interface ke peechhe chhupana.
Adapter (pattern)	Existing class ke interface ko usme convert karta hai jo client expect karta hai.
any	Wrapper jo kisi bhi copyable type ki value rakhta hai.
Async (std::async)	Factory jo asynchronous computation start karta hai aur future return karta hai.
Atomic	Wrapper jo ek single value par indivisible operations deta hai.
auto	Type-deduction placeholder jo top-level const aur references strip kar deta hai.
Builder (pattern)	Complex object ko fluent interface ke through step-by-step banata hai.
Coroutine	Function jo suspend aur baad mein resume ho sakta hai, apni state preserve karte hue.
Concurrency	Program ki structure jisme activities overlapping intervals mein progress karti hain.
const	Immutable variable; shayad compile-time-initialised ho.
constexpr	Function ya variable jo compile-time evaluation ke liye eligible hai.
consteval	Function jiska compile time par evaluate hona must hai.
constinit	Variable jiska compile time par initialise hona must hai; run time par mutable reh sakta hai.
Data race	Do threads ka ek hi memory ko unsynchronised access karna jisme kam se kam ek write ho - UB.
Decorator (pattern)	Object ko same interface waale doosre object mein wrap karke responsibilities attach karta hai.
decltype	Operator jo expression ka exact type deta hai.
Deadlock	Do ya zyada threads jo ek-doosre ki chahiye waali lock hold kiye hain.
Dependency Inversion	High-level modules ko concretions par nahi, abstractions par depend karna chahiye (SOLID ka D).
Design pattern	Kisi recurring design problem ka named, reusable solution.
Dynamic binding	Vtable-driven runtime selection of a virtual function.
Encapsulation	Data aur uske operations ko class mein bundle karna aur external access restrict karna.
Facade (pattern)	Complex subsystem ke upar ek single high-level interface.
Factory (pattern)	Object creation ko ek single function ke peechhe encapsulate karta hai.
Future (std::future)	Kisi asynchronous computation ke eventual result ka handle.
Function overriding	Derived class base ke virtual function ka apna version identical signature ke saath deti hai.
Inheritance	Derived class base class ke members acquire karti hai.

Term	Ek sentence mein definition
Interface Segregation	Ek bade interface ki jagah kai chhote focused interfaces prefer karo (SOLID ka I).
Iterator	Pointer-jaisa object jo container ke andar kisi position ko identify karta hai.
jthread	C++20 thread jo destructor mein auto-join hota hai aur stop_token support karta hai.
Liskov Substitution	Subtypes ko apne base types ki jagah substitutable hona chahiye (SOLID ka L).
lock_guard	RAII wrapper jo ek mutex lock karta hai aur destruction par unlock karta hai.
Move semantics	Jald-destroy-hone-waale object se resources ka transfer, copy se bach kar.
Mutex	Synchronisation primitive jo critical section tak one-thread-at-a-time access deta hai.
Observer (pattern)	One-to-many change-notification relationship.
Open/Closed	Extension ke liye open, modification ke liye closed (SOLID ka O).
Optional	T ka wrapper jisme value ho bhi sakti hai aur nahi bhi.
Perfect forwarding	Template technique jo har argument ko uski value category preserved ke saath relay karti hai.
Polymorphism	Ek interface, kai behaviours.
Prototype (pattern)	Existing objects ko clone karke naye objects banata hai.
Pure virtual function	Virtual function jiska base body nahi hota (= 0), apni class ko abstract bana deta hai.
Race condition	Correctness concurrent operations ke exact ordering par depend karti hai.
Rvalue reference	T&& — sirf rvalues ko bind hota hai; move semantics enable karta hai.
scoped_lock	C++17 RAII wrapper jo bina deadlock ke kitne bhi mutexes lock karta hai.
shared_ptr	Reference-counted smart pointer jo shared ownership express karta hai.
Single Responsibility	Ek class ka change hone ka sirf ek hi reason hona chahiye (SOLID ka S).
Singleton (pattern)	Exactly ek instance ki guarantee deta hai aur global access point provide karta hai.
Spaceship operator	<=> — three-way comparison jo chhah ke chhah comparison operators generate karta hai.
span	Kisi sequence ke upar non-owning contiguous view.
State (pattern)	Object state objects ko delegate karke behaviour badalta hai, jo ek-doosre ko replace karte hain.
STL	Standard library ka template-based hissa jo containers, iterators aur algorithms deta hai.
Strategy (pattern)	Interchangeable algorithms ki family, objects ke roop mein encapsulated.
string_view	Character data ke upar non-owning read-only view.
Thread	Process ke andar execution ka independent path jo uska address space share karta hai.
tuple	Pair ka generalisation — kisi bhi fixed number ke heterogeneous elements tak.
unique_ptr	Smart pointer jo exclusive ownership express karta hai; movable hai par copyable nahi.
variant	Fixed alternative types ki list ka type-safe tagged union.

Term	Ek sentence mein definition
Virtual function	Member function marked virtual, taaki base pointer ke through derived override call ho.
Virtual destructor	Base destructor marked virtual, taaki base pointer ke through `delete` par derived destructors chalein.
weak_ptr	shared_ptr ka non-owning observer; lock() ke through upgrade hota hai.
co_await	Coroutine ko suspend karta hai jab tak awaited object ready na ho.
co_yield	Coroutine ko suspend karta hai aur value baahar deta hai (generator).
co_return	Coroutine ko terminate karta hai, chahe to final value ke saath.

Ayush Jha

4. All Important Syntax in One Place

Agar aap ye single code page memory se reproduce kar sakte ho, to is course ke kisi bhi paper ka koi bhi syntax question answer kar sakte ho.

Unit I — Advanced OOP

```C++
class Base {
public:
    virtual void f();                       // virtual
    virtual void g() = 0;                   // pure virtual
    virtual ~Base() = default;              // virtual destructor
};
class Derived : public Base {
public:
    void f() override;                      // correct override
    void g() override;                      // must override pure virtual
};
class Sealed final {};                      // no further inheritance
```

Unit II — STL

```C++
#include <vector>
#include <algorithm>
#include <numeric>
std::vector<int> v { 5, 2, 9, 1, 7 };
std::sort   (v.begin(), v.end());
auto it = std::find(v.begin(), v.end(), 7);
int  n  = std::count(v.begin(), v.end(), 5);
std::transform(v.begin(), v.end(), v.begin(), [](int x){ return x*x; });
int  s  = std::accumulate(v.begin(), v.end(), 0);
```

Unit III — Modern C++20

```C++
auto           n = 42;
decltype(n)    m = n;
decltype(auto) r = getRef();

constexpr int sq(int x){ return x*x; }
consteval int cube(int x){ return x*x*x; }
constinit int counter = 0;

auto up = std::make_unique<T>(args...);
auto sp = std::make_shared<T>(args...);
std::weak_ptr<T> wp = sp;

struct V { int x;
    auto operator<=>(const V&) const = default;   // spaceship
};
enum class Color { Red, Green, Blue };            // scoped enum
```

Unit IV — Design Patterns (essential skeletons)

C++

```cpp
// Singleton (Meyers)
class Logger {
    Logger() = default;
public:
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;
    static Logger& instance() { static Logger s; return s; }
};

// Factory
class ShapeFactory {
public:
    static std::unique_ptr<Shape> create(const std::string& kind);
};

// Strategy
class Strategy { public: virtual void run() = 0; virtual ~Strategy() = default; };
class Context  { std::unique_ptr<Strategy> s; public:
    void setStrategy(std::unique_ptr<Strategy> x){ s = std::move(x); }
    void execute(){ s->run(); }
};
```

Unit V — Concurrency & Coroutines

C++

```cpp
#include <thread>
#include <mutex>
#include <atomic>
#include <future>
#include <coroutine>

std::jthread j([](std::stop_token st){ while(!st.stop_requested()){} });
std::mutex m; { std::scoped_lock g(m); }
std::atomic<int> a{0}; ++a;

auto f = std::async(std::launch::async, work, 42);
int  r = f.get();

// Coroutine
Task demo(){ co_await std::suspend_always{}; co_return; }
Generator<int> range(int n){ for(int i=0;i<n;++i) co_yield i; }
```

5. Master Comparison Table

Har comparison jo book mein kahin bhi use hua hai, last-minute revision ke liye ek hi table mein collect kiya gaya hai.

Comparison	Ek line mein difference
Overloading vs Overriding	Same class + different sigs vs base+derived + same sig + virtual.
Compile-time vs Runtime polymorphism	Compiler resolve karta hai vs run time par vtable resolve karta hai.
Virtual vs Pure virtual	Base body hoti hai vs = 0 hota hai aur override force karta hai.
Abstract vs Concrete class	≥ 1 pure virtual, objects nahi vs koi nahi, objects allowed.
Static vs Dynamic binding	Compiler pick karta hai vs run time par vtable pick karta hai.
Overriding vs Hiding	Virtual + matching sig vs same naam waala kuch bhi aur.
vector vs list	Contiguous, cache-friendly vs linked, O(1) middle insert.
array vs vector	Fixed compile-time size vs runtime par grow hota hai.
stack vs queue	LIFO vs FIFO.
set vs unordered_set	Sorted, O(log n) vs hashed, O(1) average.
map vs unordered_map	Key se sorted, tree vs hashed, faster average.
sort vs stable_sort	stable_sort equal elements ka relative order preserve karta hai.
find vs find_if	Value match karta hai vs predicate match karta hai.
lower_bound vs upper_bound	Pehla ≥ value vs pehla > value.
remove vs erase	remove reorder karta hai, erase delete karta hai — dono saath use karo.
auto vs decltype vs decltype(auto)	Strip karta hai vs exact vs auto with decltype rules.
constexpr vs consteval vs constinit	CT par ho sakta hai vs CT par hona hi hai vs sirf CT init.
unique vs shared vs weak_ptr	Exclusive vs shared vs observing.
Copy vs Move semantics	Duplicate karta hai vs rvalue se steal karta hai.
optional vs variant vs any	Nothing-ya-ek vs fixed-mein-se-ek vs kisi-bhi-type-ka-ek.
string vs string_view	Storage own karta hai vs sirf observe karta hai.
enum vs enum class	Unscoped, int mein convert hota hai vs scoped, strongly typed.
Factory vs Prototype	Naya banata hai vs existing ki copy karta hai.
Factory vs Builder	Ek call vs kai named steps.
Adapter vs Decorator	Interface badalta hai vs interface preserve + behaviour add karta hai.
Decorator vs Facade	Ek object wrap karta hai vs poora subsystem wrap karta hai.
Strategy vs State	Client choose karta hai vs object time ke saath apni strategy khud badalta hai.
Observer vs Strategy	Kayi ko notify karta hai vs ek ko delegate karta hai.

Comparison	Ek line mein difference
Concurrency vs Parallelism	Structure vs simultaneous execution.
Process vs Thread	Apna address space vs baaki threads ke saath shared.
thread vs jthread	Manual join vs auto-join + stop_token.
lock_guard vs scoped_lock	Ek mutex vs kitne bhi, deadlock-free.
mutex vs atomic	Critical section vs single-value indivisible op.
Synchronous vs Asynchronous	Caller wait karta hai vs caller aage badhta hai; result future se.
co_await vs co_yield vs co_return	Wait karna vs value baahar dena vs coroutine end karna.

Ayush Jha

6. Common Mistakes & Exam Traps

Inmein se har ek ne pichhle papers mein students ke aasaan marks ganwaaye hain.

Trap	Sahi answer
Constructor se virtual function call karna	Base version chalta hai — derived vtable abhi bani hi nahi hoti.
Derived object ko base pointer se delete karna jab base destructor non-virtual ho	Undefined behaviour — base destructor ko virtual declare karo.
Polymorphic object ko value se pass karna	Object slicing — pointer ya reference se pass karo.
Sirf return type ko overloading distinction maan na	Allowed nahi — signatures parameters mein differ karni chahiye.
Maan na ki insertion ke liye `list` `vector` se better hai	Practice mein cache locality ki wajah se vector jeet-ta hai.
Unsorted range par `lower_bound` call karna	Behaviour unspecified hai — pehle sort karo.
Maan na ki `auto` references preserve karta hai	Nahi karta — `auto&` ya `decltype(auto)` use karo.
`constinit` ko const ka synonym samajhna	`constinit` sirf initialisation pin karta hai; variable baad mein mutate ho sakta hai.
`shared_ptr` ke owned object ko manually delete karna	Double-free — kabhi manually delete mat karo.
Do `shared_ptr`s ka ek-doosre ko refer karna	Reference cycle — `weak_ptr` se todo.
Das 5s banane ke liye `vector<int> v{10,5}` likhna	Ye syntax do-element vector banata hai; `(10,5)` use karo.
Unrelated enums ke raw `enum` values compare karna	int conversion ki wajah se compile ho jaata hai par bug hai — `enum class` use karo.
`std::thread` bina join/detach ke destroy hona	Program terminate ho jaata hai — `jthread` use karo.
Do threads mein ek mutex, phir doosra, alag-alag order mein lock karna	Deadlock — `scoped_lock` use karo.
Kai related variables par `atomic` use karna	Atomicity sirf per-object hoti hai — mutex use karo.
Coroutine ke andar `return` likhna	Compilation error — `co_return` use karo.

7. Last-Day Revision Plan (Six-Hour Layout)

Agar aapke paas paper se exactly ek din pehle ka din hai, to ye schedule use karo.

Slot	Duration	Kya karna hai
Morning — Slot 1	1h	Unit I revision sheet padho + vtable diagram reproduce karo + virtual/pure/abstract/override skeletons memory se likho.
Morning — Slot 2	1h	Unit II revision sheet padho + container-choice decision table blank sheet par dobara likho + erase-remove idiom code karo.
Afternoon — Slot 3	1h	Unit III revision sheet padho + smart-pointer comparison table reproduce karo + move-constructor Buffer example.
Afternoon — Slot 4	1h	Unit IV revision sheet padho + ek Creational + ek Structural + ek Behavioural pattern C++ mein memory se reproduce karo.
Evening — Slot 5	1h	Unit V revision sheet padho + mutex-protected counter aur coroutine Generator<int> code karo.
Evening — Slot 6	1h	Is Master Revision Guide ko end-to-end skim karo, phir §6 (Common Mistakes) padho.

◆ **REMEMBER**

Revision ke last hour se zyada neend matter karti hai. Paper se ek din pehle raat ko midnight ke baad mat jaago. Thake hue dimaag sabse pehle syntax bhool jaate hain.

8. One-Hour Revision Sheet

Paper se turant pehle wale ghante mein use karo. Ek baar padho, phir book band karo.

Unit	Yaad rakhne waali ek cheez
I	Runtime polymorphism = virtual + base pointer + overriding derived class. Jab bhi polymorphic use ho, base destructor virtual. SOLID = S O L I D.
II	STL = containers + iterators + algorithms. Default = vector. Iterators: Input, Output, Forward, Bidirectional, Random-Access. Satrah ke satrah algorithm names yaad karo.
III	auto strips; decltype exact; decltype(auto) dono. constexpr/consteval/constinit family. unique / shared / weak. Move rvalues se steal karta hai. optional/variant/any. Spaceship = <=>. enum class > enum.
IV	Design pattern = named reusable idea. Creational (Factory, Singleton, Builder, Prototype); Structural (Adapter, Decorator, Facade); Behavioural (Strategy, Observer, State).
V	Concurrency = structure; parallelism = execution. jthread auto-join karta hai. RAII locks. Ek value ke liye atomic; kayi ke liye mutex. async + future. Coroutine keywords: co_await / co_yield / co_return.

Paanch Formulas Jin par Kabhi Blank Mat Hona

C++

```
1. class Base { public: virtual T f() = 0; virtual ~Base()=default; };
2. std::sort(v.begin(), v.end());
3. auto p = std::make_unique<T>(args...);
4. { std::scoped_lock g(m); /* critical section */ }
5. Generator<int> range(int n){ for(int i=0;i<n;++i) co_yield i; }
```

9. Ultra-Short Five-Minute Revision Sheet

Jab invigilator papers baant raha ho tab padho. Bees lines. Har line marks ki kaabil hai.

01. OOP pillars: Encapsulation, Abstraction, Inheritance, Polymorphism.

02. Runtime polymorphism = virtual + base pointer + overriding derived class.

03. Pure virtual = 0 ⇒ abstract class ⇒ instantiate nahi ho sakti.

04. Jab bhi class polymorphically use ho, base destructor virtual.

05. SOLID = SRP + OCP + LSP + ISP + DIP.

06. STL = containers + iterators + algorithms. Default container = vector.

07. Iterator categories: Input, Output, Forward, Bidirectional, Random-Access.

08. Erase-remove: `v.erase(remove(v.begin(),v.end(),x), v.end());`

09. lower_bound: pehla ≥ x. upper_bound: pehla > x.

10. auto ref/const strip karta hai; decltype exact hai; decltype(auto) exact-deducing hai.

11. constexpr dono time par chal sakta hai; consteval CT par hona hi hai; constinit sirf CT-init hai.

12. unique_ptr = exclusive; shared_ptr = reference-counted; weak_ptr = observer.

13. Move ctor: `T(T&& o) noexcept` o se steal karta hai.

14. optional = nothing-ya-T; variant = fixed-mein-se-ek; any = kisi-bhi-type-ka-ek.

15. Spaceship <=> + = default se chhah ke chhah comparisons milte hain.

16. enum class = scoped, int mein koi implicit conversion nahi.

17. Design patterns Creational, Structural, Behavioural mein split hote hain.

18. Singleton = private ctor + static instance(); Meyers version thread-safe hai.

19. jthread auto-join karta hai; scoped_lock kai mutexes deadlock-free lock karta hai.

20. Coroutine keywords: co_await (wait), co_yield (value baahar), co_return (end).

◆ **REMEMBER**

Ab book band karo. Apne gyaan par bharosa karo. Paper shaant hokar shuru karo.

Ayush Jha

10. Programming Practice Set (All Five Units)

Consolidated practice set. Har problem official syllabus se li gayi hai. Har ek ko pehle pen aur paper se attempt karo, phir compiler par verify karo.

Unit I — Advanced OOP

- Encapsulation demonstrate karne waali class `BankAccount` design karo, deposit aur withdraw ke saath.
- Base class `Shape` aur derived `Circle, Square, Triangle` likho, virtual `area()` ke saath.
- Upar waale ko pure virtual `draw()` ke saath extend karo. Shapes ko `vector<unique_ptr<Shape>>` mein store karo.
- DIP demonstrate karne waala `IPayment` interface design karo, CreditCard aur UPI implementations ke saath.
- Ek aisa program likho jo jaan boojh kar memory leak kare kyunki base destructor virtual nahi hai, phir use fix karo.

Unit II — STL

- `vector<int>` mein 10 integers read karo, unhe sort karo, aur reverse mein print karo.
- `std::count` use karke count karo ki diya gaya value vector mein kitni baar aata hai.
- Erase-remove idiom se kisi value ki har occurrence erase karo.
- `unordered_map<string,int>` se word-frequency counter banao.
- `priority_queue` use karke input stream ke k sabse bade numbers print karo.
- Sorted vector diya ho, to lower_bound / upper_bound se count karo ki kitne values `[lo, hi]` mein lie karte hain.

Unit III — Modern C++20

- `std::optional<double>` return karne waala `safeDivide(a,b)` likho.
- Move constructor waali `Buffer` class implement karo; dikhao ki temporary move hota hai, copy nahi.
- Designated initialisers se initialise hone waala aur structured bindings se print hone waala `Config` struct likho.
- Ek `Tree` banao jisme children apne parent ko `weak_ptr` se refer karein taaki cycles na banein.
- Aisa `Version` struct define karo jo defaulted spaceship operator se chhah ke chhah comparison operators generate kare.

Unit IV — Design Patterns

- Meyers idiom se `Config` class ke liye Singleton pattern implement karo.
- Ek `ShapeFactory` implement karo jo string se Circle, Square aur Triangle banaye.
- Ek `PizzaBuilder` implement karo jo base aur toppings ke saath Pizza assemble kare.
- Espresso plus Milk aur Sugar decorators waali Beverage-Decorator hierarchy implement karo.
- `Stock` ke liye Observer pattern implement karo jo `Screen` aur `Log` subscribers ko notify kare.
- Ek TrafficLight State machine implement karo jo Red → Green → Yellow → Red cycle kare.

Unit V — Concurrency & Coroutines

- Do threads launch karo jo dono apni id print karein; main se unhe join karo.
- Shared counter par data race demonstrate karo, phir `lock_guard` se fix karo.
- Wahi race `std::atomic<int>` se fix karo.
- `std::async` use karke bade vector ka sum do parallel halves mein compute karo.
- `jthread` plus `stop_token` se aisa worker banao jise main se rukne ko kaha ja sake.
- `co_yield` waali coroutine se Fibonacci generator implement karo.

Ayush Jha

11. Complete Glossary of Terms

Book mein kahin bhi introduce hua har technical term, alphabetically, ek simple par academically-sahi definition ke saath.

Term	Definition
Abstract class	Aisi class jisme kam se kam ek pure virtual function ho; instances nahi ban sakte.
Abstraction	Public interface ke through sirf essential features expose karna.
Access specifier	public / protected / private — member visibility control karte hain.
Adapter	Structural design pattern jo existing interface ko usme convert karta hai jo client expect karta hai.
Aggregate	Aisa struct ya class jisme sirf public data ho, koi user-declared constructor nahi aur koi virtuals nahi.
Algorithm (STL)	Function template jo iterator ranges par operate karta hai.
Allocator	STL object jo container ke memory management ke liye responsible hota hai.
any (std::any)	Kisi bhi copyable type ki value rakhta hai.
Argument-dependent lookup	Name lookup jo argument types ke namespaces ko bhi consider karta hai.
Async	Factory jo task launch karta hai aur uske result ke liye std::future return karta hai.
Atomic	Aisa object jiske operations concurrent access ke baawajood indivisible hote hain.
Attribute	Compiler ke liye bracketed hint (`[[nodiscard]]`, etc.).
auto	Placeholder type jo initialiser se deduce hota hai.
Behavioural pattern	Design pattern jo algorithms aur responsibilities ki assignment se concerned hota hai.
Bidirectional iterator	Iterator jo aage aur peeche dono move kar sakta hai.
Binding	Naam ko implementation se jodne ka process; static (compile time) ya dynamic (run time).
Builder	Creational pattern jo complex objects ko step-by-step banata hai.
C++20	Is course mein padha jaane waala C++ language standard.
Callable	Kuch bhi jo call kiya ja sake: function, function pointer, functor, lambda.
chrono	Header jo strongly-typed durations, time-points aur clocks deta hai.
Class	User-defined type jisme data aur function members hote hain.
const	Qualifier jo entity ko immutable banata hai.
constexpr	Specifier jo compile-time evaluation permit karta hai.
consteval	Specifier jo compile-time evaluation require karta hai.
constinit	Specifier jo compile-time initialisation require karta hai.
Container adaptor	Sequence container ke upar bana STL container (stack, queue, priority_queue).
Coroutine	Function jo suspend aur resume ho sakta hai, apni local state preserve karte hue.

Term	Definition
Coroutine frame	Heap-allocated storage jo suspended coroutine ki state rakhti hai.
Covariant return type	Overriding function base return type ka derived type return kar sakta hai.
Creational pattern	Design pattern jo control karta hai ki objects kaise bante hain.
Data race	Unsynchronised concurrent access (jisme kam se kam ek write ho) se aane waala undefined behaviour.
decltype	Compile-time operator jo expression ka exact type deta hai.
decltype(auto)	Placeholder jo decltype rules se deduce karta hai.
Decorator	Structural pattern jo wrap karke responsibilities attach karta hai.
Deep copy	Copy jo har pointer ke target ko duplicate karti hai, shared ownership se bachati hai.
Deferred launch	std::async ki policy jo task ko calling thread par tab chalaati hai jab get() invoke ho.
Delegate	Object jis par doosra object koi responsibility karne ke liye rely karta hai.
Deleted function	Function declared with `= delete` — wo operation disable ho jaata hai.
Dependency Inversion	Concretions par nahi, abstractions par depend karo (SOLID ka D).
Design pattern	Named, reusable design idea.
Detach (thread)	Thread ko independently chalne ke liye release karna; handle ab join nahi karta.
Dynamic binding	Vtable-driven runtime selection of a virtual function.
Encapsulation	Data aur uske operations ko bundle karna, restricted external access ke saath.
enum class	Scoped, strongly-typed enumeration.
Erase-remove idiom	Combined `v.erase(std::remove(..), v.end());`
Exception	Object jo error signal karne ke liye throw hota hai aur stack ke upar propagate karta hai.
Explicit constructor	Constructor jo implicit conversions forbid karta hai.
Facade	Structural pattern jo subsystem ke upar simplified interface deta hai.
Factory	Creational pattern jo object creation ko encapsulate karta hai.
filesystem	Portable file-system operations ke liye C++17 library.
final	Specifier jo further overriding ya inheritance rokta hai.
for_each	STL algorithm jo range ke har element par callable apply karta hai.
Forward iterator	Iterator jo sirf multi-pass forward traversal support karta hai.
Forwarding reference	Deduced context mein `T&&`; lvalues aur rvalues dono ko bind hota hai.
Function objects	Objects jo `()` ke saath usable hain; lambdas included.
Function overloading	Ek scope mein same naam par different signatures waale multiple functions.
Function overriding	Derived class base virtual ko identical signature se redefine karti hai.
Future	Kisi asynchronous task ke eventual result ka handle.

Term	Definition
Generic programming	Templates ke saath programming taaki code kisi bhi suitable type par kaam kare.
hardware_concurrency	Static function jo hardware ka suggested thread count return karta hai.
Hash table	Unordered associative data structure jo hash function use karti hai.
Header (STL)	Standard header jo STL component expose karta hai (`<vector>`, etc.).
Immediate function	Function marked `consteval`.
Inheritance	Reuse mechanism jisme derived class base-class members acquire karti hai.
Initializer list	`std::initializer_list<T>` jo brace-initialised constructors use karte hain.
Input iterator	Iterator jo single-pass reading support karta hai.
Interface	C++ mein purely abstract class ke liye convention name.
Interface segregation	Ek bade interface ki jagah kai chhote interfaces prefer karo (SOLID ka I).
Iterator	Pointer-jaisa object jo container ke andar kisi position ko identify karta hai.
Join (thread)	Thread ke finish hone ka wait karna aur uske resources release karna.
jthread	C++20 thread jo destructor mein auto-join hota hai aur stop_token support karta hai.
Lambda expression	Anonymous callable object, syntax `[capture](args){ body }` ke saath.
Late binding	Dynamic binding ka doosra naam.
likely / unlikely	Branch probability hint dene waale attributes.
Liskov substitution	Subtypes ko base types ki jagah substitutable hona chahiye (SOLID ka L).
Literal type	Type jo compile-time constant ke roop mein usable ho.
lock_guard	Single mutex ke liye RAII wrapper.
Loose coupling	Design property jisme components concretions par nahi abstractions par depend karte hain.
Lvalue	Expression jiski identity persist karti hai (address hota hai).
make_shared / make_unique	Smart pointers ke liye preferred factories.
Map	Sorted associative container jo unique keys ko values se map karta hai.
Memory order	Atomic operations par tag jo control karta hai ki surrounding accesses kaise ordered honge.
Meyers singleton	Function-local static se implemented singleton; C++11 se thread-safe.
Modern C++	C++11 aur baad ke versions; yahan specifically C++20.
Move constructor	Constructor jo `T&&` leta hai; temporary se resources steal karta hai.
Move semantics	Resources copy karne ki jagah transfer karna.
Multiple inheritance	Class ka ek se zyada bases se inherit karna.
Mutex	Mutual-exclusion primitive.

Term	Definition
Namespace	Identifiers ke liye named scope.
Non-owning	Aise object ko refer karta hai jise wo manage nahi karta.
Observer	One-to-many notification ke liye behavioural pattern.
Open/closed	Extension ke liye open, modification ke liye closed (SOLID ka O).
Optional	`std::optional<T>` — nothing ya T.
Output iterator	Iterator jo single-pass writing support karta hai.
Overload resolution	Overloaded functions mein se compiler-time selection.
Overloading	Same naam, different signatures, same scope.
Override (keyword)	Compile time par verify karta hai ki member base virtual ko override karta hai.
Overriding	Base virtual ka derived-class version dena.
Pair	Do-element heterogeneous tuple `std::pair<A,B>`.
Parallelism	Separate cores par tasks ka simultaneous execution.
Partition (algorithm)	Range ko reorder karta hai taaki predicate-true elements false waalon se pehle aayein.
Perfect forwarding	Arguments ki value category ko template ke through preserve karna.
Polymorphism	Ek interface, kai behaviours.
Predicate	Callable jo bool return karta hai, STL algorithms use karte hain.
priority_queue	STL adaptor jo sabse bade element tak O(log n) access deta hai.
Process	Chalte hue program ka instance jiska apna address space hota hai.
Promise (coroutine)	Object jo coroutine ke suspension aur result behaviour ko control karta hai.
Prototype	Creational pattern jo existing objects ko clone karke naye banata hai.
Pure virtual	Virtual function jiske saath `= 0` hota hai.
RAII	Resource Acquisition Is Initialisation — resource lifetime ko object scope se baandhna.
Race condition	Correctness concurrent accesses ke ordering par depend karti hai.
Random-access iterator	Iterator jo `it + n` support karta hai.
Range (STL)	Half-open interval `[first, last)`.
Reference	Kisi existing object ka alias.
Regular type	Type jisme value-jaise operations ka standard set hota hai.
Reserve	Future reallocations se bachne ke liye vector capacity pre-allocate karna.
Rvalue	Expression jiski identity persist nahi karti (ek temporary).
Rvalue reference	`T&&` jo sirf rvalues ko bind hota hai.
scoped_lock	C++17 RAII wrapper jo kai mutexes deadlock-free lock kar sakta hai.

Term	Definition
Sequence container	Container jo elements ko definite order mein store karta hai.
Set	Unique elements ka sorted associative container.
Shallow copy	Copy jo top-level object duplicate karti hai par pointer targets share karti hai.
Shared pointer	Shared ownership ke liye reference-counted smart pointer.
Single responsibility	Har class ke liye change ka ek hi reason (SOLID ka S).
Singleton	Creational pattern jo ek instance ki guarantee deta hai.
Slicing	Polymorphic object ko value se copy karne par derived-class parts ka loss.
Smart pointer	RAII object jo heap-allocated resource own karta hai.
SOLID	Paanch design principles: SRP, OCP, LSP, ISP, DIP.
Spaceship operator	<=> three-way comparison.
Span	Sequence ke upar non-owning contiguous view.
Stack	LIFO adaptor.
Standard attribute	Bracketed hint jo standard recognise karta hai.
State (pattern)	Behavioural pattern jisme object state change hone par strategy badalta hai.
Static binding	Function ka compile-time selection.
STL	C++ standard library ka template portion.
stop_token	Cooperative-cancellation object jo std::jthread support karta hai.
Strategy	Interchangeable algorithms waala behavioural pattern.
Streaming operator	Overloaded << / >>.
String view	Character data ke upar non-owning read-only view.
Structural pattern	Classes aur objects ki composition ke baare mein design pattern.
Structured binding	Tuple-like objects ko named variables mein decompose karta hai.
Subject (Observer)	Object jiske changes observer updates trigger karte hain.
Suspend / Resume	Coroutine ke core operations.
template	Language feature jo generic code enable karta hai.
this pointer	Non-static members ke andar current object ka pointer.
Thread	Process ke andar execution ka independent path.
Time point	Clock par value (chrono).
Tuple	Fixed-size heterogeneous collection.
Type deduction	Compiler ka context se type infer karna.
Undefined behaviour	Program ka aisa behaviour jis par standard koi requirement impose nahi karta.

Term	Definition
Uniform initialisation	Brace-based syntax jo har type par kaam karta hai.
Unique pointer	Smart pointer jo exclusive ownership express karta hai.
Unordered container	Hash-based associative container.
Value category	Expression ki classification (lvalue, xvalue, prvalue, rvalue).
Variant	Type-safe tagged union.
Vector	Contiguous dynamic array — default STL container.
Virtual destructor	Base destructor marked virtual; polymorphic deletion ke liye required.
Virtual function	Member function marked virtual; dynamically dispatch hota hai.
vptr	Hidden per-object pointer jo apni class ki vtable ko point karta hai.
vtable	Virtual functions ke pointers ka per-class array.
Weak pointer	shared_ptr ka non-owning observer.
co_await	Coroutine ko suspend karta hai jab tak awaitable ready na ho.
co_yield	Suspend karta hai aur value baahar deta hai.
co_return	Coroutine ko end karta hai, chahe to value ke saath.

Ayush Jha

12. Final Exam Checklist

Exam wali subah har ek ko tick karo. Har unchecked line aisa topic hai jise abhi ek last minute ki revision chahiye.

Knowledge

☐ Main encapsulation, abstraction, inheritance aur polymorphism ko ek-ek sentence mein define kar sakta hoon.

☐ Main paanchon SOLID principles ka naam aur har ek ka ek C++ example de sakta hoon.

☐ Main bina dekhe virtual function, override, pure virtual aur virtual destructor likh sakta hoon.

☐ Main paanchon iterator categories ko weakest se strongest tak list kar sakta hoon.

☐ Main kam se kam chhe STL containers ke naam bata sakta hoon aur diye gaye problem ke liye sahi container choose kar sakta hoon.

☐ Main syllabus ke satrah algorithms ko naam se bula sakta hoon.

☐ Main `auto`, `decltype` aur `decltype(auto)` mein difference kar sakta hoon.

☐ Main `constexpr`, `consteval` aur `constinit` mein difference kar sakta hoon.

☐ Main `unique_ptr`, `shared_ptr` aur `weak_ptr` mein difference kar sakta hoon.

☐ Main move semantics aur perfect forwarding explain kar sakta hoon.

☐ Main syllabus ke das design patterns ko unki categories ke saath list kar sakta hoon.

☐ Main concurrency vs parallelism, aur process vs thread ka difference explain kar sakta hoon.

☐ Main teeno coroutine keywords ke naam aur unka kaam bata sakta hoon.

Practical

☐ Main memory se ek poora C++ program likh sakta hoon jo `vector` aur ek algorithm use karta ho.

☐ Main demand par Singleton, Factory, Observer aur State skeletons produce kar sakta hoon.

☐ Main mutex-protected shared counter ka working example likh sakta hoon.

☐ Main ek working coroutine likh sakta hoon jo pehle n integers yield karti ho.

Exam-hall Habits

☐ Main likhna shuru karne se pehle har question poora padhunga.

☐ Main pehle aasaan questions attempt karunga, phir mushkil.

☐ Main polymorphism, vtable, smart pointer ya design-pattern questions ke liye ek chhota diagram banaunga.

☐ Main har definition ko full sentence se shuru karunga, fragment se nahi.

☐ Main har design-pattern answer ko ek-sentence advantage aur disadvantage ke saath end karunga.

> **! DO NOT FORGET**
>
> **Do not forget.** Programming paper mein presentation, indentation aur comment quality bhi marks ka hissa hain. Saaf-suthra code communicate karta hai ki aap use samajhte ho.

Ayush Jha

Best of Luck
for your Examinations

Aap is book ke aakhri page tak pahunch gaye hain. Jo ek blank slate ke roop mein shuru hua tha, wo page by page C++ par working command ban gaya hai — object-oriented programming ke chaar pillars se lekar, Standard Template Library ke through, modern C++20 features mein, classical design patterns ke across, aur concurrent programming aur coroutines tak.

Syllabus finite hai; language, aur uske around ka craft, nahi. Is field mein jin bhi professionals ko aap admire karoge, wo kabhi bilkul wahin baithe the jahan aap ab baithe ho — unsure, curious, aage badhte rehne ko tayyar. Aage badhte raho.

Code likho. Doosron ka code padho. Fail fast, fix faster. Aur exam day par saans lo: aapne exhaustively taiyaari ki hai.

> *"Any fool can write code that a computer can understand.*
> *Good programmers write code that humans can understand."*
>
> — Martin Fowler

In notes ke saath padhne ke liye dhanyavaad.
Aapka compiler silent rahe, aapke tests green rahein, aur aapke terminals warm rahein.

— Ayush Jha
Batch 2CSE-4 · School of Computer Science & Engineering
IILM University, Greater Noida

End of Book